

web3 для фронтендера

Методичка для трудоустройства
React + блокчейн + DeFi + собеседование

Составил: Sergei Krk & Hermes AI

Дата сборки: 2026-07-23

Содержание

web3-фронтендер: план трудоустройства

- Кто такой web3-фронтендер
- План подготовки: 3 фазы
- Что спрашивают на собеседовании web3-фронтендера
- Технологический минимум (must-have в резюме)
- Твоё преимущество как фронтендера
- Как показать опыт без коммерческого web3-опыта
- Связанное

Главная: дорожная карта web3

- Зачем web3 фронтендеру
- Что входит в web3 (обзор)
- План изучения (5 этапов)
- Технологический стек (рекомендуемый)
- Первые шаги (сегодня)
- Связанное

Блокчейн — как это работает

- Что такое блокчейн: две фразы
- Как устроен блокчейн
- Ethereum — детальнее
- Практика: первое знакомство с сетью
- Ключевые выводы первого этапа
- Что дальше

Словарь терминов web3

- A
- B
- C
- D
- E
- F
- G
- H

Solidity — основы синтаксиса

- Что такое смарт-контракт
- Структура контракта
- Типы данных
- Видимость функций
- Мутабельность (читать/писать)

- Storage vs Memory — критично для газа
- Глобальные переменные
- Модификаторы (собственные)

ERC-20: стандарт токенов

- Уровень 1. Для пятилетнего ребёнка
- Уровень 2. Для новичка в web3
- Уровень 3. Для разработчика (интерфейс и механика)
- Уровень 4. Для продвинутого разработчика (реализация и нюансы)
- Связанное

ERC-721: NFT стандарт

- Уровень 1. Для пятилетнего ребёнка
- Уровень 2. Для новичка в web3
- Уровень 3. Для разработчика (интерфейс и механика)
- Уровень 4. Для продвинутого разработчика (реализация и нюансы)
- Связанное

Hardhat — среда разработки

- Уровень 1: Для новичка (что это и зачем)
- Уровень 2: Рабочий минимум (каждодневные операции)
- Уровень 3: Понимание (как это устроено внутри)
- Уровень 4: Разбор Counter.sol через Hardhat
- Связанное
- Источники

OpenZeppelin — безопасные контракты

- Уровень 1: Зачем нужен OpenZeppelin
- Уровень 2: Основные модули
- Уровень 3: Продвинутые темы
- Уровень 4: Architectural Decisions
- Быстрый старт (чит-лист)
- Связанные страницы
- Источники

wagmi + RainbowKit — фронтенд для dApps

- Связь wagmi, RainbowKit и viem
- Уровень 1. wagmi: React-хуки
- Уровень 2. RainbowKit: готовый UI для Connect Wallet
- Уровень 3. viem: низкоуровневые операции
- Уровень 4. Практический гайд: от Connect Wallet до вызова смарт-контракта
- Таблица миграции: wagmi v2 → v3
- Best Practices (из опыта)
- Частые ошибки

Сравнение ethers.js, viem, wagmi

- ethers.js v6
- viem
- wagmi
- Как они соотносятся
- Рекомендация для пет-проекта
- Минимальный пример (wagmi + viem)

Subgraph / The Graph — индексация

- Зачем нужен Subgraph (прямой вызов контракта медленный)
- Как работает The Graph
- Архитектура Subgraph
- Создание своего Subgraph для ERC-20
- Запросы из React-приложения
- Goldsky — альтернатива The Graph
- Типичные ошибки
- Когда НЕ нужен Subgraph

Паттерны транзакций в React

- Жизненный цикл транзакции
- useWriteContract — пишем в смарт-контракт
- useSendTransaction — отправка нативного ETH
- useWaitForTransactionReceipt — ожидание квитанции
- Обработка revert: decodeErrorResult
- Gas estimation: почему падает и как чинить
- Замена транзакции: speed up и cancel
- Toast-уведомления: полноценный хук

DeFi для фронтендера

- Uniswap: децентрализованная биржа (DEX)
- Aave: кредитование и займы
- Стейкинг (Staking)
- Layer 2: Arbitrum и Optimism
- MEV: Miner/Maximal Extractable Value
- Что спрашивают на собеседовании по DeFi (для фронтендера)
- Чек-лист: что должен уметь DeFi-фронтендер

Вопросы web3-собеседования

- Блокчейн-основы
- 1. Объясни, как работает блокчейн на примере Ethereum
- 2. Что такое газ (gas)? Почему одна транзакция стоит \$1, а другая \$50?
- 3. Чем отличается EOA от Contract Account?
- 4. Что такое nonce и зачем он нужен?

- 5. Как работает Proof of Stake в Ethereum?
- Смарт-контракты
- 6. Что такое ABI и зачем он нужен фронтендеру?

GitHub Commit Notary (пет-проект)

- Проблема
- Решение
- Как работает
- Ключевые технологии
- Почему это хороший проект для портфолио
- Стек
- Связанное

Proof of Skill (пет-проект)

- Проблема
- Решение
- Как работает
- Структура сертификата
- Почему это интересно
- Потенциал развития
- Стек
- Связанное

Open Source Sponsor Escrow (пет-проект)

- Проблема
- Решение
- Как работает
- Ключевые механики
- Почему это сильный проект для портфолио
- План развития
- Стек
- Связанное

web3-фронтендер: план трудоустройства

Кто такой web3-фронтендер

Это React/TypeScript-разработчик, который пишет интерфейсы к смарт-контрактам. От обычного фронтендера отличается:

- Работает не с REST API, а с блокчейном (через ethers.js / viem / wagmi)
- Понимает, как устроены смарт-контракты (даже если не пишет их с нуля)
- Знает web3-специфичный UX: wallet connect, транзакции, gas, подписи
- Разбирается в DeFi-концепциях (на уровне понимания продукта)

План подготовки: 3 фазы

Фаза 1: База (2-3 недели)

Цель: понимать блокчейн и уметь читать смарт-контракты.

День	Тема	Ресурс
1-2	Как работает блокчейн (блоки, хеши, консенсус)	wiki/Блокчейн-как-это-работает
3-4	Ethereum: аккаунты, газ, nonce, MetaMask, Etherscan	wiki/Блокчейн-как-это-работает + практика
5-7	Solidity: типы, функции, storage/memory, события	wiki/Solidity-основы
8-10	ERC-20: стандарт токенов, approve/transferFrom	wiki/ERC-20-стандарт-токенов
11-13	ERC-721 (NFT): mint, метаданные, Soulbound	wiki/ERC-721-NFT-стандарт
14	Hardhat: развернуть контракт в тестовой сети	wiki/Hardhat-среда-разработки

Чекпоинт: можешь написать простой ERC-20 токен, задеплоить в Sepolia и прочитать его данные через Etherscan.

Фаза 2: Фронтенд dApps (3-4 недели)

Цель: писать React-приложения, подключённые к смарт-контрактам.

День	Тема	Ресурс
1-3	viem/ethers.js: чтение контрактов, отправка транзакций	wiki/Сравнение-ethers-viem-wagmi
4-7	wagmi: React-хуки (useReadContract, useWriteContract)	wiki/wagmi-RainbowKit-фронтенд
8-10	RainbowKit: connect wallet UI, кастомизация	wiki/wagmi-RainbowKit-фронтенд
11-14	Транзакции: pending→confirmed→receipt, revert, gas	wiki/Паттерны-транзакций-React
15-17	События: слушать Transfer/Approval, история операций	—
18-21	Subgraph: индексация ончейн-данных для быстрого чтения	wiki/Subgraph-The-Graph

Чекпоинт: dApp с wallet connect + чтение баланса + отправка токенов + история транзакций.

Фаза 3: DeFi + Собеседование (2-3 недели)

Цель: понимать web3-продукты и пройти собеседование.

День	Тема	Ресурс
1-3	DEX и AMM (Uniswap): как работают свопы	wiki/DeFi-для-фронтера
4-5	Lending (Aave): заём, collateral, ликвидации	wiki/DeFi-для-фронтера
6-7	Стейкинг, yield farming, Layer 2	wiki/DeFi-для-фронтера
8-10	OpenZeppelin: AccessControl, Upgradeable, безопасность	wiki/OpenZeppelin-безопасные-контракты
11-14	Web3-собеседование: вопросы, мок-интервью	wiki/Вопросы-web3-собеседование
15-21	Пет-проект: доделать до состояния «показать на собесе»	wiki/GitHub-Commit-Notary / wiki/Proof-of-Skill

Что спрашивают на собеседовании web3-фронтера

Всегда:

- 1. **«Объясни, как работает блокчейн, на примере Ethereum»** — ждут: блоки, хеши, консенсус, газ, nonce
- 2. **«Как подключить кошелёк к dApp?»** — RainbowKit/WalletConnect, flow: connect → sign → send

Как читать connect → sign → send: «три шага любого dApp: (1) connect — пользователь разрешает сайту видеть адрес кошелька, (2) sign — пользователь подписывает транзакцию в кошельке, (3) send — транзакция улетает в мемпул и майнится». Мнемоника: connect = "кто ты?", sign = "ты точно этого хочешь?", send = "понеслось в блокчейн".

- 1. **«Жизненный цикл транзакции»** — pending → mempool → confirmed → receipt, как обрабатывать в UI

Как читать pending → mempool → confirmed → receipt: «после подписи транзакция висит в мемпуле (очереди), майнер/валидатор включает её в блок (confirmed), и ты получаешь квитанцию (receipt) с точным количеством потраченного газа и статусом success/revert». Мнемоника: мемпул = зал ожидания, блок = поезд ушёл, receipt = билет с печатью «приехали».

- 1. **«Чем отличается ERC-20 от ERC-721?»** — fungible vs non-fungible, интерфейсы
- 2. **«Как читать данные из контракта быстро?»** — Subgraph/The Graph vs прямой вызов

Часто:

1. «**Как работает Uniswap?**» — АММ, пулы ликвидности, $x*y=k$, slippage
2. «**Что такое MEV и фронтраннинг?**» — sandwich attack, как защищаться
3. «**Как хранить офчейн-данные для NFT?**» — IPFS, Arweave, tokenURI
4. «**Какие уязвимости знаешь?**» — reentrancy, integer overflow, frontrunning
5. «**Как дебажить транзакцию?**» — Etherscan, Tenderly, Hardhat console

Технологический минимум (must-have в резюме)

- **React + TypeScript** (твой основной стек)
- **wagmi + viem** или **ethers.js** (подключение к блокчейну)
- **RainbowKit** или **AppKit** (connect wallet)
- **Solidity** на уровне «прочитать контракт, понять ABI, написать простой»
- **Hardhat** (деплой, тесты)
- **The Graph / Subgraph** (индексация — большое преимущество)

Твоё преимущество как фронтендера

В web3 много бэкэнд-разработчиков, которые пишут контракты, но мало хороших фронтендеров. Твой стек (React + TypeScript + архитектура компонентов + тестирование) — это именно то, чего не хватает web3-проектам. Смарт-контракты ты можешь изучать как «новый тип API», не углубляясь в написание сложной логики.

Как показать опыт без коммерческого web3-опыта

1. **Пет-проект с открытым кодом** — GitHub Commit Notary или Proof of Skill
2. **Контрибьюции в open-source** — wagmi, RainbowKit, viem (issues с меткой «good first issue»)
3. **Разбор контракта** — взять популярный контракт (Uniswap V2), разобрать и описать в блоге
4. **Демо-видео** — 2-минутное видео работы твоего dApp

Связанное

- [wiki/Главная](#) — полная дорожная карта web3
- [wiki/Словарь-web3](#) — глоссарий терминов
- [wiki/Блокчейн-как-это-работает](#) — фундамент
- [wiki/wagmi-RainbowKit-фронтенд](#) — главный инструмент

- [wiki/Паттерны-транзакций-React](#) — обработка транзакций
- [wiki/Subgraph-The-Graph](#) — индексация данных
- [wiki/DeFi-для-фронтендера](#) — DeFi-ликбез
- [wiki/Вопросы-web3-собеседование](#) — подготовка к интервью

Главная: дорожная карта web3

Дорожная карта: web3 для frontend-разработчика

Зачем web3 фронтендеру

Web3 — это децентрализованные приложения (dApps), где бэкэнд заменён смарт-контрактами в блокчейне. Для фронтендера это означает:

- **Новый тип бэкенда** — не REST API, а вызовы смарт-контрактов через ethers.js / wagmi
- **Новый тип состояния** — не Redux, а блокчейн (ончейн) + локальное (оффчейн)
- **Новые UX-паттерны** — подтверждение транзакций, ожидание блоков, gas fees, wallet connect
- **Новый тип пользователя** — с криптокошельком (MetaMask, WalletConnect) вместо логина/пароля

Что входит в web3 (обзор)

```
graph TD
    A[Web3] --> B[Блокчейны]
    A --> C[Смарт-контракты]
    A --> D[Фронтенд dApps]
    A --> E[Инфраструктура]

    B --> B1[Ethereum/EVM]
    B --> B2[L2: Arbitrum, Optimism, Base]
    B --> B3[Solana non-EVM]

    C --> C1[Solidity]
    C --> C2[Hardhat / Foundry]
    C --> C3[OpenZeppelin]

    D --> D1[ethers.js / viem / wagmi]
    D --> D2[React + Wallet Connect]
    D --> D3[Subgraph / индексация]

    E --> E1[Infura / Alchemy]
    E --> E2[IPFS / Arweave]
    E --> E3[The Graph]
```

План изучения (5 этапов)

Этап 1. Блокчейн-основы (1-2 недели)

Цель: понимать, как работает блокчейн на концептуальном уровне.

Что изучить:

- [] Как устроен блокчейн: блоки, хеши, консенсус (PoW vs PoS)
- [] Ethereum: аккаунты (EOA vs контракт), транзакции, gas, nonce
- [] Кошельки: MetaMask, приватные ключи, seed-фразы, публичный адрес
- [] Транзакции: что происходит при отправке ETH / вызове контракта
- [] Etherscan: как читать транзакции и контракты

Практика:

- [] Установить MetaMask, получить тестовый ETH из крана (Sepolia faucet)
- [] Отправить тестовую транзакцию, посмотреть её на Etherscan

Этап 2. Смарт-контракты (Solidity) (2-3 недели)

Цель: написать и задеплоить простой смарт-контракт.

Что изучить:

- [] Solidity basics: типы данных, функции, модификаторы, события
- [] Хранение: storage vs memory vs calldata
- [] Маппинги, массивы, структуры
- [] Payable функции, transfer, send
- [] ERC-20 (токены) — стандарт, как устроен
- [] ERC-721 (NFT) — стандарт, mint, метаданные
- [] Hardhat — среда разработки: компиляция, тесты, деплой
- [] OpenZeppelin — библиотека проверенных контрактов

Практика:

- [] Написать простой контракт-счётчик (increment/decrement + события)
- [] Написать и задеплоить ERC-20 токен в тестовую сеть
- [] Написать тесты на Hardhat (Chai + waffle)

Этап 3. Фронтенд dApps (2-4 недели)

Цель: подключить React-приложение к смарт-контракту.

Что изучить:

- [] ethers.js v6 / viem — чтение и отправка транзакций
- [] wagmi — React-хуки для web3 (useAccount, useContractRead, useContractWrite)
- [] WalletConnect / RainbowKit — connect wallet UI
- [] Обработка транзакций: pending → confirmed → receipt

- [] Чтение событий (events/logs) из контракта
- [] Subgraph (The Graph) — индексация ончейн-данных для быстрого чтения

Практика:

- [] Простое dApp: connect wallet + прочитать баланс + отправить транзакцию
- [] dApp к своему токену: mint, transfer, просмотр баланса
- [] Добавить события: слушать Transfer, показывать историю

Этап 4. DeFi и продвинутые темы (2-4 недели)

Цель: понимать, как устроены реальные web3-продукты.

- [] DEX (Uniswap): AMM, пулы ликвидности, свопы
- [] Lending (Aave): заём и кредитование, collateral
- [] Стейкинг, yield farming
- [] Layer 2: Arbitrum, Optimism, Base — зачем и как работают
- [] MEV, фронترаннинг, безопасность
- [] IPFS — децентрализованное хранение (для метаданных NFT)

Этап 5. Пет-проект

Цель: собрать полноценное dApp от контракта до фронтенда.

Идеи (ранжированы по сложности и охвату технологий):

- **wiki/GitHub-Commit-Notary** — нотаризация коммитов (сложность: ***) — криптография, хеши, Ethereum calldata, CLI
- **wiki/Proof-of-Skill** — децентрализованные сертификаты (сложность: ***) — DID, Soulbound NFT, IPFS, EIP-712 подписи
- **wiki/Open-Source-Sponsor-Escrow** — эскроу для опенсорса (сложность: ***) — смарт-контракты, оракулы, GitHub API, безопасность

Рекомендуемый порядок

1. **GitHub Commit Notary** — простой старт: контракт не нужен, учит работе с хешами и транзакциями
2. **Proof of Skill** — средняя сложность: ERC-721, цифровые подписи, DID, IPFS
3. **Open Source Sponsor Escrow** — сильный проект для портфолио: смарт-контракты, безопасность, интеграция с GitHub, реальные деньги

Каждый следующий проект опирается на знания из предыдущего. Вместе они охватывают три ключевые области Web3: неизменяемые доказательства, цифровую идентичность и автоматизацию доверия через смарт-контракты.

Технологический стек (рекомендуемый)

- **Блокчейн:** Ethereum (Sepolia testnet), потом Base/Arbitrum — самая большая экосистема
- **Смарт-контракты:** Solidity + Hardhat + OpenZeppelin — индустриальный стандарт
- **Фронтенд:** React + TypeScript + wagmi + viem — привычный стек + web3
- **Wallet:** RainbowKit / AppKit — готовый UI для connect wallet
- **RPC-провайдер:** Alchemy / Infura — доступ к блокчейну без своей ноды
- **Индексация:** The Graph (Subgraph) — быстрое чтение ончейн-данных

Первые шаги (сегодня)

1. Установить MetaMask, переключиться на Sepolia testnet
2. Получить тестовый ETH: <https://sepoliafaucet.com>
3. Прочитать первый концепт: wiki/Блокчейн-как-это-работает

Связанное

- wiki/Словарь-web3|Словарь терминов web3
- wiki/Сравнение-ethers-viem-wagmi|Сравнение ethers.js, viem, wagmi
- wiki/ERC-20-стандарт-токенов — стандарт токенов
- wiki/ERC-721-NFT-стандарт — NFT стандарт
- wiki/Solidity-основы — синтаксис Solidity
- wiki/Hardhat-среда-разработки — среда разработки
- wiki/OpenZeppelin-безопасные-контракты — безопасные контракты
- wiki/wagmi-RainbowKit-фронтенд — React-фронтенд для dApps

Блокчейн — как это работает

Блокчейн — как это работает

Объяснение для frontend-разработчика: без математики, на знакомых аналогиях.

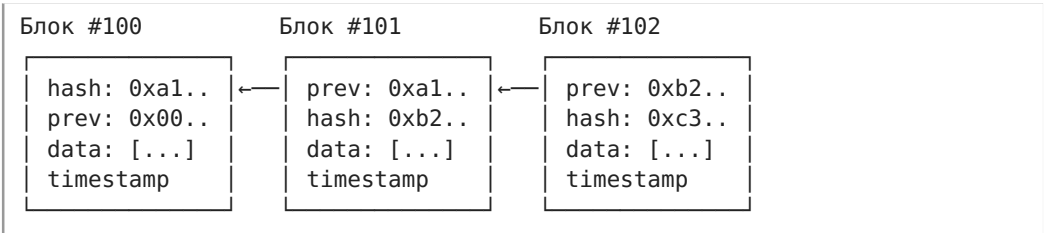
Что такое блокчейн: две фразы

Блокчейн — это база данных, которую никто не может подделать. Данные хранятся не на одном сервере, а на тысячах компьютеров одновременно. Каждый новый «файл» (блок) ссылается на предыдущий — как git, но без force push.

Аналогия для фронтендера: представь Redux-стор, который хранится не в браузере, а в облаке. Каждый экшен навсегда остаётся в истории. Нельзя `reset()`, нельзя `REVERT_STATE`. Только `dispatch()`.

Как устроен блокчейн

Блоки



Каждый блок содержит:

- **Хеш** этого блока (уникальный отпечаток)
- **Хеш предыдущего блока** (связь в цепочку)
- **Данные** (транзакции)
- **Timestamp** — когда создан

Если изменить хоть байт в блоке #100, его хеш поменяется → блок #101 укажет на несуществующий хеш → цепочка порвётся.

Обнаруживается мгновенно.

Хеш (криптографический отпечаток)

Хеш-функция берёт любые данные и выдаёт строку фиксированной длины:

```
sha256("hello") →  
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824  
sha256("Hello") →  
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Видишь: «h» → «H», а хеш другой полностью. Это свойство — лавинный эффект — делает блокчейн защищённым от подделок.

Как читать `sha256("hello") → 2cf24dba...`: читай как «засунь любые данные в криптографическую мясорубку — получишь 64-символьный шестнадцатеричный отпечаток, который невозможно прокрутить назад». Мнемоника: хеш — это как отпечаток пальца для данных: одинаковые данные → одинаковый отпечаток, но по отпечатку не восстановишь лицо.

Консенсус: как все договариваются

Тысячи компьютеров (нод) хранят копию блокчейна. Как они договариваются, какая версия правильная?

Proof of Work (PoW, Bitcoin): реши математическую задачу → получи право создать блок. Тратит электричество.

Proof of Stake (PoS, Ethereum сейчас): положи ETH в залог → получи право создавать блоки. Если обманешь → потеряешь залог. Экологично, быстро.

PoS валидация блока:

1. Пользователи отправляют транзакции

2. Валидатор (случайно выбранный из залогов) собирает их в блок

3. Остальные валидаторы проверяют блок

4. Если всё ок → блок добавляется, валидатор получает награду

Ethereum — детальнее

Ethereum — это не просто «биткоин с программами». Это **мировой компьютер**: тысячи нод выполняют один и тот же код и приходят к одному результату.

Два типа аккаунтов

Тип	Кто контролирует	Пример
EOA (Externally Owned Account)	Приватный ключ	Твой кошелёк MetaMask
Contract Account	Код смарт-контракта	Контракт токена USDC

Важно: **только EOA может инициировать транзакцию**. Контракт может выполнять код только когда его дёрнули извне.

Транзакция — что внутри

```
{
  "from": "0xТвойАдрес",           // отправитель
  "to": "0xАдресКонтракта",        // получатель (или 0x0 для деплоя)
  "value": "1000000000000000000", // 1 ETH в wei
  "data": "0xa9059cbb...",         // закодированный вызов функции
  "gasLimit": "21000",              // макс. gas на исполнение
  "maxFeePerGas": "50000000000",   // макс. цена за gas (50 gwei)
  "nonce": 5                        // порядковый номер
}
```

Gas — это плата за вычисления. Каждая операция стоит определённое количество gas. Сложил два числа — 3 gas. Записал в storage — 20000 gas.

Как читать JSON транзакции: читай "value": "1000000000000000000" как «1 ETH, записанный в wei — это как 100 копеек, записанные в копейках, а не в рублях». А "data": "0xa9059cbb..." читай как «закодированный вызов функции контракта: первые 4 байта — селектор функции, остальное — аргументы, упакованные по

правилам ABI». Мнемоника: всё в блокчейне — целые числа, а *data* — это просто «позвони контракту и скажи вот это».

Зачем gas? Чтобы никто не заспамил сеть бесконечным циклом. Gas кончится → транзакция откатится.

EIP-1559 (текущий механизм gas):

- *maxFeePerGas* — максимум, что ты готов заплатить за единицу gas
- *maxPriorityFeePerGas* — чаевые валидатору (чем выше, тем быстрее включают)
- Неиспользованный gas возвращается

Как читать EIP-1559: читай *maxFeePerGas* как «потолок цены: больше этой суммы за единицу газа с меня не снимут», а *maxPriorityFeePerGas* как «чаевые: вот сколько сверху даю валидатору, чтобы взяли мой перевод побыстрее». Мнемоника: это как такси — *maxFeePerGas* = максимальная цена поездки, *maxPriorityFeePerGas* = чаевые водителю за скорость.

Nonce — защита от повторной отправки

Каждый твой аккаунт имеет счётчик nonce. Отправил транзакцию → nonce++. Следующая транзакция должна иметь nonce на 1 больше.

Зачем: предотвращает повторную отправку одной и той же транзакции. Даже если злоумышленник перехватит подписанную транзакцию и попытается отправить её снова — nonce уже не совпадёт, транзакцию отвергнут.

Практика: первое знакомство с сетью

1. Установи MetaMask

Расширение для Chrome/Brave: <https://metamask.io>

При создании кошелька тебе дадут 12 слов (seed-фраза). Это твой мастер-ключ.

Запиши на бумаге. Не в облако. Не в заметки. Если потеряешь — потеряешь всё.

2. Переключись на тестовую сеть

В MetaMask: нажми на сеть (сверху) → «Показать тестовые сети» → выбери **Sepolia**.

Тестовые сети — это полные копии Ethereum, где ETH ничего не стоит. Можно ломать, терять, экспериментировать.

3. Получи тестовый ETH

1. Иди на <https://sepoliafaucet.com> (нужен аккаунт Alchemy, бесплатно)

2. Или <https://cloud.google.com/application/web3/faucet/ethereum/sepolia>

3. Вставь свой адрес (0x...) → получи 0.5 SepoliaETH

Адрес можно скопировать в MetaMask — клик на название аккаунта.

4. Отправь первую транзакцию

В MetaMask: «Отправить» → любой тестовый адрес (хоть свой же) → 0.01 ETH → подтвердить.

5. Посмотри на Etherscan

Etherscan — это «браузер блокчейна». Как DevTools для сети.

- Найди свою транзакцию на <https://sepolia.etherscan.io> (вставь свой адрес в поиск)
- Что ты увидишь: статус (Success), хеш транзакции, блок, gas, from, to

Поздравляю — ты только что написал в мировую базу данных, которая живёт на тысячах компьютеров.

Ключевые выводы первого этапа

1. Блокчейн = распределённая неизменяемая БД
2. Каждый блок ссылается на предыдущий через хеш → цепочка
3. Ethereum = «мировой компьютер», выполняющий код на всех нодах
4. Газ = плата за вычисления, защита от спама
5. MetaMask + Sepolia + Etherscan = твой инструментарий разработчика

Что дальше

- [wiki/Смарт-контракты-введение](#)|Смарт-контракты — введение — что такое смарт-контракт и как он работает
- [wiki/Словарь-web3](#)|Словарь терминов — если встретил незнакомое слово

Словарь терминов web3

Словарь терминов web3

А

ABI (Application Binary Interface) — JSON-описание интерфейса смарт-контракта: какие функции есть, какие параметры принимают, что возвращают. Нужен фронтенду, чтобы знать, как вызывать контракт.

Account (аккаунт) — в Ethereum два типа:

- **EOA (Externally Owned Account)** — обычный кошелёк пользователя, контролируется приватным ключом
- **Contract Account** — смарт-контракт, контролируется кодом

AMM (Automated Market Maker) — математическая модель обмена токенов без книги ордеров. $x \times y = k$. Используется в Uniswap.

В

Block — набор транзакций, подтверждённых сетью. Каждый блок ссылается на предыдущий (цепочка).

Blockchain — распределённый реестр, где данные хранятся в цепочке блоков. Нельзя изменить прошлое, не пересчитав всю цепочку.

С

Consensus (консенсус) — механизм, которым сеть договаривается о едином состоянии. Ethereum: PoS (Proof of Stake). Раньше: PoW (Proof of Work, как у Bitcoin).

Д

dApp (Decentralized Application) — приложение, чья логика выполняется в смарт-контрактах, а не на сервере.

DeFi (Decentralized Finance) — финансовые сервисы на блокчейне: обмен, кредиты, стейкинг.

Deploy (деплой) — публикация смарт-контракта в блокчейн. После деплоя контракт получает адрес и его код нельзя изменить.

Е

EOA — см. Account.

ERC-20 — стандарт взаимозаменяемых токенов (fungible tokens). Все токены одного контракта одинаковы (как доллары).

ERC-721 — стандарт невзаимозаменяемых токенов (NFT). Каждый токен уникален.

EVM (Ethereum Virtual Machine) — «компьютер», выполняющий код смарт-контрактов. Работает на всех нодах Ethereum.

Event (событие) — лог, который смарт-контракт записывает в блокчейн. Дешевле хранения, но нельзя прочитать из самого контракта (только извне).

Ф

Faucet (кран) — сервис, выдающий тестовые токены (ETH) для разработки в тестовых сетях.

G

Gas — плата за вычисления в Ethereum. Измеряется в gwei (1 gwei = 0.000000001 ETH). Каждая операция стоит определённое количество gas.

Gas limit — максимальное количество gas, которое ты готов потратить на транзакцию.

Gas price — цена за единицу gas. Чем выше, тем быстрее майнеры включают транзакцию в блок.

H

Hash (хеш) — односторонняя криптографическая функция.

Одинаковый ввод → одинаковый хеш. По хешу нельзя восстановить исходные данные.

L

Layer 1 (L1) — основной блокчейн (Ethereum, Solana).

Layer 2 (L2) — решение поверх L1 для ускорения и удешевления транзакций (Arbitrum, Optimism, Base).

M

Mempool — «очередь» неподтверждённых транзакций, ожидающих включения в блок.

MetaMask — самый популярный браузерный криптокошелёк.

Mint — создание нового токена (обычно NFT).

N

Node (нода) — компьютер, участвующий в сети блокчейна. Хранит копию блокчейна и проверяет транзакции.

Nonce — порядковый номер транзакции от конкретного адреса. Предотвращает повторную отправку.

P

Private key (приватный ключ) — секретный ключ, дающий полный контроль над кошельком. Никому не показывать!

Public key / address (адрес) — публичный идентификатор кошелька (0x...). Можно показывать всем.

PoS (Proof of Stake) — механизм консенсуса: валидаторы ставят ETH как залог. Если обманут — теряют залог.

PoW (Proof of Work) — механизм консенсуса: майнеры решают сложные математические задачи. Энергозатратно (Bitcoin).

S

Seed phrase (сид-фраза) — 12/24 слов, из которых генерируются все приватные ключи кошелька. Хранить офлайн!

Smart contract (смарт-контракт) — программа, выполняющаяся на блокчейне. Код + состояние. После деплоя код неизменяем.

Solidity — основной язык программирования для Ethereum-смарт-контрактов. Похож на JavaScript.

Stablecoin (стейблкоин) — токен, привязанный к фиатной валюте (USDT, USDC = 1 USD).

T

Testnet (тестовая сеть) — копия Ethereum для разработки. Токены бесплатные, ошибки не страшны. Sepolia — текущая основная тестовая сеть.

Transaction (транзакция) — операция в блокчейне: перевод ETH, вызов функции контракта, деплой контракта.

W

Wallet (кошелёк) — программа/расширение для управления криптоактивами. Хранит приватные ключи и подписывает транзакции.

Wei — мельчайшая единица ETH. $1 \text{ ETH} = 10^{18} \text{ wei}$.

Как читать $1 \text{ ETH} = 10^{18} \text{ wei}$: читай как «в одном эфире 1 000 000 000 000 000 000 (квинтиллион) неделимых частиц-вей». Мнемоника: wei для ETH — как копейка для рубля, только копеек не 100, а квинтиллион, и дробных чисел в блокчейне нет, поэтому все расчёты в этих «копейках».

Solidity — основы синтаксиса

Solidity — основы синтаксиса

Что такое смарт-контракт

Код, который живёт в блокчейне по адресу (как аккаунт). Любой может его вызвать — он выполняется на всех нодах сети. Код неизменяем после деплоя, состояние хранится в блокчейне.

Контракт = класс в ООП. Имеет: состояние (storage), функции (методы), события (emit/logs).

Структура контракта

```
// SPDX-License-Identifier: MIT      ← обязательная лицензия
pragma solidity ^0.8.20;              ← версия компилятора

contract Имя {
    // 1. Переменные состояния (storage)
    // 2. События (events)
    // 3. Модификаторы (modifiers)
    // 4. Конструктор (constructor)
    // 5. Функции
}
```

Типы данных

Примитивы

Тип	Размер	Пример
uint256	0..2 ²⁵⁶ -1	uint256 count = 0;
int256	знаковое	int256 x = -5;
uint8	0..255	экономит газ в массивах
address	20 байт	address owner = 0x...;
address payable	20 байт + transfer	можно отправлять ETH
bool	true/false	bool active = true;
bytes32	32 байта фикс.	bytes32 hash = 0x...;
string	динамический	дорого, избегать где можно

Составные

```
// Мэппинг – аналог объекта/Map. НЕЛЬЗЯ итерировать
mapping(address => uint256) public balances;

> **Как читать `mapping(address => uint256) public balances`:** читай
как «словарь: набираешь адрес – получаешь число, как
`balances[0xABCD...]` вернёт баланс этого адреса». Мнемоника: mapping в S
olidity – это `Map<KeyType, ValueType>` из TypeScript, но без `.keys()`,
без `.values()`, без возможности перебора; только «дай значение по
ключу».

// Массив
uint256[] public dynamicArray;           // динамический
address[10] public fixedArray;           // фиксированный размер

// Структура
struct User {
    address addr;
    uint256 balance;
    bool active;
}
```

Видимость функций

Модификатор	Снаружи	Внутри контракта	В наследниках
public	да	да	да
private	нет	да	нет
internal	нет	да	да
external	да	нет (только this.f())	да

Мутабельность (читать/писать)

Ключ	Меняет состояние	Газ (внешний вызов)
(нет)	да	да
view	нет, только читает	нет
pure	нет, даже не читает	нет
payable	да + принимает ETH	да

```
function getBalance() public view returns (uint256) {
    return address(this).balance;    // читает, газ не тратит
}

function add(uint a, uint b) public pure returns (uint) {
    return a + b;                    // не трогает состояние вообще
}

function deposit() public payable {
    // msg.value – присланные ETH
}
```

Storage vs Memory — критично для газа

	Storage	Memory
Где	Блокчейн (диск)	Оперативная память
Живёт	Навсегда	Только на время вызова
Стоимость	ОЧЕНЬ дорого	Дёшево
По умолчанию	Переменные состояния	Локальные переменные

```
uint256[] public stored;    // storage (неявно)

function example() public {
    uint256[] memory temp = new uint256[](10); // memory (явно)
    temp[0] = 1;
    stored = temp;           // копия storage ← memory
    (дорого!)
}
```

Глобальные переменные

```
msg.sender    // address — кто вызвал функцию
msg.value     // uint256 — сколько ETH прислали
block.timestamp // uint256 — время блока (не точное!)
tx.gasprice   // цена газа
address(this) // адрес самого контракта
```

Модификаторы (собственные)

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _; // ← здесь выполняется тело функции
}
```

> ****Как читать `modifier onlyOwner() { require(...); \_; }`:** читай как «перед тем как выполнить тело функции, проверь условие; `\_` — это место, куда подставится код функции». Мнемоника: модификатор — это как `middleware` в Express.js или `beforeEach` в тестах: сначала проверка, потом `\_` = `next()`.

```
function reset() public onlyOwner {
    count = 0;
}
```

События (Events)

Пишутся в логи блокчейна. Нечитаемы из контракта (только извне через web3-библиотеки).

Дёшево (~375 gas vs ~20 000 за storage-запись). Основной способ коммуникации контракт → фронтенд.

```
event Transfer(address indexed from, address indexed to, uint256 value);

function transfer(address to) public {
    emit Transfer(msg.sender, to, amount);
}
```

indexed — по этим полям можно фильтровать события на фронтенде (до 3 полей).

Как читать *event Transfer(address indexed from, address indexed to, uint256 value)*: читай как «объявляю лог-запись с тремя полями; *indexed* значит "по этому полю можно искать и фильтровать снаружи", а *value* без *indexed* — лежит в данных события, но фильтровать по нему нельзя». Мнемоника: *indexed* = «индексируемое поле» как *WHERE* в SQL, а не-*indexed* = данные, которые просто лежат в теле события.

Контракт-счётчик (итоговый пример)

См. `raw/Counter.sol` — полный код:

- Состояние: `count (uint256)`, `owner (address)`
- События: `Incremented`, `Decrement`, `Reset`
- Модификатор: `onlyOwner`
- Функции: `increment()`, `decrement()`, `reset()`, `getCount()`

Связанное

- [wiki/Блокчейн-как-это-работает](#) — база по блокчейну
- [wiki/Словарь-web3](#) — термины (gas, storage, event, ABI)
- [wiki/Сравнение-ethers-viem-wagmi](#) — как вызывать контракты с фронтенда

ERC-20: стандарт токенов

ERC-20 — стандарт токенов

Уровень 1. Для пятилетнего ребёнка

Представь, что у тебя есть игровые жетоны в парке аттракционов. Все жетоны одинаковые — один жетон ничем не отличается от другого. Ты можешь:

- Посмотреть, сколько у тебя жетонов
- Передать жетоны другу
- Узнать, сколько всего жетонов существует в парке

ERC-20 — это правила игры для таких жетонов, но в интернете.

Благодаря этим правилам все программы (кошельки, биржи, игры) понимают друг друга, потому что «говорят на одном языке». Если токен следует правилам ERC-20 — любой кошелёк сразу знает, как с ним работать.

Уровень 2. Для новичка в web3

ERC-20 — это **технический стандарт взаимозаменяемых токенов** (fungible tokens) в Ethereum и совместимых блокчейнах. Стандарт описан в документе **EIP-20** (Ethereum Improvement Proposal), предложенном Fabian Vogelsteller в ноябре 2015 года.

Что такое взаимозаменяемый токен?

Взаимозаменяемый (fungible) — значит, что один токен абсолютно идентичен другому. Как доллар: одна купюра в \$1 равна любой другой купюре в \$1. В отличие от **ERC-721 (NFT)** — невзаимозаменяемых токенов, где каждый уникален (как билет на концерт с конкретным местом).

Зачем нужен стандарт?

До ERC-20 каждый разработчик придумывал свой интерфейс токена. Кошельки и биржи не могли работать с новыми токенами без индивидуальной интеграции. Стандарт решил эту проблему: **любой**

контракт, реализующий ERC-20, автоматически поддерживается всеми кошельками, биржами и dApps.

Где используются ERC-20 токены?

- **Стейблкоины** — USDT, USDC, DAI (привязаны к курсу доллара)
 - **Governance-токены** — UNI (Uniswap), AAVE (Aave), MKR (MakerDAO) — дают право голоса
 - **Токены проектов** — LINK (Chainlink), MATIC (Polygon)
 - **Wrapped ETH (WETH)** — «обёрнутый» ETH в формате ERC-20 для совместимости с DeFi
-

Уровень 3. Для разработчика (интерфейс и механика)

Интерфейс ERC-20 (EIP-20)

Смарт-контракт считается ERC-20, если реализует все перечисленные методы и события:

Обязательные методы

```
// Имя токена (человекочитаемое)
function name() public view returns (string)
// Пример: "USD Coin"

// Символ (тикер)
function symbol() public view returns (string)
// Пример: "USDC"

// Количество знаков после запятой
function decimals() public view returns (uint8)
// Пример: 6 (значит 1 USDC = 1_000_000 минимальных единиц)

// Общее количество токенов
function totalSupply() public view returns (uint256)

// Баланс конкретного адреса
function balanceOf(address _owner) public view returns (uint256)

// Перевод токенов отправителем транзакции
function transfer(address _to, uint256 _value) public returns (bool)

// Одобрение списания токенов третьим лицом
function approve(address _spender, uint256 _value) public returns (bool)

// Перевод токенов от имени другого адреса (требуется approve)
function transferFrom(address _from, address _to, uint256 _value) public
returns (bool)

// Сколько spender может потратить с адреса _owner
function allowance(address _owner, address _spender) public view returns
(uint256)
```

Обязательные события

```
// Эмитится при любом переводе токенов (включая mint и burn)
event Transfer(address indexed _from, address indexed _to, uint256
_value)
// Особый случай: mint — _from = address(0), burn — _to = address(0)

// Эмитится при вызове approve()
event Approval(address indexed _owner, address indexed _spender, uint256
_value)
```

indexed позволяет фильтровать события по этим полям при чтении логов (до 3 indexed-параметров на событие).

Как работает approve + transferFrom

Это **двухшаговая модель делегирования** — один из ключевых паттернов ERC-20:

1. **Шаг 1** — `approve(spender, amount)`: Алиса разрешает контракту-бирже списать до 1000 токенов с её адреса.
2. **Шаг 2** — `transferFrom(alice, bob, amount)`: Биржа вызывает `transferFrom`, переводя токены от имени Алисы Бобу.

Алиса --`approve`(биржа, 1000)--> ERC-20 контракт

|
Биржа --`transferFrom`(Алиса, Боб, 500)--> ERC-20 контракт

Важно: `transfer` переводит токены от `msg.sender`, а `transferFrom` — от любого адреса, который предварительно сделал `approve`.

Как читать связку `approve` + `transferFrom`: читай `approve` как «я, Алиса, говорю контракту: разрешаю вот этому адресу-бирже снять с моего баланса до N токенов», а `transferFrom` как «биржевая логика говорит контракту: переведи токены от Алисы к Бобу, используя выданное мне разрешение». Мнемоника: *approve* = подписать доверенность, *transferFrom* = предъявить доверенность и совершить перевод.

Decimals — почему баланс не в токенах

ERC-20 хранит балансы в **минимальных неделимых единицах** (как `wei` для ETH). Параметр `decimals` указывает, на сколько знаков нужно сдвинуть запятую:

`decimals = 6` → `balanceOf = 1_000_000` → реально 1.000000 токенов
`decimals = 18` → `balanceOf = 1_000_000_000_000_000_000_000` → реально 1.0 токен

Solidity **не поддерживает дробные числа**, поэтому все расчёты в целых числах, а `decimals` — чисто информационное поле для фронтенда. Большинство токенов используют `decimals = 18` (аналогично ETH).

Как читать `totalSupply = _initialSupply * 10**decimals`: читай как «запиши начальную эмиссию в минимальных единицах: если хочу 1000 токенов с 18 знаками после запятой, умножь 1000 на 10^{18} ». Мнемоника: *decimal = 18* значит «храни как `wei`, показывай как ETH» — баланс `1000000000000000000` в контракте = 1.0 токенов в интерфейсе.

Уровень 4. Для продвинутого разработчика (реализация и нюансы)

Минимальная реализация ERC-20 на Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract MyToken {
    string public name = "My Token";
    string public symbol = "MTK";
    uint8 public decimals = 18;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    > **Как читать `mapping(address => mapping(address => uint256)) public allowance`: ** читай как «двухэтажный словарь: `allowance[владелец][кому_разрешил]` возвращает сколько токенов разрешено потратить». Мнемоника: это как `Map<Address, Map<Address, BigInt>>` – таблица разрешений «кто → кому → сколько».

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(uint256 _initialSupply) {
        totalSupply = _initialSupply * 10**decimals;
        balanceOf[msg.sender] = totalSupply;
        emit Transfer(address(0), msg.sender, totalSupply);
    }

    function transfer(address _to, uint256 _value) public returns (bool)
    {
        require(balanceOf[msg.sender] >= _value, "Insufficient balance");
        balanceOf[msg.sender] -= _value;
        balanceOf[_to] += _value;
        emit Transfer(msg.sender, _to, _value);
        return true;
    }

    function approve(address _spender, uint256 _value) public returns (bool) {
        allowance[msg.sender][_spender] = _value;
        emit Approval(msg.sender, _spender, _value);
        return true;
    }

    function transferFrom(address _from, address _to, uint256 _value) public returns (bool) {
        require(balanceOf[_from] >= _value, "Insufficient balance");
        require(allowance[_from][msg.sender] >= _value, "Insufficient allowance");
```

```

        balanceOf[_from] -= _value;
        balanceOf[_to] += _value;
        allowance[_from][msg.sender] -= _value;
        emit Transfer(_from, _to, _value);
        return true;
    }
}

```

OpenZeppelin — промышленная реализация

В реальных проектах **никто не пишет ERC-20 с нуля** — используют проверенную библиотеку OpenZeppelin:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {
    constructor(uint256 initialSupply) ERC20("My Token", "MTK") {
        _mint(msg.sender, initialSupply * 10**decimals());
    }
}

```

Что даёт OpenZeppelin сверх базового стандарта:

- Защита от целочисленного переполнения (overflow/underflow) — встроено в Solidity 0.8+
- `_mint()` и `_burn()` — внутренние методы для создания и уничтожения токенов
- `increaseAllowance / decreaseAllowance` — безопасная работа с `approve` (защита от race condition)
- `permit()` (в `ERC20Permit`) — `gasless-approve` через EIP-2612 подписи
- Расширения: `ERC20Burnable`, `ERC20Capped`, `ERC20Pausable`, `ERC20Votes`

Известные проблемы и уязвимости

1. Проблема «потерянных токенов» (reception issue)

Если отправить ERC-20 токены на адрес смарт-контракта, который не умеет с ними работать — токены **навсегда потеряны**. Контракт-получатель не получает уведомления о входящем переводе. По состоянию на 2024 год таким образом потеряно более **\$83 млн**.

Способы защиты:

- Запретить `transfer` на адрес самого токена: `require(_to != address(this))`
- Использовать `approve + transferFrom` вместо `transfer` для депозитов в контракты

- Рассмотреть альтернативные стандарты: ERC-223, ERC-1363, ERC-777
- Всегда предусматривать функцию экстренного вывода (emergency withdraw) в контрактах-получателях

2. Race condition в approve

Классическая атака: Алиса одобрила 100 токенов для Боба, потом хочет изменить на 50, но Боб успевает снять 100 до того как транзакция Алисы попадёт в блок.

Решение: использовать `increaseAllowance` / `decreaseAllowance` из OpenZeppelin, которые изменяют `allowance` атомарно относительно текущего значения.

3. Отсутствие обработки возвращаемого bool

Некоторые токены (особенно USDT) не возвращают `bool` из `transfer` и `transferFrom`, хотя стандарт требует. OpenZeppelin использует `safeTransfer` из SafeERC20 для совместимости.

Gas-оптимизации

Как читать gas-оптимизацию `uint256 fromBalance = balanceOf[msg.sender]`: читай как «вместо того чтобы дважды читать из дорогого блокчейн-хранилища (SLOAD), прочитай один раз в локальную переменную памяти и работай с ней». Мнемоника: `storage-чтение (SLOAD)` — это как запрос к удалённой базе, а `memory-переменная` — как кеш в оперативке; всегда кешируй `storage-переменные` перед использованием в функции.

```
// . Дорого: двойное чтение из storage
function transfer(address to, uint256 value) public returns (bool) {
    require(balanceOf[msg.sender] >= value); // SLOAD 1
    balanceOf[msg.sender] -= value;           // SSTORE
    balanceOf[to] += value;                   // SLOAD + SSTORE
}

// Дёшево: кеширование в memory
function transfer(address to, uint256 value) public returns (bool) {
    uint256 fromBalance = balanceOf[msg.sender]; // SLOAD (1 раз)
    require(fromBalance >= value);
    balanceOf[msg.sender] = fromBalance - value;
    balanceOf[to] += value;
}
```

ERC-20 на практике: ethers.js / viem

```
// Чтение баланса (бесплатно, view-функция)
const balance = await contract.balanceOf(address);

// Чтение с учётом decimals (формат для пользователя)
const raw = await contract.balanceOf(address);
const decimals = await contract.decimals();
const formatted = Number(raw) / 10 ** Number(decimals);

// Отправка токенов (транзакция, стоит gas)
const tx = await contract.transfer(to, amount);
await tx.wait(); // ждём подтверждения

// Подписка на события Transfer в реальном времени
contract.on("Transfer", (from, to, value) => {
  console.log(`${from} → ${to}: ${value}`);
});
```

Альтернативные стандарты

Стандарт	Отличие от ERC-20
ERC-223	Уведомляет контракт-получатель через tokenFallback, решает проблему потерянных токенов
ERC-777	Хуки tokensReceived + операторы по умолчанию, более гибкий, но сложнее (есть уязвимости)
ERC-1363	transferAndCall — перевод + вызов функции получателя в одной транзакции
ERC-4626	Стандарт для токенизированных хранилищ (vaults) — надстройка над ERC-20
ERC-2612	permit() — approve через подпись, без отдельной транзакции (gasless)

Связанное

- [wiki/Solidity-основы](#) — синтаксис Solidity, необходимый для написания ERC-20
- [wiki/Словарь-web3](#) — термины: event, gas, ABI, storage, mapping
- [wiki/Главная](#) — дорожная карта изучения web3, этап 2 (смарт-контракты)
- [wiki/Блокчейн-как-это-работает](#) — база по блокчейну: аккаунты, транзакции, gas

ERC-721 — NFT стандарт

Уровень 1. Для пятилетнего ребёнка

Представь, что у тебя есть коллекция уникальных игрушек. У каждой игрушки есть свой номер и паспорт:

- У робота — паспорт №1, цвет синий
- У динозавра — паспорт №2, цвет зелёный
- У куклы — паспорт №3, платье красное

Эти игрушки нельзя просто так обменять одну на другую — они **разные**, у каждой своя ценность. В отличие от игровых жетонов из парка аттракционов, где все жетоны одинаковые.

ERC-721 — это правила игры для таких уникальных игрушек, но в интернете. Ты можешь:

- Узнать, какие игрушки у тебя есть (по номерам)
- Подарить игрушку другу (она переедет в его коллекцию)
- Посмотреть паспорт игрушки (картинку и описание)
- Назначить друга смотрителем, который может передать игрушку кому-то ещё

Самые известные «игрушки» по этим правилам: коллекционные картинки (CryptoPunks, Bored Apes), игровые предметы, доменные имена.

Уровень 2. Для новичка в web3

ERC-721 — это **технический стандарт невзаимозаменяемых токенов** (Non-Fungible Tokens, NFT) в Ethereum и EVM-совместимых блокчейнах. Стандарт описан в документе **EIP-721** (Ethereum Improvement Proposal), предложенном William Entriken, Dieter Shirley, Jacob Evans и Nastassia Sachs в январе 2018 года.

Что такое невзаимозаменяемый токен?

Non-Fungible (невзаимозаменяемый) — значит, что каждый токен уникален и имеет собственную ценность. В отличие от **ERC-20 (fungible)**, где все токены одинаковы (как доллары), NFT — как билеты на концерт с конкретными местами: билет в первом ряду ≠ билет в последнем.

Почему не подходит ERC-20?

ERC-20 оперирует балансами: «у Алисы 100 токенов, у Боба 50». Но для NFT нужно знать **какими именно токенами** владеет Алиса — токен №42 или токен №777. Каждый NFT идентифицируется парой (адрес контракта, tokenId), которая **глобально уникальна** во всём блокчейне.

Где используются ERC-721 токены?

- **Коллекционное искусство** — CryptoPunks, Bored Ape Yacht Club (BAYC), Azuki
- **Игровые предметы** — оружие, персонажи, земля в метавселенных (Axie Infinity, Decentraland)
- **Доменные имена** — ENS (Ethereum Name Service): vitalik.eth — это NFT
- **Proof-of-Attendance** — POAP: бесплатные NFT за участие в мероприятиях
- **Сертификаты** — Soulbound NFT: дипломы, сертификаты навыков (см. wiki/Proof-of-Skill)
- **Недвижимость и real-world активы** — токенизация физических объектов

ERC-721 vs ERC-20: ключевые отличия

Характеристика	ERC-20	ERC-721
Тип токена	Взаимозаменяемый (fungible)	Уникальный (non-fungible)
Единица учёта	Баланс (balanceOf)	Владение конкретным ID (ownerOf)
Перевод	transfer(кому, сколько)	transferFrom(от, кому, tokenId)
Делимость	Да (через decimals)	Нет (каждый токен — единица)
Метаданные	Только name/symbol	tokenURI — JSON с картинкой и описанием

Уровень 3. Для разработчика (интерфейс и механика)

Интерфейс ERC-721 (EIP-721)

Смарт-контракт считается ERC-721, если реализует интерфейс IERC721, интерфейс IERC165 (supportsInterface), и опционально — расширения IERC721Metadata и IERC721Enumerable.

Обязательные методы

```
// ===== Базовые методы IERC721 =====

// Количество NFT у конкретного адреса
function balanceOf(address _owner) external view returns (uint256)

// Владелец конкретного токена (кидает ошибку, если токен не существует)
function ownerOf(uint256 _tokenId) external view returns (address)

// Безопасный перевод с проверкой: контракт-получатель должен реализовать
// IERC721Receiver.onERC721Received, иначе транзакция ревертится.
// Защищает от безвозвратной потери токенов при переводе на контракт.
function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable
function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes calldata _data) external payable

// «Опасный» перевод – НЕ проверяет, умеет ли получатель работать с NFT.
// Использовать ТОЛЬКО когда уверен, что _to – это EOA или проверенный контракт.
function transferFrom(address _from, address _to, uint256 _tokenId) external payable

// Одобрить адрес для управления одним конкретным токеном
function approve(address _approved, uint256 _tokenId) external payable

// Кто одобрен для управления конкретным токеном (или address(0), если никто)
function getApproved(uint256 _tokenId) external view returns (address)

// Одобрить/запретить оператора для ВСЕХ токенов владельца
function setApprovalForAll(address _operator, bool _approved) external

// Проверить, является ли адрес оператором для владельца
function isApprovedForAll(address _owner, address _operator) external view returns (bool)
```

Обязательные события

```
// Эмитится при любом переводе (включая mint и burn)
event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId)

// Особые случаи:
//   mint:  _from = address(0)
//   burn:  _to   = address(0)

// Эмитится при вызове approve() – одобрение для одного токена
event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId)

// Эмитится при вызове setApprovalForAll() – одобрение/отзыв оператора
event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved)
```

Важное отличие от ERC-20: все три параметра событий Transfer и Approval в ERC-721 — indexed, что позволяет фильтровать по любому из них. В ERC-20 value не был indexed (ограничение: до 3 indexed-параметров на событие).

Расширение IERC721Metadata (опциональное)

```
// Название коллекции
function name() external view returns (string) // Пример: "Bored Ape Yacht Club"

// Символ (тикер)
function symbol() external view returns (string) // Пример: "BAYC"

// URI с метаданными конкретного токена (JSON)
function tokenURI(uint256 _tokenId) external view returns (string)
```

Расширение IERC721Enumerable (опциональное)

```
// Общее количество токенов
function totalSupply() external view returns (uint256)

// Токен по порядковому индексу (для перебора ВСЕХ токенов)
function tokenByIndex(uint256 _index) external view returns (uint256)

// Токен владельца по индексу (для перебора токенов ОДНОГО владельца)
function tokenOfOwnerByIndex(address _owner, uint256 _index) external view returns (uint256)
```

Как работает approve и transferFrom

Модель делегирования, похожая на ERC-20, но работающая на уровне отдельных токенов:

```
Алиса --approve(маркетплейс, tokenId=42)--> ERC-721 контракт
                                     |
Маркетплейс --transferFrom(Алиса, Боб, 42)--> ERC-721 контракт
```

Два уровня делегирования в ERC-721:

1. **approve** — одобрение для одного конкретного токена. Только один адрес может быть approved для заданного tokenId одновременно. При Transfer — approved **сбрасывается** в address(0).
2. **setApprovalForAll** — одобрение оператора для **всех** токенов владельца сразу. Используется маркетплейсами (OpenSea, Blur) для листинга всей коллекции без approve на каждый токен.

safeTransferFrom — защита от потерянных токенов

safeTransferFrom проверяет, что получатель **умеет** принимать NFT:

```
// Интерфейс, который должен реализовать контракт-получатель
interface IERC721Receiver {
    function onERC721Received(
        address operator,
        address from,
        uint256 tokenId,
        bytes calldata data
    ) external returns (bytes4);
}
```

Если _to — смарт-контракт, safeTransferFrom вызывает на нём onERC721Received и проверяет, что возвращённый bytes4 равен селектору IERC721Receiver.onERC721Received.selector. Если нет — транзакция ревёртится, токен остаётся у отправителя.

transferFrom (без «safe») этой проверки НЕ делает — если отправить токен на контракт без onERC721Received, токены будут **безвозвратно потеряны**.

Метаданные: JSON schema

tokenURI(uint256 tokenId) возвращает URI, указывающий на JSON-файл с метаданными токена. Сам файл обычно хранится на:

- **IPFS** — ipfs://Qm... (децентрализованно, нельзя изменить)
- **Arweave** — перманентное хранение
- **Централизованный сервер** — <https://api.example.com/metadata/42> (можно изменить)

- **On-chain (Base64)** — JSON закодирован прямо в контракте (дорого, но неизменяемо)

Стандартная JSON-схема метаданных (из EIP-721)

```
{
  "title": "Asset Metadata",
  "type": "object",
  "properties": {
    "name": {
      "type": "string",
      "description": "Название актива, который представляет NFT"
    },
    "description": {
      "type": "string",
      "description": "Описание актива"
    },
    "image": {
      "type": "string",
      "description": "URI, указывающий на изображение (mime type image/*). Рекомендуется ширина 320–1080px, соотношение сторон от 1.91:1 до 4:5."
    }
  }
}
```

Расширенная схема (OpenSea de-facto стандарт)

```
{
  "name": "Thor's Hammer",
  "description": "Mjöltnir, the legendary hammer of the Norse god of thunder.",
  "image": "https://game.example/item-id-8u5h2m.png",
  "external_url": "https://game.example/item/8u5h2m",
  "attributes": [
    {"trait_type": "Strength", "value": 20},
    {"trait_type": "Element", "value": "Lightning"},
    {"trait_type": "Rarity", "value": "Legendary"}
  ],
  "background_color": "000000",
  "animation_url": "https://game.example/item-id-8u5h2m.mp4"
}
```

Поля сверх name/description/image — это расширения, не входящие в EIP-721, но поддерживаемые маркетплейсами (OpenSea, Rarible):

- **attributes** — массив свойств с trait_type и value (могут быть строкой, числом или boost_percentage)

- `external_url` — ссылка на внешнюю страницу
- `animation_url` — видео, 3D-модель или интерактивный контент
- `background_color` — фоновый цвет в HEX без #

Mint — создание токена

Mint (чеканка) — это создание нового NFT. В ERC-721 mint **не входит в стандартный интерфейс** — в спецификации нет функции `mint()`. Вместо этого стандарт определяет, что создание токена = событие `Transfer(address(0), to, tokenId)`.

Базовая реализация mint через OpenZeppelin

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC721URIStorage, ERC721} from "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

contract GameItem is ERC721URIStorage {
    uint256 private _nextTokenId;

    constructor() ERC721("GameItem", "ITM") {}

    function awardItem(address player, string memory tokenURI) public returns (uint256) {
        uint256 tokenId = _nextTokenId++;
        _mint(player, tokenId);
        _setTokenURI(tokenId, tokenURI);
        return tokenId;
    }
}
```

`_mint(address to, uint256 tokenId)` — внутренняя функция OpenZeppelin:

1. Проверяет, что `to != address(0)` и токен ещё не существует
2. Записывает владельца в mapping
3. Обновляет `balanceOf`
4. Эмитит `Transfer(address(0), to, tokenId)`

ERC721URIStorage — расширение OpenZeppelin, сохраняющее `tokenURI` для каждого токена в `storage` контракта. Альтернативно можно переопределить `tokenURI()` и вычислять URI динамически (например, через `baseURI + tokenId`).

Уровень 4. Для продвинутого разработчика (реализация и нюансы)

Минимальная реализация ERC-721 на Solidity


```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract SimpleNFT {
    string public name = "SimpleNFT";
    string public symbol = "SNFT";

    mapping(uint256 => address) private _owners;
    mapping(address => uint256) private _balances;
    mapping(uint256 => address) private _tokenApprovals;
    mapping(address => mapping(address => bool)) private _operatorApprovals;
}
```

> ****Как читать четыре mapping'a ERC-721:**** читай `_owners[tokenId]` → `address` как «чей это токен», `_balances[address]` → `count` как «сколько токенов у адреса», `_tokenApprovals[tokenId]` → `address` как «кому разрешено перевести этот конкретный токен», `_operatorApprovals[владелец][оператор]` → `bool` как «разрешено ли оператору управлять ВСЕМИ токенами владельца». Мнемоника: mapping'и ERC-721 – это четыре журнала: журнал владельцев, журнал балансов, журнал доверенностей на токен и журнал полных доверенностей.

```
    event Transfer(address indexed from, address indexed to, uint256 indexed tokenId);
    event Approval(address indexed owner, address indexed approved, uint256 indexed tokenId);
    event ApprovalForAll(address indexed owner, address indexed operator, bool approved);
```

```
    function balanceOf(address owner) public view returns (uint256) {
        require(owner != address(0), "ERC721: zero address");
        return _balances[owner];
    }
```

```
    function ownerOf(uint256 tokenId) public view returns (address) {
        address owner = _owners[tokenId];
        require(owner != address(0), "ERC721: invalid token ID");
        return owner;
    }
```

```
    function _isApprovedOrOwner(address spender, uint256 tokenId)
private view returns (bool) {
        address owner = ownerOf(tokenId);
        return (spender == owner ||
                getApproved(tokenId) == spender ||
                isApprovedForAll(owner, spender));
    }
```

> ****Как читать `_isApprovedOrOwner` с тройным `|||`:**** читай как «разрешаю перевод, если вызывающий удовлетворяет ХОТЯ БЫ ОДНОМУ из трёх у

словий: (1) он и есть владелец токена, (2) ему выдали approve на этот конкретный токен, (3) ему выдали setApprovalForAll на ВСЕ токены владельца». Мнемоника: это шлюз с тремя дверьми – владелец проходит в первую, доверенный на токен во вторую, оператор всей коллекции в третью.

```
function transferFrom(address from, address to, uint256 tokenId) public payable {
    require(_isApprovedOrOwner(msg.sender, tokenId), "ERC721: caller is not owner nor approved");
    require(ownerOf(tokenId) == from, "ERC721: transfer from incorrect owner");
    require(to != address(0), "ERC721: transfer to zero address");

    // Сброс approved при переводе
    _tokenApprovals[tokenId] = address(0);

    _balances[from] -= 1;
    _balances[to] += 1;
    _owners[tokenId] = to;

    emit Transfer(from, to, tokenId);
}

function approve(address to, uint256 tokenId) public payable {
    address owner = ownerOf(tokenId);
    require(to != owner, "ERC721: approval to current owner");
    require(msg.sender == owner || isApprovedForAll(owner, msg.sender),
        "ERC721: approve caller is not owner nor approved for all");

    _tokenApprovals[tokenId] = to;
    emit Approval(owner, to, tokenId);
}

function getApproved(uint256 tokenId) public view returns (address) {
    require(_owners[tokenId] != address(0), "ERC721: invalid token ID");
    return _tokenApprovals[tokenId];
}

function setApprovalForAll(address operator, bool approved) public {
    require(operator != msg.sender, "ERC721: approve to caller");
    _operatorApprovals[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}

function isApprovedForAll(address owner, address operator) public view returns (bool) {
    return _operatorApprovals[owner][operator];
}
```

```

// Внутренний mint – не входит в стандарт, но необходим
function _mint(address to, uint256 tokenId) internal {
    require(to != address(0), "ERC721: mint to zero address");
    require(_owners[tokenId] == address(0), "ERC721: token already
minted");

    _balances[to] += 1;
    _owners[tokenId] = to;

    emit Transfer(address(0), to, tokenId);
}
}

```

OpenZeppelin — промышленная реализация

В реальных проектах используют OpenZeppelin:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/
ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyNFT is ERC721URIStorage, Ownable {
    uint256 private _nextTokenId;
    uint256 public constant MAX_SUPPLY = 10000;
    uint256 public constant MINT_PRICE = 0.05 ether;

    constructor() ERC721("MyNFT", "MNFT") Ownable(msg.sender) {}

    function mint(address to, string memory uri) public payable returns
(uint256) {
        require(msg.value >= MINT_PRICE, "Insufficient payment");
        require(_nextTokenId < MAX_SUPPLY, "Max supply reached");

        uint256 tokenId = _nextTokenId++;
        _mint(to, tokenId);
        _setTokenURI(tokenId, uri);
        return tokenId;
    }

    function withdraw() public onlyOwner {
        payable(owner()).transfer(address(this).balance);
    }
}

```

Расширения OpenZeppelin для ERC-721:

Расширение	Назначение
ERC721URIStorage	Хранение <code>tokenURI</code> в <code>storage</code> (газозатратно, но гибко)
ERC721Enumerable	<code>totalSupply</code> , <code>tokenByIndex</code> , <code>tokenOfOwnerByIndex</code>
ERC721Burnable	<code>burn(tokenId)</code> — уничтожение токена
ERC721Pausable	Пауза всех переводов (экстренная остановка)
ERC721Royalty (ERC-2981)	Роялти: комиссия автору при перепродаже
ERC721Consecutive	Пакетный <code>mint</code> с последовательными ID

Soulbound-токены (non-transferable NFT)

Soulbound-токен (SBT) — это NFT, который **нельзя передать** другому адресу после `mint`. Концепция предложена Виталиком Бутериным в 2022 году для цифровой идентичности (дипломы, сертификаты, KYC, репутация).

Реализация Soulbound

Способ 1. Блокировка `transferFrom`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

contract SoulboundCertificate is ERC721URIStorage {
    uint256 private _nextTokenId;
    mapping(uint256 => bool) private _soulbound; // можно ли передавать?

    constructor() ERC721("Skill Certificate", "SKILL") {}

    function mint(address to, string memory uri, bool soulbound) public returns (uint256) {
        uint256 tokenId = _nextTokenId++;
        _mint(to, tokenId);
        _setTokenURI(tokenId, uri);
        _soulbound[tokenId] = soulbound;
        return tokenId;
    }

    // Блокируем перевод для soulbound-токенов
    function transferFrom(address from, address to, uint256 tokenId) public override {
        require(!_soulbound[tokenId], "Soulbound: token is non-transferable");
        super.transferFrom(from, to, tokenId);
    }

    function safeTransferFrom(address from, address to, uint256 tokenId, bytes memory data) public override {
        require(!_soulbound[tokenId], "Soulbound: token is non-transferable");
        super.safeTransferFrom(from, to, tokenId, data);
    }
}
```

Способ 2. Полная блокировка (все токены контракта)

```
// Блокируем ВСЕ переводы – проще и надёжнее
function _update(address to, uint256 tokenId, address auth) internal override returns (address) {
    address from = _ownerOf(tokenId);
    // Разрешаем только mint (from = 0) и burn (to = 0)
    require(from == address(0) || to == address(0), "Soulbound: non-transferable");
    return super._update(to, tokenId, auth);
}
```

Как читать `require(from == address(0) || to == address(0))` в

Soulbound: читай как «транзакция разрешена только в двух случаях: либо токен рождается из ничего (`from = 0x0`, это `mint`), либо токен уходит в никуда (`to = 0x0`, это `burn`). Всё остальное — обычный перевод — запрещено». Мнемоника: `address(0)` — это «нулевой адрес», как `null` в JavaScript; `mint = from=null`, `burn = to=null`, а если ни то ни другое — реверт.

Где используется Soulbound:

- **UNI-дипломы и сертификаты** (см. [wiki/Proof-of-Skill](#))
- **Proof-of-Attendance** (ПОАР — их тоже нельзя передавать, если организатор так настроил)
- **KYC/identity** (верификация личности)
- **Голоса в governance** (голос привязан к личности, не продаётся)

ERC-721A (Azuki) — gas-оптимизация пакетного минта

ERC-721A — улучшенная реализация ERC-721 от команды **Azuki** (Chiru Labs). Ключевое преимущество: **пакетный mint нескольких токенов стоит почти как mint одного**. Стандартный ERC-721 (OpenZeppelin) делает $N \times \text{SSTORE}$ при `mint N` токенов — это очень дорого.

Как работает оптимизация (lazy initialization)

ERC-721A использует механизм **отложенной инициализации**:

1. **При mint:** токены выпускаются последовательными ID. Контракт делает **всего 3 SSTORE** независимо от количества:
2. Инициализирует слот владельца только для **первого** токена в партии
3. Обновляет `balanceOf` получателя
4. Обновляет `_nextTokenId`
5. События `Transfer` эмитятся для каждого токена (но event дешевле SSTORE на порядок)
6. **При transfer:** инициализация слотов владельцев происходит **лениво**, в момент первого перевода токена. Это стоит дороже, чем обычный `transfer`, но эту цену платят при перепродаже, а не при `mint`.

Идея: газ на `mint` дороже (BASEFEE выше из-за ажиотажа), а `transfer` происходит позже, когда сеть спокойнее.

Как читать ERC-721A «lazy initialization»: читай как «при чеканке не записываю владельца каждого токена по отдельности (это $N \times \text{SSTORE}$ = дорого), а записываю только первый токен партии и говорю "владелец у токенов #42-46 — Алиса, потом разберёмся"».

Реальная запись владельца откладывается до первого transfer'a этого токена. Мнемоника: это как билеты на концерт — при покупке пачки записываешь только номер первого билета и что «билеты 42-46 купила Алиса», а номера мест уточняешь когда она начнёт их передавать.

Сравнение gas-расходов

Операция	OpenZeppelin ERC-721	ERC-721A
Mint 5 токенов	155 949 gas	63 748 gas (−59%)
Transfer 5 токенов	226 655 gas	334 450 gas (+47%)
Mint BASEFEE	200 gwei	200 gwei
Transfer BASEFEE	40 gwei	40 gwei
Итого комиссия	0.0403 ETH	0.0261 ETH (−35%)

Даже при консервативной разнице в BASEFEE (200 vs 40), экономия на практике ещё выше, потому что массовые минты разгоняют BASEFEE экспоненциально (при заполнении gas limit блока).

Первый transfer vs повторные

Операция	OpenZeppelin	ERC-721A
Первый transfer токена	45 331 gas	92 822 gas (дороже: 2 SSTORE + 5 SLOAD)
Повторный transfer	45 331 gas	44 499 gas (дешевле: слот уже инициализирован)

- Дополнительные затраты первого transfer:
- **2 extra SSTORE** — инициализация текущего и следующего слотов
 - **5 extra SLOAD** — чтение предыдущих слотов и следующего слота

Использование ERC-721A

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.4;

import "erc721a/contracts/ERC721A.sol";

contract Azuki is ERC721A {
    uint256 public constant MAX_SUPPLY = 10000;
    uint256 public constant MAX_PER_WALLET = 5;
    uint256 public constant MINT_PRICE = 0.05 ether;

    constructor() ERC721A("Azuki", "AZUKI") {}

    function mint(uint256 quantity) external payable {
        require(totalSupply() + quantity <= MAX_SUPPLY, "Max supply
exceeded");
        require(_numberMinted(msg.sender) + quantity <= MAX_PER_WALLET, "
Max per wallet");
        require(msg.value >= MINT_PRICE * quantity, "Insufficient
payment");

        // _mint(address to, uint256 quantity) — a не tokenId!
        _mint(msg.sender, quantity);
    }

    function _baseURI() internal view virtual override returns (string m
emory) {
        return "https://api.azuki.com/metadata/";
    }
}
```

Ключевое отличие API: `_mint(address to, uint256 quantity)` ВМЕСТО `_mint(address to, uint256 tokenId)`.

Важные нюансы ERC-721A

- **Токены должны быть последовательными** — transfer ломает последовательность, поэтому ERC-721A отслеживает «дырки» через слоты владельцев
- **balanceOf(address)** — остаётся O(1) через внутренний mapping балансов (команда отказалась от его удаления ради совместимости)
- **_numberMinted(address)** — дополнительная функция: сколько токенов адрес заминтил всего (включая проданные); используется для whitelist-проверок

- **_getAux / _setAux** — вспомогательные 64-битные слоты на адрес для кастомных данных (например, whitelist-биты) с минимальными затратами газа
- **Совместимость** — ERC-721A полностью совместим с EIP-721 (реализует IERC721, IERC721Metadata, IERC721Enumerable) и проходит все тесты стандарта
- **Расширения:** ERC721ABurnable, ERC721AQueryable, ERC4907A (аренда NFT)

Когда использовать ERC-721A, а когда стандартный ERC-721

Сценарий	Выбор
Массовый mint (коллекции 1000+)	ERC-721A — экономия газа на mint
Одиночные мints / сертификаты	Стандартный ERC-721 — проще, меньше байткода
Игровые предметы с частыми transfer	Стандартный ERC-721 — дешевле повторные transfer
Коллекции с reveal-механикой	ERC-721A — базовая экономия + расширения
Soulbound (нет transfer)	Стандартный ERC-721 — gas-оптимизация ERC-721A не даёт преимуществ без transfer

ERC-721 на практике: ethers.js / viem

```
// ===== viem (рекомендуется) =====
import { createPublicClient, http } from 'viem';

const client = createPublicClient({ /* ... */ });

// Чтение владельца токена
const owner = await client.readContract({
  address: nftContractAddress,
  abi: erc721Abi,
  functionName: 'ownerOf',
  args: [42n],
});

// Чтение метаданных
const tokenURI = await client.readContract({
  address: nftContractAddress,
  abi: erc721Abi,
  functionName: 'tokenURI',
  args: [42n],
});

// Если URI — ipfs://..., преобразуем в HTTP
const httpUrl = tokenURI.replace('ipfs://', 'https://ipfs.io/ipfs/');
const metadata = await fetch(httpUrl).then(r => r.json());
console.log(metadata.name, metadata.image);

// Подписка на события Transfer в реальном времени
client.watchEvent({
  address: nftContractAddress,
  event: parseAbiItem('event Transfer(address indexed from, address indexed to, uint256 indexed tokenId)'),
  onLogs: (logs) => {
    for (const log of logs) {
      console.log(`${log.args.from} → ${log.args.to}: #${log.args.tokenId}`);
    }
  },
});
```

Известные уязвимости и проблемы

1. Потеря токенов через transferFrom на контракт

Если использовать transferFrom ВМЕСТО safeTransferFrom, и получатель — контракт без onERC721Received, NFT теряется навсегда.

Защита: всегда использовать safeTransferFrom, если получатель может быть контрактом.

2. Reentrancy в safeTransferFrom

safeTransferFrom делает внешний вызов в контракт получателя. Если контракт-получатель вызывает отправителя обратно до завершения перевода — возможна рекурсивная атака. OpenZeppelin защищает от этого, обновляя балансы ДО внешнего вызова (checks-effects-interactions).

3. Неуникальные tokenURI

Если tokenURI отдаёт одинаковый URI для разных токенов — токены технически разные (разные ID), но выглядят одинаково. Это не баг, но может ввести пользователей в заблуждение.

4. Централизованные метаданные

Если tokenURI указывает на централизованный сервер (https://...), владелец сервера может изменить метаданные в любой момент.

Решение: IPFS, Arweave или on-chain Base64.

5. Отсутствие supportsInterface

Контракт обязан реализовать IERC165 с supportsInterface. Некоторые маркетплейсы и кошельки не распознают NFT без корректного supportsInterface, что приводит к ошибкам отображения.

Связанное

- [wiki/Proof-of-Skill](#) — проект на Soulbound NFT
- [wiki/ERC-20-стандарт-токенов](#) — стандарт взаимозаменяемых токенов (отличия ERC-20 vs ERC-721)
- [wiki/Solidity-основы](#) — синтаксис Solidity, необходимый для написания ERC-721
- [wiki/Главная](#) — дорожная карта изучения web3, этап 2 (смарт-контракты)
- [wiki/Proof-of-Skill](#) — проект на ERC-721 Soulbound: децентрализованные сертификаты навыков
- [wiki/Словарь-web3](#) — термины: event, gas, ABI, storage, mapping, IPFS
- [wiki/Блокчейн-как-это-работает](#) — база по блокчейну: аккаунты, транзакции, gas

Hardhat — среда разработки смарт-контрактов

Hardhat — это **фреймворк полного цикла** для разработки смарт-контрактов на Ethereum. Компиляция, тестирование, деплой, отладка и скрипты — всё в одном инструменте.

Документация: hardhat.org | **Репозиторий:** github.com/NomicFoundation/hardhat

Уровень 1: Для новичка (что это и зачем)

Hardhat — это как **create-react-app + Jest + Webpack** для блокчейна. Ты пишешь Solidity-код, а Hardhat:

- **Компилирует** его в байт-код (то, что понимает EVM — Ethereum Virtual Machine)
- **Запускает локальный блокчейн** у тебя на машине (Hardhat Network) — мгновенный, бесплатный, с 10 000 тестовых ETH
- **Прогоняет тесты** — на JavaScript/TypeScript, с привычными Chai-ассертами
- **Деплоит** контракты в тестовые сети (Sepolia) или mainnet
- **Даёт консоль** — интерактивный REPL, где можно дёргать контракты на лету

Аналогия: если Solidity — это язык (как JavaScript), то Hardhat — это среда выполнения (как Node.js). Без Hardhat ты бы компилировал через solc вручную, деплоил сырыми транзакциями и отлаживал через print-дебаг.

Минимальный старт за 2 минуты

```
mkdir my-project && cd my-project
npm init -y
npm install --save-dev hardhat
npx hardhat init           # выбрать «Create a JavaScript project»
npx hardhat compile        # скомпилировать
npx hardhat test           # прогнать тесты
```

Всё. Ты уже в экосистеме Hardhat. Дальше — копай вглубь.

Уровень 2: Рабочий минимум (каждодневные операции)

Установка

Устанавливается **локально в проект** (не глобально). Это важно: версия Hardhat фиксируется в `package.json`, и проект воспроизводим.

```
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
```

@nomicfoundation/hardhat-toolbox — батарейка в комплекте: тянет ethers.js, hardhat-chai-matchers, hardhat-network-helpers, typechain и ещё 10 плагинов. **Рекомендуется всегда.**

Инициализация проекта

```
npx hardhat init
```

Hardhat предложит 3 варианта:

- **JavaScript project** — стандарт, для большинства
- **TypeScript project** — если хочешь типы (рекомендую)
- **Empty config** — только hardhat.config.js, без примера

После инициализации структура:

```
my-project/
├── contracts/           ← смарт-контракты (.sol)
│   └── Lock.sol         ← пример от Hardhat
├── ignition/           ← модули деплоя (Hardhat Ignition — новый
способ)
│   └── modules/
├── test/               ← тесты (.js/.ts)
│   └── Lock.js
├── hardhat.config.js    ← главный конфиг
└── package.json
```

hardhat.config.js — центр управления

```
require("@nomicfoundation/hardhat-toolbox");

/** @type import('hardhat/config').HardhatUserConfig */
module.exports = {
  solidity: "0.8.24",           // версия компилятора
  networks: {
    hardhat: {},               // локальная сеть
    (автоматически)
    sepolia: {
      url: `https://sepolia.infura.io/v3/${INFURA_KEY}`,
      accounts: [PRIVATE_KEY]   // НИКОГДА не коммитить ключи!
    }
  }
};
```

Важно: для переменных окружения используй dotenv:

```
npm install --save-dev dotenv
```

```
require("dotenv").config();
// ...
accounts: [process.env.PRIVATE_KEY]
```

Компиляция

```
npx hardhat compile
```

Что происходит:

1. `solc` (Solidity-компилятор) — скачивается автоматически, первая компиляция дольше
2. Артефакты кладутся в `artifacts/contracts/<Имя>.sol/<Имя>.json`
3. В артефактах: ABI (интерфейс), байт-код, AST, source map
4. Кеш: `cache/` — повторная компиляция почти мгновенная

```
npx hardhat compile --force # перекомпилировать всё, игнорируя кеш
```

Hardhat Network — локальный блокчейн

Запускается **автоматически** при `npx hardhat test` или `npx hardhat run`. Не нужен Ganache, не нужен Docker.

```
npx hardhat node # явно запустить ноду (HTTP: localhost:8545)
```

Возможности:

- **20 предустановленных аккаунтов** с 10 000 ETH каждый (мнемоника: `test test test...`)
- **Мгновенный майнинг** — транзакции подтверждаются сразу
- **console.log в Solidity** — `import "hardhat/console.sol";` и вызывай `console.log(x)`
- **Форкинг mainnet** — копирует состояние Ethereum на твой локальный блок:

```
networks: {
  hardhat: {
    forking: {
      url: `https://mainnet.infura.io/v3/${INFURA_KEY}`,
    }
  }
}
```

Деплой в тестовую сеть (Sepolia)

Sepolia — основная тестовая сеть Ethereum (заменила Goerli). Для деплоя нужны:

1. **RPC-URL** — Infura, Alchemy или публичный
2. **Приватный ключ** аккаунта с Sepolia ETH
3. **Тестовые ETH** — sepoliafaucet.com или кран Alchemy

Скрипт деплоя (`scripts/deploy.js`):

```
const hre = require("hardhat");

async function main() {
  // Получаем фабрику контракта
  const Counter = await hre.ethers.getContractFactory("Counter");

  > **Как читать `hre.ethers.getContractFactory("Counter")`:** «найди скомпилированный контракт по имени и подготовь фабрику для его развёртывания — это как взять чертёж перед стройкой». Мнемоника: `getContractFactory` = взял чертёж, `deploy()` = построил дом на блокчейне.

  // Деплоим (это создаёт транзакцию)
  const counter = await Counter.deploy();

  // Ждём подтверждения
  await counter.waitForDeployment();

  console.log(`Counter deployed to: ${counter.target}`);
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

Запуск:

```
npx hardhat run scripts/deploy.js --network sepolia
```

Проверить деплой можно на sepolia.etherscan.io — вставь адрес контракта.

Верификация контракта (чтобы код был виден на Etherscan):

```
npm install --save-dev @nomicfoundation/hardhat-verify
npx hardhat verify --network sepolia DEPLOYED_ADDRESS
```

Тестирование (Chai + hardhat-chai-matchers)

Hardhat использует **Mocha** + **Chai** + специальные Ethereum-ассерты. Файлы тестов лежат в `test/`.

Базовый тест для Counter.sol (разбираем наш контракт из `raw/Counter.sol`):

```

const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("Counter", function () {
  let counter, owner, other;

  beforeEach(async function () {
    [owner, other] = await ethers.getSigners();

    const Counter = await ethers.getContractFactory("Counter");
    counter = await Counter.deploy();
    await counter.waitForDeployment();
  });

  it("должен начинаться с нуля", async function () {
    expect(await counter.getCount()).to.equal(0);
  });

  it("должен увеличивать счётчик", async function () {
    await counter.increment();
    expect(await counter.getCount()).to.equal(1);

    await counter.increment();
    expect(await counter.getCount()).to.equal(2);
  });

  it("должен уменьшать счётчик", async function () {
    await counter.increment();
    await counter.increment();
    await counter.decrement();
    expect(await counter.getCount()).to.equal(1);
  });

  it("только owner может сбросить счётчик", async function () {
    // Ожидаем revert от не-owner
    await expect(
      counter.connect(other).reset()
    ).to.be.revertedWith("Only owner can call this");

    > **Как читать `counter.connect(other).reset()`:** «вызови функцию `reset`
    ` от имени другого аккаунта `other` — Hardhat притворится, что
    транзакцию отправил `other`, а не деплойер». Мнемоника: `.connect(аккаунт
    )` = надень чужую шляпу и действуй от его лица в тесте.

    // Owner может
    await counter.connect(owner).reset();
    expect(await counter.getCount()).to.equal(0);
  });

  it("должен эмитить событие Incremented", async function () {

```



```
    await expect(counter.increment())
      .to.emit(counter, "Incremented")
      .withArgs(1);
  });

  it("должен эмитить событие Reset с правильными параметрами", async function () {
    await counter.increment();
    await counter.increment(); // count = 2

    await expect(counter.reset())
      .to.emit(counter, "Reset")
      .withArgs(owner.address, 2);
  });
});
```

Ключевые ассерты hardhat-chai-matchers:

Ассерт	Для чего
expect(tx).to.emit(contract, "Event")	Проверка событий
expect(tx).to.be.revertedWith("msg")	Ожидание revert
expect(tx).to.be.revertedWithoutReason()	Revert без сообщения
expect(tx).to.changeEtherBalance(addr, delta)	Изменение ETH-баланса
expect(tx).to.changeTokenBalance(token, addr, delta)	Изменение токен-баланса
expect(value).to.be.properHex(length)	Валидный hex

Запуск тестов:

```
npx hardhat test                # все тесты
npx hardhat test test/Counter.js # конкретный файл
npx hardhat test --grep "должен увеличивать" # по названию
```

Hardhat Console — интерактивная песочница

```
npx hardhat console
```

Открывает Node.js REPL с подключённой сетью Hardhat. Можно на лету деплоить контракты и вызывать функции:

```
// Внутри консоли:
const [owner] = await ethers.getSigners();

const Counter = await ethers.getContractFactory("Counter");
const counter = await Counter.deploy();
await counter.waitForDeployment();

await counter.increment();
(await counter.getCount()).toString(); // "1"

// Быстрая проверка revert:
await counter.connect(ethers.ZeroAddress).reset();
// Error: VM Exception... "Only owner can call this"
```

Консоль с подключением к Sepolia:

```
npx hardhat console --network sepolia
```

Скрипты — автоматизация задач

Скрипты в `scripts/` делают что угодно: деплой, вызов функций, проверку состояния.

```
// scripts/check-counter.js
const hre = require("hardhat");

async function main() {
  const counter = await hre.ethers.getContractAt(
    "Counter",
    "0x..." // адрес задеплойенного контракта
  );
  const count = await counter.getCount();
  console.log(`Текущее значение: ${count}`);
}

main().catch(console.error);
```

```
npx hardhat run scripts/check-counter.js --network sepolia
```

Уровень 3: Понимание (как это устроено внутри)

Архитектура Hardhat

Hardhat — модульная система. Ядро минимально, всё остальное — плагины:

Hardhat Runtime Environment (HRE)

└─ hardhat-core	← компилятор, нода, таск-раннер
└─ Плагины (npm-пакеты)	
└─ @nomicfoundation/hardhat-toolbox	← батарейка
└─ @nomicfoundation/hardhat-verify	← верификация на Etherscan
└─ @nomicfoundation/hardhat-ignition	← декларативный деплой
└─ hardhat-gas-reporter	← отчёт по газу
└─ solidity-coverage	← покрытие тестами
└─ Твои скрипты и тесты	

HRE (Hardhat Runtime Environment) — глобальный объект, доступный в скриптах и тестах. Содержит `ethers`, `network`, `artifacts`, `config`, `run`. Импортируется как `const hre = require("hardhat")`.

Компиляция: что внутри

```
.sol → solc → байт-код (creation + runtime) + ABI + source map
      ↓
artifacts/contracts/Имя.sol/Имя.json:
{
  "abi": [...],                // интерфейс контракта
  "bytecode": "0x6080...",    // байт-код для деплоя
  "deployedBytecode": "...",  // байт-код который живёт на chain
  "sourceMap": "...",        // маппинг байт-код → исходник (для
отладки)
}
```

Creation bytecode содержит constructor-логику — выполняется один раз при деплое. **Deployed (runtime) bytecode** — то, что хранится в блокчейне и выполняется при вызовах.

Hardhat Network: как работает локальный блокчейн

Hardhat Network — это **in-process EVM**, написанный на TypeScript. Не отдельный процесс (как `ganache`), а внутри `Node.js`:

- **Мгновенный майнинг:** по умолчанию каждый вызов майнит блок сразу (`auto-mining`)
- **Интервальный майнинг:** задаёшь интервал в секундах или вручную через `evm_mine`
- **Снэпшоты:** `await network.provider.send("evm_snapshot") / evm_revert` — git stash для блокчейна
- **impersonate:** вызывай функции от имени любого адреса — `await network.provider.request({method: "hardhat_impersonateAccount", params: ["0x..."]})`

```
// В тесте: заморозить время
await network.provider.send("evm_increaseTime", [3600]); // +1 час
await network.provider.send("evm_mine");                 // майним блок
```

Hardhat Ignition (новый способ деплоя)

На смену scripts/deploy.js пришёл **Hardhat Ignition** — декларативный подход:

```
// ignition/modules/CounterModule.js
const { buildModule } = require("@nomicfoundation/hardhat-ignition/
modules");

module.exports = buildModule("CounterModule", (m) => {
  const counter = m.contract("Counter");

  return { counter };
});

> **Как читать `buildModule("CounterModule", (m) => { const counter =
m.contract("Counter"); return { counter }; })`:** «опиши декларативно, *ч
то* задеплоить: модуль — это именованный набор контрактов;
`m.contract()` регистрирует контракт, возвращаемый объект передаёт резул
ьтаты зависимым модулям». Мнемоника: Ignition = скажи *что* задеплоить, а н
е *как*; повторный запуск не создаст дубликат.
```

```
npx hardhat ignition deploy ignition/modules/CounterModule.js --network s
epolia
```

Преимущества Ignition:

- **Декларативность:** описываешь что задеплоить, а не как
- **Идемпотентность:** повторный деплой не создаст дубликат
- **Зависимости:** один модуль может ссылаться на результат другого
- **Верификация:** автоматическая после деплоя (--verify)

Система задач (Tasks)

Задачи — аналог npm scripts, но с аргументами и типами. Живут в hardhat.config.js или tasks/.

```
task("accounts", "Показать список аккаунтов").setAction(async (_, hre)
=> {
  const accounts = await hre.ethers.getSigners();
  for (const acc of accounts) {
    console.log(acc.address, ":", ethers.formatEther(
      await hre.ethers.provider.getBalance(acc.address)
    ), "ETH");
  }
});

> **Как читать `task("accounts", "описание").setAction(async (_, hre) =>
{ ... }`:** «зарегистрируй новую команду `npx hardhat accounts`,
которая выполнит эту асинх-функцию с доступом ко всему окружению Hardhat
(hre)». Мнемоника: `task` = npm-скрипт с аргументами, но с полным доступ
ом к блокчейну и контрактам.
```

```
npx hardhat accounts
```

С параметрами:

```
task("counter-status", "Проверить счётчик")
  .addParam("address", "Адрес контракта")
  .setAction(async ({ address }, hre) => {
    const counter = await hre.ethers.getContractAt("Counter", address);
    console.log("Count:", (await counter.getCount()).toString());
  });
```

```
npx hardhat counter-status --address 0x... --network sepolia
```

Gas-оптимизация

```
npx hardhat test # обычные тесты
REPORT_GAS=true npx hardhat test # с отчётом по газу (hardhat-gas-reporter)
```

hardhat-gas-reporter показывает таблицу: какая функция сколько газа тратит. Критично для продакшена.

Обновление до Hardhat 3

Hardhat v3 (вышел в 2025) принёс:

- **Hardhat Ignition** как основной способ деплоя (замена scripts deploy)
- **Ускоренную компиляцию** через кеширование на уровне файлов
- **Встроенную поддержку ESM** (import/export вместо require)
- **Viem** как альтернативу ethers.js (через @nomicfoundation/hardhat-viem)

Чтобы обновиться:

```
npm install --save-dev hardhat@latest @nomicfoundation/hardhat-toolbox@latest
```

Уровень 4: Разбор Counter.sol через Hardhat

Контракт raw/Counter.sol — наш учебный пример. Разберём его **полный цикл разработки** через Hardhat.

Код контракта

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Counter {
    uint256 public count;
    address public owner;

    event Incremented(uint256 newValue);
    event Decremented(uint256 newValue);
    event Reset(address indexed by, uint256 oldValue);

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can call this");
        _;
    }

    function increment() public {
        count += 1;
        emit Incremented(count);
    }

    function decrement() public {
        count -= 1;
        emit Decremented(count);
    }

    function reset() public onlyOwner {
        uint256 oldValue = count;
        count = 0;
        emit Reset(msg.sender, oldValue);
    }

    function getCount() public view returns (uint256) {
        return count;
    }
}
```

Шаг 1: Инициализация проекта под Counter

```
mkdir counter-project && cd counter-project
npm init -y
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npx hardhat init          # выбрать JavaScript
```

Копируем Counter.sol в contracts/, удаляем Lock.sol.

Шаг 2: Конфигурация

```
// hardhat.config.js
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

module.exports = {
  solidity: "0.8.20",    // совпадает с прагмой в контракте!
  networks: {
    sepolia: {
      url: `https://sepolia.infura.io/v3/${process.env.INFURA_KEY}`,
      accounts: [process.env.PRIVATE_KEY],
    }
  }
};
```

Шаг 3: Компиляция

```
npx hardhat compile
# → Compiled 1 Solidity file successfully
```

После компиляции в `artifacts/contracts/Counter.sol/Counter.json` можно найти:

- **ABI** — JSON с описанием функций (нужен фронтенду)
- **bytecode** — hex-строка, которая загружается в блокчейн

Шаг 4: Тестирование (полный набор)

Файл `test/Counter.js`:

```
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("Counter – полный цикл", function () {
  let counter, owner, other;

  beforeEach(async function () {
    [owner, other] = await ethers.getSigners();
    const Counter = await ethers.getContractFactory("Counter");
    counter = await Counter.deploy();
    await counter.waitForDeployment();
  });

  describe("Развёртывание", function () {
    it("owner = деплойер", async function () {
      expect(await counter.owner()).to.equal(owner.address);
    });

    it("начальное значение = 0", async function () {
      expect(await counter.getCount()).to.equal(0);
    });
  });

  describe("increment()", function () {
    it("увеличивает на 1", async function () {
      await counter.increment();
      expect(await counter.getCount()).to.equal(1);
    });

    it("эмитит Incremented с новым значением", async function () {
      await expect(counter.increment())
        .to.emit(counter, "Incremented")
        .withArgs(1);
    });

    it("может вызывать кто угодно", async function () {
      await counter.connect(other).increment();
      expect(await counter.getCount()).to.equal(1);
    });
  });

  describe("decrement()", function () {
    it("уменьшает на 1", async function () {
      await counter.increment();
      await counter.increment(); // 2
      await counter.decrement(); // 1
      expect(await counter.getCount()).to.equal(1);
    });

    it("может уйти в underflow (опасно!)", async function () {
```



```

    // count = 0, decrement → ошибка в Solidity 0.8+ (built-in overflow
check)
    await expect(counter.decrement()).to.be.revertedWithPanic(0x11);
    // 0x11 = арифметическое переполнение
  });
});

describe("reset()", function () {
  it("сбрасывает в 0", async function () {
    await counter.increment();
    await counter.increment(); // 2
    await counter.reset();
    expect(await counter.getCount()).to.equal(0);
  });

  it("только owner", async function () {
    await expect(
      counter.connect(other).reset()
    ).to.be.revertedWith("Only owner can call this");
  });

  it("эмитит Reset со старым значением", async function () {
    await counter.increment();
    await counter.increment();
    await expect(counter.reset())
      .to.emit(counter, "Reset")
      .withArgs(owner.address, 2);
  });
});

describe("getCount()", function () {
  it("view-функция: не тратит газ при внешнем вызове", async function () {
    const tx = await counter.getCount();
    // view-функция, вызванная через ethers call, не создаёт транзакцию
  });
});
});

```

Запускаем:

```
npx hardhat test
```

Шаг 5: Деплой в Sepolia

scripts/deploy.js:

```
const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.ethers.getSigners();
  console.log("Деплой с адреса:", deployer.address);

  const Counter = await hre.ethers.getContractFactory("Counter");
  const counter = await Counter.deploy();
  await counter.waitForDeployment();

  console.log("Counter задеплоен по адресу:", counter.target);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

```
npm run hardhat run scripts/deploy.js --network sepolia
```

Шаг 6: Проверка через консоль

```
npm run hardhat console --network sepolia
```

```
const counter = await ethers.getContractAt("Counter", "0x...");
(await counter.getCount()).toString();
// → "0"

await counter.increment();
(await counter.getCount()).toString();
// → "1"
```

Чему учит этот пример

Counter.sol демонстрирует **все ключевые концепции Solidity**, развёрнутые через Hardhat:

Концепция	В контракте	В тестах
Состояние (storage)	count, owner	Проверка начальных значений
Модификатор	onlyOwner	revertedWith("Only owner...")
События	Incremented, Decremented, Reset	.to.emit().withArgs()
View-функция	getCount()	Вызов без газа
msg.sender	constructor, onlyOwner	.connect(other) — смена отправителя
Access Control	require(msg.sender == owner)	Тест на несанкционированный вызов
SafeMath (встроен)	underflow при decrement от 0	revertedWithPanic(0x11)

Связанное

- [wiki/OpenZeppelin-безопасные-контракты](#) — безопасные контракты для Hardhat-проектов
- [wiki/Solidity-основы](#) — синтаксис, типы, события, модификаторы
- [wiki/ERC-20-стандарт-токенов](#) — следующий шаг: токены
- [wiki/Блокчейн-как-это-работает](#) — база: газ, транзакции, консенсус
- [wiki/Словарь-web3](#) — термины (EVM, ABI, байт-код, gas)
- [wiki/Сравнение-ethers-viem-wagmi](#) — библиотеки для вызова контрактов с фронтенда
- [wiki/Главная](#) — дорожная карта обучения

Источники

- hardhat.org/docs — официальная документация (v3)
- hardhat.org/hardhat-runner/docs/guides/project-setup
- [@nomicfoundation/hardhat-chai-matchers](https://nomicfoundation.org/hardhat-chai-matchers) — документация по ассертам
- hardhat.org/hardhat-network/docs/overview — Hardhat Network
- hardhat.org/ignition/docs/getting-started — Hardhat Ignition

OpenZeppelin — безопасные контракты

OpenZeppelin — безопасные смарт-контракты

OpenZeppelin Contracts — самая популярная и проверенная библиотека смарт-контрактов для Ethereum и EVM-совместимых блокчейнов. Прошедшие аудит реализации стандартов (ERC-20,

ERC-721, ERC-1155), контроль доступа, утилиты безопасности и обновляемые контракты. Де-факто стандарт индустрии.

Документация: docs.openzeppelin.com | **GitHub:** [OpenZeppelin/openzeppelin-contracts](https://github.com/OpenZeppelin/openzeppelin-contracts) | **Форум:** forum.openzeppelin.com

Уровень 1: Зачем нужен OpenZeppelin

В блокчейне нельзя просто нажать «Ctrl+Z». Транзакция выполнена — и навсегда. Баг в смарт-контракте = деньги потеряны безвозвратно.

Вспомни:

- **The DAO hack (2016)** — \$60М украдено из-за reentrancy-атаки
- **Parity wallet (2017)** — \$280М заморожено навсегда из-за бага в multisig
- **Wormhole bridge (2022)** — \$326М украдено

OpenZeppelin решает эту проблему — даёт тебе контракты, которые:

- **Прошли аудит** профессиональных security-фирм (каждый релиз!)
- **Следуют EIP/ERC стандартам** — твой токен будут понимать все кошельки и биржи
- **Покрываются тестами** на 100% (почти каждый контракт)
- **Используются в продакшене** крупнейшими проектами: Uniswap, Aave, Compound, Optimism, Arbitrum

Аналогия: как не строить дом с нуля каждый раз, а взять проверенные стройматериалы, которые инженеры уже испытали на прочность.

Установка

```
# npm (Hardhat, стандарт)
npm install @openzeppelin/contracts

# Foundry (git)
forge install OpenZeppelin/openzeppelin-contracts
```

В коде — просто импортируешь и наследуешь:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {
    constructor(uint256 initialSupply) ERC20("MyToken", "MTK") {
        _mint(msg.sender, initialSupply);
    }
}
```

Как читать `contract MyToken is ERC20 { constructor(uint256 initialSupply) ERC20("MyToken", "MTK") { _mint(msg.sender, initialSupply); } }`: «создай свой токен, унаследовав ВСЮ стандартную логику ERC-20: в конструкторе передай имя и символ родительскому контракту, затем напечатай стартовый запас на кошелёк деплойера». Мнемоника: `is ERC20` = скопировал весь стандарт бесплатно, `_mint` = напечатал токены из воздуха.

Готово. Твой токен совместим с MetaMask, Etherscan, Uniswap и всеми ERC-20-инструментами.

Уровень 2: Основные модули

1. ERC-20: взаимозаменяемые токены

Стандарт для «обычных» токенов — как USDT, UNI, LINK. Каждый токен равен другому (fungible).

Ключевой контракт: `ERC20.sol`

```
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract GoldToken is ERC20 {
    constructor() ERC20("Gold", "GLD") {
        _mint(msg.sender, 1000 * 10 ** decimals());
    }
}
```

Важные расширения ERC-20:

Расширение	Зачем
ERC20Burnable	Возможность сжигать токены (burn)
ERC20Capped	Ограничение максимального supply
ERC20Pausable	Админ может приостановить трансферы
ERC20Permit	Безгазовые approval через подписи (EIP-2612)
ERC20Votes	Токен как голос в governance
ERC20FlashMint	Flash-loans (займи и верни в одной транзакции)

Decimals: По умолчанию 18 (как у ETH). Если хочешь изменить — переопредели `decimals()`.

Связано: [wiki/ERC-20-стандарт-токенов](https://wiki.erc-20-standard.com/)

2. ERC-721: NFT (невзаимозаменяемые токены)

Каждый токен уникален — `tokenId`. Идеально для коллекционных предметов, игровых айтемов, билетов.

Ключевой контракт: `ERC721.sol`

```
import {ERC721URIStorage, ERC721} from "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

contract GameItem is ERC721URIStorage {
    uint256 private _nextTokenId;

    constructor() ERC721("GameItem", "ITM") {}

    function awardItem(address player, string memory tokenURI)
        public
        returns (uint256)
    {
        uint256 tokenId = _nextTokenId++;
        _mint(player, tokenId);
        _setTokenURI(tokenId, tokenURI);
        return tokenId;
    }
}
```

Важные расширения ERC-721:

Расширение	Зачем
ERC721URIStorage	Хранит метаданные каждого токена (JSON с image, name, attributes)
ERC721Enumerable	Возможность перебирать все токены владельца (дорого по газу!)
ERC721Burnable	Сжигание токенов
ERC721Pausable	Пауза трансферов
ERC721Royalty (EIP-2981)	Роялти создателю при перепродаже

tokenURI — ссылка на JSON-метаданные токена. Может быть:

- HTTP URL: `https://example.com/metadata/1.json`
- IPFS: `ipfs://Qm...`
- Data URI (on-chain): `data:application/json;base64,...`

Связано: [wiki/ERC-721-NFT-стандарт](#)

3. Access Control: кто что может делать

Ownable — простейший контроль

Самый базовый паттерн: у контракта есть **owner** — один адрес с особыми правами.

```
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract MyContract is Ownable {
    constructor(address initialOwner) Ownable(initialOwner) {}

    function normalThing() public {
        // кто угодно может вызвать
    }

    function adminThing() public onlyOwner {
        // только owner!
    }
}
```

- `transferOwnership(newOwner)` — передать владение другому
- `renounceOwnership()` — отказаться от владения навсегда
- **Ownable2Step** — safer-версия: новый owner должен подтвердить принятие (`acceptOwnership()`)

Проблема Ownable: один ключ = одна точка отказа. Для серьёзных проектов используйте RBAC.

AccessControl — ролевой доступ (RBAC)

Role-Based Access Control: роли как «должности» с разными правами.

```

import {AccessControl} from "@openzeppelin/contracts/access/
AccessControl.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract TokenWithRoles is ERC20, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant BURNER_ROLE = keccak256("BURNER_ROLE");

    constructor(address minter, address burner)
        ERC20("MyToken", "MTK")
    {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(MINTER_ROLE, minter);
        _grantRole(BURNER_ROLE, burner);
    }

    function mint(address to, uint256 amount) public
    onlyRole(MINTER_ROLE) {
        _mint(to, amount);
    }

    function burn(address from, uint256 amount) public onlyRole(BURNER_R
    OLE) {
        _burn(from, amount);
    }
}

```

Как читать `bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE")`: «преврати человекочитаемое имя роли в уникальный 32-байтовый идентификатор — хеш, который Solidity будет использовать для проверок `onlyRole(MINTER_ROLE)`». Мнемоника: `keccak256("ИМЯ_РОЛИ")` = строку в `bytes32`; роль — это просто хеш, а не enum.

Ключевые концепты:

- `DEFAULT_ADMIN_ROLE` — супер-админ, может назначать/снимать любые роли
- Роль = `bytes32` (хеш от названия, например `keccak256("MINTER_ROLE")`)
- Каждый адрес может иметь несколько ролей
- `grantRole(role, account) / revokeRole(role, account)` — динамическое управление
- **AccessControlDefaultAdminRules** — безопасная версия с 2-шаговой передачей админа и задержкой

TimelockController — прокси с временной задержкой. Действия проходят стадии: предложение → ожидание (например, 48 часов) →

исполнение. Пользователи успевают выйти, если им не нравится изменение.

4. Upgradeable-контракты: обновление логики

Смарт-контракты в блокчейне неизменяемы... если не использовать **proxy-паттерн**.

Идея: разделяем контракт на две части:

- **Proxy** — хранит состояние (балансы, storage), его адрес не меняется
- **Implementation** — содержит логику, может быть заменён на новую версию

OpenZeppelin поддерживает два типа прокси:

Тип	Как работает	Плюсы	Минусы
Transparent Proxy	Админ-адрес всегда вызывает через проху; пользователи — напрямую	Простой, проверенный	Дороже gas, админ проверяется каждый вызов
UUPS (Universal Upgradeable Proxy Standard)	Логика обновления в самом implementation	Дешевле gas (проверка только при upgrade)	Сложнее в реализации, риск забыть вызвать upgrade-функцию

Установка плагина Hardhat для upgrades:

```
npm install --save-dev @openzeppelin/hardhat-upgrades
```

Деплой через UUPS:

```
// V1
import {UUPSUpgradeable} from "@openzeppelin/contracts-upgradeable/proxy/
utils/UUPSUpgradeable.sol";
import {OwnableUpgradeable} from "@openzeppelin/contracts-upgradeable/
access/OwnableUpgradeable.sol";

contract MyContractV1 is Initializable, UUPSUpgradeable, OwnableUpgradea
ble {
    function initialize() public initializer {
        __Ownable_init(msg.sender);
        __UUPSUpgradeable_init();
    }

    function _authorizeUpgrade(address newImplementation)
        internal
        override
        onlyOwner
    {}
}
```

```
// deploy.js (Hardhat)
const { ethers, upgrades } = require("hardhat");

const MyContract = await ethers.getContractFactory("MyContractV1");
const proxy = await upgrades.deployProxy(MyContract, [], {
  kind: "uups",
  initializer: "initialize",
});
await proxy.waitForDeployment();
```

Как читать `upgrades.deployProxy(MyContract, [], { kind: "uups", initializer: "initialize" })`: «задеплой прокси-контракт, который делегирует все вызовы на логику *MyContract*, а начальную инициализацию сделай через функцию *initialize* — потому что у обновляемых контрактов конструктор не работает». Мнемоника: прокси = вечная обёртка с неизменным адресом, реализация = сменная начинка, *initializer* = конструктор на стероидах.

Важные правила `upgradeable`-контрактов:

- **Не используй конструктор** — используй `initialize()` с модификатором `initializer`
- **Не меняй порядок переменных** в `storage` между версиями
- **Не удаляй переменные** — только добавляй новые в конец
- **Пакет `@openzeppelin/contracts-upgradeable`** — специальные версии, где конструкторы заменены на `__Xxx_init()`

5. SafeERC20: безопасная работа с токенами

Стандарт ERC-20 не требует, чтобы `transfer` и `transferFrom` возвращали `bool`. Некоторые токены (например, USDT) не возвращают ничего. Без SafeERC20 твой контракт может сломаться на таких токенах.

```
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/
SafeERC20.sol";

contract Vault {
    using SafeERC20 for IERC20;

    > **Как читать `using SafeERC20 for IERC20;` + `token.safeTransfer(to, a
mount)`:** «прикрепи безопасные обёртки к стандартному интерфейсу
токена: теперь `safeTransfer` работает даже с „неправильными“ токенами в
роде USDT, которые не возвращают `bool` после перевода». Мнемоника: `Saf
eERC20` = страховка от кривых реализаций ERC-20; всегда используй `safet
ransfer`, а не `transfer`.

    function withdraw(IERC20 token, address to, uint256 amount) external
    {
        // Безопасно: работает и с USDT, и с DAI
        token.safeTransfer(to, amount);
    }
}
```

Методы SafeERC20:

- safeTransfer(token, to, amount)
- safeTransferFrom(token, from, to, amount)
- safeApprove(token, spender, amount) — безопаснее, чем обычный approve
- safeIncreaseAllowance(token, spender, amount) — избегает race condition approve-фронтраннинга
- safeDecreaseAllowance(token, spender, amount)

Правило: всегда используй SafeERC20 при взаимодействии с внешними токенами. Никогда не делай token.transfer() напрямую.

6. Утилиты

Address — проверки адресов и вызовы

```
import {Address} from "@openzeppelin/contracts/utils/Address.sol";

// Проверить, что адрес – контракт
Address.isContract(someAddress);

// Отправить ETH через call (безопаснее, чем .transfer())
Address.sendValue(payable(recipient), amount);

// Вызвать функцию контракта – вернёт ошибку с bubbling
Address.functionCall(target, data);
Address.functionDelegateCall(target, data);
```

Strings — работа со строками

```
import {Strings} from "@openzeppelin/contracts/utils/Strings.sol";

Strings.toString(123);           // "123"
Strings.toHexString(address);    // "0xabcd..."
Strings.toHexString(42);          // "0x2a"
Strings.equal(str1, str2);        // сравнение строк (газ-эффективно)
```

Math — безопасная арифметика

Solidity 0.8+ имеет встроенные overflow-проверки, но OpenZeppelin даёт дополнительные утилиты:

```
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

Math.max(a, b);                  // максимум
Math.min(a, b);                  // минимум
Math.average(a, b);              // среднее без overflow
Math.ceilDiv(a, b);              // деление с округлением вверх
Math.sqrt(x);                    // квадратный корень
Math.log2(x);                    // логарифм по основанию 2
Math.log10(x);                   // логарифм по основанию 10
```

SignedMath — abs, average, max, min для int256.

7. Безопасность (Security)

ReentrancyGuard — защита от повторных вызовов

Reentrancy — атака #1 в смарт-контрактах. Злоумышленник вызывает твою функцию, она вызывает его контракт, а тот — снова твою функцию до завершения первой.

```
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
ReentrancyGuard.sol";

contract Vault is ReentrancyGuard {
    mapping(address => uint256) public balances;

    function withdraw() external nonReentrant {
        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0;

        (bool success, ) = msg.sender.call{value: amount}("");
        require(success);
        // Без nonReentrant: атакующий мог бы повторно вызвать withdraw
        // через fallback до обнуления balances
    }
}
```

- nonReentrant — блокирует повторный вход в ту же функцию
- _nonReentrantBefore() / _nonReentrantAfter() — ручное управление для сложных случаев

Pausable — экстренная пауза

Возможность приостановить критические операции. Например, если обнаружен баг.

```
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract MyToken is ERC20, Pausable {
    constructor() ERC20("MyToken", "MTK") {}

    function transfer(address to, uint256 amount)
        public
        override
        whenNotPaused
        returns (bool)
    {
        return super.transfer(to, amount);
    }

    function pause() external onlyOwner {
        _pause(); // эмитит Paused
    }

    function unpause() external onlyOwner {
        _unpause(); // эмитит Unpaused
    }
}
```

Модификаторы:

- whenNotPaused — требует, чтобы контракт был активен
- whenPaused — требует, чтобы контракт был на паузе

Уровень 3: Продвинутые темы

ERC-20 Permit — безгазовые approval

Обычный процесс: `approve(spender, amount)` → жди транзакцию → `transferFrom`. ERC-20 Permit заменяет первую транзакцию подписью вне цепи (EIP-2612).

```
import {ERC20Permit} from "@openzeppelin/contracts/token/ERC20/
extensions/ERC20Permit.sol";

contract MyToken is ERC20, ERC20Permit {
    constructor() ERC20("MyToken", "MTK") ERC20Permit("MyToken") {}
}
```

Пользователь подписывает сообщение (бесплатно), а кто угодно может отправить `permit()` — и `approval` готов. Экономия газа для пользователя.

ERC-1155 — мульти-токены

Один контракт управляет и fungible, и non-fungible токенами. Один NFT и 1000 золотых монет — в одном контракте.

```
import {ERC1155} from "@openzeppelin/contracts/token/ERC1155/
ERC1155.sol";

contract GameItems is ERC1155 {
    uint256 public constant GOLD = 0;
    uint256 public constant SWORD = 1;

    constructor() ERC1155("https://game.example/api/item/{id}.json") {
        _mint(msg.sender, GOLD, 1000, ""); // 1000 золотых (fungible)
        _mint(msg.sender, SWORD, 1, ""); // 1 меч (NFT, quantity=1)
    }
}
```

Преимущество: `safeBatchTransferFrom` — отправить несколько типов токенов в одной транзакции.

Контракты Governance

OpenZeppelin даёт полный набор для DAO-голосований:

- Governor + GovernorCountingSimple — создание и подсчёт `proposal`'ов
- GovernorVotes — голосование токенами (привязка к `ERC20Votes`)
- GovernorTimelockControl — интеграция с `TimelockController`
- Tally — off-chain подсчёт голосов (экономия газа)

Wizard — генератор контрактов

Не хочешь писать руками? wizard.openzeppelin.com — выбери ERC-20/ERC-721/ERC-1155/Governor, отметь галками фичи, получи готовый код.

Уровень 4: Architectural Decisions

Почему наследование, а не библиотеки?

OpenZeppelin Contracts — **фреймворк на наследовании**. Ты наследуешь контракт и опционально переопределяешь функции (virtual / override). Не библиотеки, которые вызываешь через delegatecall. Причины:

- **Газовая эффективность:** прямые вызовы дешевле delegatecall
- **Прозрачность:** код твоего контракта — это весь код (Flat, проверяемый в Etherscan)
- **Компилятор всё проверяет:** никаких сюрпризов с layout'ом памяти

Аудит и версионирование

OpenZeppelin использует **семантическое версионирование**:

- @openzeppelin/contracts@5.x — MAJOR: breaking changes (меняют storage layout)
- @openzeppelin/contracts@5.0.x — MINOR: новые фичи, обратная совместимость
- NPM-теги: latest (аудирован), dev (финальный, но без аудита), next (в разработке)

Каждый major-релиз проходит независимый аудит. Баунти-программа на Immunefi.

Выбор: Transparent Proxy vs UUPS

Transparent Proxy:

User → Proxy → Implementation

Admin → Proxy → ProxyAdmin → Implementation (дороже, но проще)

UUPS:

User → Proxy → Implementation (как обычно)

Upgrade: Proxy → Implementation.upgradeTo(...) (логика в implementation)

Когда UUPS:

- Хочешь экономить газ
- Готов следить, чтобы _authorizeUpgrade не потерялся при наследовании

Когда Transparent:

- Предпочитаешь простоту и проверенность
- Не критично дороже на ~2100 gas за вызов

Принцип наименьших привилегий

Не давай DEFAULT_ADMIN_ROLE всем подряд. Используй granular-роли:

DEFAULT_ADMIN_ROLE:	multisig (2-of-3), только назначает роли
MINTER_ROLE:	контракт бриджа (автоматический)
BURNER_ROLE:	multisig (ручное сжигание)
PAUSER_ROLE:	дежурный бот (может поставить паузу)

Быстрый старт (чит-лист)

```
# 1. Установка
npm install @openzeppelin/contracts

# 2. ERC-20 токен
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

# 3. ERC-721 NFT
import {ERC721URIStorage} from "@openzeppelin/contracts/token/ERC721/
extensions/ERC721URIStorage.sol";

# 4. Контроль доступа
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {AccessControl} from "@openzeppelin/contracts/access/
AccessControl.sol";

# 5. Безопасность
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
ReentrancyGuard.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/
SafeERC20.sol";

# 6. Upgradeable
npm install @openzeppelin/contracts-upgradeable @openzeppelin/hardhat-
upgrades

# 7. Wizard (генератор кода)
# https://wizard.openzeppelin.com/
```

Связанные страницы

- [wiki/ERC-20-стандарт-токенов](#) — детальный разбор стандарта ERC-20
- [wiki/ERC-721-NFT-стандарт](#) — детальный разбор NFT-стандарта
- [wiki/Solidity-основы](#) — синтаксис Solidity, необходимый для понимания контрактов
- [wiki/Hardhat-среда-разработки](#) — среда разработки, в которой используется OpenZeppelin

Источники

- 1. [OpenZeppelin Contracts 5.x Documentation](#) — официальная документация
- 2. [OpenZeppelin Access Control Guide](#) — гайд по Ownable и AccessControl
- 3. [OpenZeppelin Tokens Guide](#) — ERC-20/ERC-721/ERC-1155
- 4. [OpenZeppelin ERC-20 Guide](#)
- 5. [OpenZeppelin ERC-721 Guide](#)
- 6. [OpenZeppelin Upgrades Plugins](#) — Transparent/UUPS proxy
- 7. [OpenZeppelin GitHub Repository](#)
- 8. [EIP-1967: Proxy Storage Slots](#) — стандарт прокси-слотов

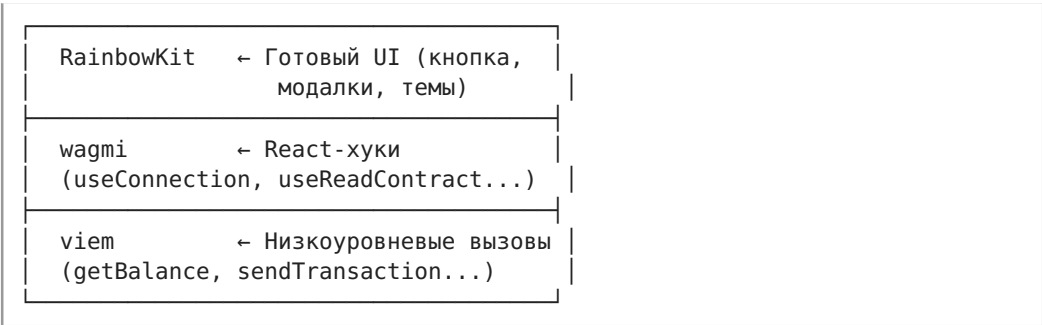
wagmi + RainbowKit — фронтенд для dApps

wagmi + RainbowKit: фронтенд для dApps

Современный стек для React-фронтенда web3-приложений. wagmi даёт React-хуки для чтения и записи в блокчейн, RainbowKit — готовый UI для подключения кошелька, viem — низкоуровневый движок под капотом.

Актуальность: июль 2026, wagmi v3, viem v2.x, RainbowKit v2.x. API проверен по официальной документации.

Связь wagmi, RainbowKit и viem



- **viem** — лёгкая TypeScript-библиотека для JSON-RPC запросов к Ethereum (аналог ethers.js, но современнее). Создана той же командой (wevm).
- **wagmi** — React-хуки поверх viem. Даёт useConnection, useReadContract, useWriteContract и ещё 60+ хуков.
- **RainbowKit** — UI-библиотека поверх wagmi. Кнопка «Connect Wallet», модалка выбора кошелька, смена сети — из коробки.

Мнемоника: `viem` — двигатель, `wagmi` — руль, `RainbowKit` — приборная панель.

Уровень 1. `wagmi`: **React-хуки**

Установка и настройка

```
npm install wagmi viem@2.x @tanstack/react-query
```

config.ts — создаём конфигурацию:

```
import { createConfig, http } from 'wagmi'
import { mainnet, sepolia } from 'wagmi/chains'
import { injected, walletConnect } from 'wagmi/connectors'

const projectId = 'BAШ_WALLETCONNECT_PROJECT_ID' // бесплатно на
cloud.walletconnect.com

export const config = createConfig({
  chains: [mainnet, sepolia],
  connectors: [
    injected(), // MetaMask и другие браузерные
    walletConnect({ projectId }), // WalletConnect (мобильные)
  ],
  transports: {
    [mainnet.id]: http(), // публичный RPC
    [sepolia.id]: http(),
  },
})
```

App.tsx — оборачиваем приложение в провайдеры:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { WagmiProvider } from 'wagmi'
import { config } from './config'

const queryClient = new QueryClient()

function App() {
  return (
    <WagmiProvider config={config}>
      <QueryClientProvider client={queryClient}>
        {/* Ваше приложение */}
      </QueryClientProvider>
    </WagmiProvider>
  )
}
```

TanStack Query обязателен в `wagmi v3` — он отвечает за кэширование, дедупликацию и автоматический рефетч.

useConnection — статус подключения

На смену useAccount (wagmi v2) пришёл useConnection (wagmi v3).

Возвращает адрес, сеть, статус подключения.

```
import { useConnection } from 'wagmi'

function AccountInfo() {
  const { address, chain, isConnected, status } = useConnection()

  if (status === 'connecting') return <div>Подключение...</div>
  if (!isConnected) return <div>Кошелёк не подключён</div>

  > **Как читать `const { address, chain, isConnected, status } = useConnection():`** «узнай состояние кошелька одним хуком: подключён ли, какой адрес, в какой сети, и что сейчас происходит — `connecting`, `connected` или `disconnected`». Мнемоника: `useConnection` = датчик кошелька: кто подключён, где находится, жив ли.

  return (
    <div>
      <p>Адрес: {address}</p>
      <p>Сеть: {chain?.name} (chainId: {chain?.id})</p>
    </div>
  )
}
```

Ключевые поля useConnection():

Поле	Тип	Описание
address	Address \ undefined	Адрес подключённого кошелька
addresses	Address[] \ undefined	Все адреса (если кошелёк даёт несколько)
chain	Chain \ undefined	Текущая сеть (mainnet, sepolia...)
chainId	number \ undefined	ID сети
connector	Connector \ undefined	Текущий коннектор
status	'connecting' \ 'reconnecting' \ 'connected' \ 'disconnected'	Статус
isConnected	boolean	Удобный флаг

Type narrowing: когда `status === 'connected'`, поля `address` и `chain` гарантированно определены.

useConnect — подключить кошелёк

В wagmi v3 useConnect — это **TanStack Query mutation**. Можно использовать через деструктуризацию { connect, connectors }:

```
import { useConnect, useConnectors } from 'wagmi'

function WalletOptions() {
  const { connect } = useConnect()
  const connectors = useConnectors()

  return (
    <div>
      {connectors.map((connector) => (
        <button key={connector.uid} onClick={() => connect({ connector })}
      )}
      {connector.name}
    </button>
  ))}
    </div>
  )
}
```

Полный API useConnect():

- connect({ connector, chainId? }) — вызвать подключение
- connectors (deprecated, используй useConnectors()) — список доступных коннекторов
- isPending, isSuccess, isError — статус мутации
- error — ошибка подключения
- data — { accounts, chainId } после успешного подключения

Типы коннекторов (из wagmi/connectors):

```
import { injected, walletConnect, metaMask, coinbaseWallet, safe } from '
wagmi/connectors'

// injected()      — любой браузерный кошелёк (MetaMask, Brave, Rabby)
// metaMask()      — только MetaMask (EIP-6963)
// walletConnect() — мобильные кошельки через QR-код
// coinbaseWallet() — Coinbase Wallet
// safe()          — Safe (multisig)
```

useBalance — чтение баланса нативного токена

```
import { useBalance, useConnection } from 'wagmi'
import { formatEther } from 'viem'

function Balance() {
  const { address } = useConnection()
  const { data, isError, isLoading } = useBalance({
    address,
  })

  if (isLoading) return <div>Загрузка...</div>
  if (isError) return <div>Ошибка загрузки баланса</div>

  return (
    <div>
      Баланс: {formatEther(data!.value)} {data!.symbol}
    </div>
  )
}
```

useBalance возвращает { *value: bigint, symbol: string, decimals: number* }. *formatEther()* из *viem* конвертирует *wei* (*bigint*) в читаемую строку.

useReadContract — чтение из смарт-контракта

Для *view/pure* функций смарт-контракта. Не требует газа, работает как TanStack Query.

```
import { useReadContract } from 'wagmi'
import { erc20Abi } from 'viem'

function TokenBalance() {
  const { address } = useConnection()

  const { data: balance } = useReadContract({
    address: '0x6B175474E89094C44Da98b954EedeAC495271d0F', // DAI
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: [address!],
    query: {
      enabled: !!address, // не запрашивать, пока нет адреса
    },
  })

  return <div>Токенов: {balance?.toString()}</div>
}
```

Как читать `useReadContract``{ address, abi, functionName, args, query: { enabled: !!address } }`): «прочитай данные из смарт-контракта React-хуком: вот адрес контракта, вот его ABI-интерфейс, вот какую view-функцию вызвать и с какими аргументами; `enabled: !!address` откладывает запрос до появления кошелька». Мнемоника: `useReadContract` = бесплатный GET в блокчейн через React Query — кэшируется, рефетчится, не платишь газ.

Подробнее о параметрах:

- `address` — адрес контракта
- `abi` — ABI контракта (из `viem` есть готовые: `erc20Abi`, `erc721Abi`)
- `functionName` — имя функции
- `args` — аргументы (типы выводятся из ABI)
- `blockNumber / blockTag` — на каком блоке читать
- `account` — адрес для `msg.sender` (опционально)
- `query.enabled` — отключить авто-запрос (полезно, пока нет адреса)

Типизация из ABI: если передать правильно типизированный ABI, TypeScript сам выведет типы аргументов и возвращаемого значения.

`useWriteContract` — отправка транзакции к контракту

Для функций, меняющих состояние блокчейна. Это мутация — требует подтверждения пользователем в кошельке.

```

import { useWriteContract, useWaitForTransactionReceipt } from 'wagmi'
import { erc20Abi } from 'viem'

function TransferToken() {
  const { data: hash, isPending, writeContract } = useWriteContract()

  const { isLoading: isConfirming, isSuccess: isConfirmed } =
    useWaitForTransactionReceipt({ hash })

  return (
    <div>
      <button
        disabled={isPending}
        onClick={() =>
          writeContract({
            address: '0x6B175474E89094C44Da98b954EedeAC495271d0F',
            abi: erc20Abi,
            functionName: 'transfer',
            args: ['0xRecipientAddress...', 1000000000000000000n], // 1
            token в wei
          })
        }
      >
        {isPending ? 'Подтвердите в кошельке...' : 'Отправить 1 DAI'}
      </button>

      {hash && <div>Хеш транзакции: {hash}</div>}
      {isConfirming && <div>Ожидание подтверждения...</div>}
      {isConfirmed && <div>Транзакция подтверждена!</div>}
    </div>
  )
}

```

Как читать связку `writeContract({ address, abi, functionName, args })` $\rightarrow$ `useWaitForTransactionReceipt({ hash })`: «пошли транзакцию к смарт-контракту — кошелёк запросит подпись, ты получишь хеш; затем передай этот хеш в `useWaitForTransactionReceipt`, чтобы React-хук сам опрашивал сеть и сообщил, когда транзакция попала в блок».

Мнемоника: `writeContract` = платный POST $\rightarrow$ хеш $\rightarrow$ `useWaitForTransactionReceipt` = жди квитанцию.

Паттерн: `useWriteContract` $\rightarrow$ получаем `hash` $\rightarrow$ передаём в `useWaitForTransactionReceipt` $\rightarrow$ ждём подтверждения.

Альтернатива — `useWriteContractSync`: ждёт включения транзакции в блок синхронно (блокирует UI до майнинга). Используй редко.

useWaitForTransactionReceipt — ждать подтверждения

Ждёт, пока транзакция попадёт в блок, и возвращает receipt.

```
import { useWaitForTransactionReceipt } from 'wagmi'

function TransactionStatus({ hash }: { hash: `0x${string}` }) {
  const { data: receipt, isPending, isSuccess, isError, error } =
    useWaitForTransactionReceipt({
      hash,
      confirmations: 2, // ждать 2 блока для надёжности
    })

  if (isPending) return <div>Ожидание...</div>
  if (isError) return <div>Ошибка: {error.message}</div>
  if (isSuccess) return <div> Блок: {receipt.blockNumber.toString()}</div>
  return null
}
```

Параметры:

- hash — хеш транзакции
- confirmations — сколько блоков ждать (по умолчанию 1)
- pollingInterval — интервал опроса в мс
- onReplaced — колбэк, если транзакцию заменили (sped up)

useDisconnect — отключить кошелёк

```
import { useDisconnect } from 'wagmi'

function DisconnectButton() {
  const { disconnect } = useDisconnect()
  return <button onClick={() => disconnect()}>Отключить</button>
}
```


Полезные хуки (краткий справочник)

Хук	Назначение
useBalance	Баланс нативного токена (ETH, MATIC...)
useReadContract	Чтение view-функции контракта
useReadContracts	Множественное чтение (батч)
useWriteContract	Отправка транзакции к контракту
useWaitForTransactionReceipt	Ожидание подтверждения транзакции
useSendTransaction	Простая отправка ETH (не контракт)
useSignMessage	Подпись сообщения
useSignTypedData	Подпись типизированных данных (EIP-712)
useSwitchChain	Переключение сети
useWatchContractEvent	Подписка на события контракта
useEnsName / useEnsAvatar	ENS-имена и аватары
useBlockNumber	Текущий номер блока
useSimulateContract	Симуляция вызова до отправки
useEstimateGas	Оценка газа
useChainId	Текущий chainId
useConnectors	Список доступных коннекторов
useDisconnect	Отключение кошелька

Уровень 2. RainbowKit: готовый UI для Connect Wallet

Установка

```
npm install @rainbow-me/rainbowkit wagmi viem@2.x @tanstack/react-query
```

Настройка (с getDefaultConfig)

RainbowKit предоставляет getDefaultConfig() — обёртку над wagmi createConfig, которая автоматически добавляет популярные коннекторы.

```
import '@rainbow-me/rainbowkit/styles.css'
import { getDefaultConfig, RainbowKitProvider } from '@rainbow-me/rainbowkit'
import { WagmiProvider } from 'wagmi'
import { mainnet, sepolia, optimism, arbitrum, base, polygon } from 'wagmi/chains'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

const config = getDefaultConfig({
  appName: 'Мой dApp',
  projectId: 'BAШ_WALLETCONNECT_PROJECT_ID',
  chains: [mainnet, sepolia, optimism, base],
  ssr: true, // если SSR (Next.js)
})

> Как читать `getDefaultConfig({ appName, projectId, chains, transports?, ssr? })`: «собери всю конфигурацию RainbowKit одной функцией: имя приложения, WalletConnect ID, список сетей – и получи готовый wagmi-конфиг с автоподключением MetaMask, WalletConnect и Coinbase». Мнемоника: `getDefaultConfig` = `createConfig` + все популярные коннекторы из коробки.

const queryClient = new QueryClient()

function App() {
  return (
    <WagmiProvider config={config}>
      <QueryClientProvider client={queryClient}>
        <RainbowKitProvider>
          {/* Ваше приложение */}
        </RainbowKitProvider>
      </QueryClientProvider>
    </WagmiProvider>
  )
}
```

ConnectButton — кнопка подключения

```
import { ConnectButton } from '@rainbow-me/rainbowkit'

export default function Header() {
  return (
    <header>
      <h1>Мой dApp</h1>
      <ConnectButton />
    </header>
  )
}
```

Что даёт ConnectButton из коробки:

- Кнопка «Connect Wallet» (если не подключён)

- Адрес + аватар + баланс (если подключён)
- Модалка выбора кошелька (MetaMask, WalletConnect, Coinbase...)
- Переключение сети
- Кнопка «Disconnect»

Кастомизация ConnectButton:

```
<ConnectButton
  label="Войти"                                // свой текст кнопки
  accountStatus="address"                      // только адрес, без аватара
  chainStatus="icon"                          // только иконка сети
  showBalance={false}                         // скрыть баланс
/>

{/* Адаптивная настройка: */}
<ConnectButton
  accountStatus={{
    smallScreen: 'avatar',                    // на телефоне только аватар
    largeScreen: 'full',                     // на десктопе адрес + аватар
  }}
  showBalance={{
    smallScreen: false,
    largeScreen: true,
  }}
/>
```

Темы и кастомизация

RainbowKit поставляется с тремя встроенными темами:

```
import { lightTheme, darkTheme, midnightTheme } from '@rainbow-me/
rainbowkit'

<RainbowKitProvider theme={darkTheme()}>
```

Кастомизация темы:

```
<RainbowKitProvider
  theme={darkTheme({
    accentColor: '#7b3fe4',                  // основной цвет
    accentColorForeground: 'white',          // цвет текста на акценте
    borderRadius: 'medium',                 // large | medium | small | none
    fontStack: 'system',                    // rounded (default) | system
    overlayBlur: 'small',                   // none (default) | small
  })}
>
```

Готовые пресеты акцентных цветов:

```
darkTheme({ ...darkTheme.accentColors.pink })
darkTheme({ ...darkTheme.accentColors.green })
darkTheme({ ...darkTheme.accentColors.purple })
darkTheme({ ...darkTheme.accentColors.orange })
darkTheme({ ...darkTheme.accentColors.red })
darkTheme({ ...darkTheme.accentColors.blue })
```

Поддержка тёмной/светлой темы (авто):

```
<RainbowKitProvider
  theme={{
    lightMode: lightTheme(),
    darkMode: darkTheme(),
  }}
>
```

Цепочки (chains)

RainbowKit через wagmi поддерживает все EVM-совместимые цепочки из коробки:

```
import {
  mainnet, sepolia, holesky,
  optimism, optimismSepolia,
  arbitrum, arbitrumSepolia,
  base, baseSepolia,
  polygon, polygonMumbai,
  avalanche, avalancheFuji,
  bsc, bscTestnet,
  gnosis, gnosisChiado,
  // ... десятки других
} from 'wagmi/chains'
```

Добавление кастомной цепи:

```
import { defineChain } from 'viem'

export const myL2 = defineChain({
  id: 12345,
  name: 'My L2',
  nativeCurrency: { name: 'Ether', symbol: 'ETH', decimals: 18 },
  rpcUrls: {
    default: { http: ['https://rpc.myl2.io'] },
  },
  blockExplorers: {
    default: { name: 'MyScan', url: 'https://scan.myl2.io' },
  },
})
```

Кастомные RPC-провайдеры (рекомендуется для продакшена):

```
const config = getDefaultConfig({
  appName: 'Мой dApp',
  projectId: '...',
  chains: [mainnet, sepolia],
  transports: {
    [mainnet.id]: http('https://eth-mainnet.g.alchemy.com/v2/ВАШ_КЛЮЧ'),
    [sepolia.id]: http('https://eth-sepolia.g.alchemy.com/v2/ВАШ_КЛЮЧ'),
  },
})
```

Уровень 3. viem: низкоуровневые операции

Хотя wagmi покрывает 95% потребностей, иногда нужен прямой доступ к viem. Он уже установлен как зависимость wagmi.

Чтение баланса (напрямую через viem)

```
import { createPublicClient, http, formatEther } from 'viem'
import { mainnet } from 'viem/chains'

const publicClient = createPublicClient({
  chain: mainnet,
  transport: http(),
})

const balance = await publicClient.getBalance({
  address: '0xA0Cf798816D4b9b9866b5330EEa46a18382f251e',
})

console.log(formatEther(balance)) // '6.942'
```

Через wagmi (рекомендуется):

```
import { useBalance } from 'wagmi'
// useBalance уже использует publicClient.getBalance под капотом
```

Отправка транзакции (напрямую через viem)

```
import { createWalletClient, custom, parseEther } from 'viem'
import { mainnet } from 'viem/chains'

const walletClient = createWalletClient({
  chain: mainnet,
  transport: custom(window.ethereum!),
})

const hash = await walletClient.sendTransaction({
  account: '0x...',
  to: '0x70997970c51812dc3a010c7d01b50e0d17dc79c8',
  value: parseEther('1'), // 1 ETH в wei
})
```

Через wagmi (рекомендуется):

```
import { useSendTransaction } from 'wagmi'
import { parseEther } from 'viem'

const { sendTransaction } = useSendTransaction()
sendTransaction({ to: '0x...', value: parseEther('0.01') })
```

viem-утилиты (часто используемые)

```
import { formatEther, parseEther, formatUnits, parseUnits } from 'viem'

formatEther(1000000000000000000n)    // '1'
parseEther('1.5')                    // 1500000000000000000n
formatUnits(1000000n, 6)              // '1' (для USDC с 6 decimals)
parseUnits('100', 6)                 // 100000000n

// Конвертация адресов
import { getAddress, isAddress } from 'viem'
getAddress('0xa0cf...')               // checksummed address
isAddress('0x...')                    // boolean

// Хеширование
import { keccak256, encodePacked, toHex } from 'viem'
keccak256(toHex('Transfer(address,address,uint256)'))
```

Прямой вызов контракта через viem (если не хватает wagmi)

```
import { createPublicClient, http, getContract } from 'viem'
import { erc20Abi } from 'viem'
import { mainnet } from 'viem/chains'

const publicClient = createPublicClient({
  chain: mainnet,
  transport: http(),
})

// Вариант 1: через getContract
const contract = getContract({
  address: '0x6B175474E89094C44Da98b954EedeAC495271d0F',
  abi: erc20Abi,
  client: publicClient,
})

const totalSupply = await contract.read.totalSupply()

// Вариант 2: через readContract напрямую
const balance = await publicClient.readContract({
  address: '0x6B175474E89094C44Da98b954EedeAC495271d0F',
  abi: erc20Abi,
  functionName: 'balanceOf',
  args: ['0x...'],
})
```

Уровень 4. Практический гайд: от Connect Wallet до вызова смарт-контракта

Собираем всё вместе. Создадим dApp, который:

1. Подключает кошелёк (RainbowKit)
2. Показывает баланс ETH (wagmi)
3. Читает баланс ERC-20 токена (wagmi)
4. Отправляет перевод токенов (wagmi)
5. Ждёт подтверждения и показывает результат

Полный листинг (Next.js App Router)

1. Установка:

```
npx create-next-app@latest my-dapp
cd my-dapp
npm install @rainbow-me/rainbowkit wagmi viem@2.x @tanstack/react-query
```

2. src/config.ts — конфигурация:

```
import { getDefaultConfig } from '@rainbow-me/rainbowkit'
import { sepolia } from 'wagmi/chains'

export const config = getDefaultConfig({
  appName: 'Мой первый dApp',
  projectId: 'BAШ_PROJECT_ID',
  chains: [sepolia], // тестовая сеть
  ssr: true,
})
```

3. src/app/providers.tsx — провайдеры (клиентский компонент):

```
'use client'

import '@rainbow-me/rainbowkit/styles.css'
import { RainbowKitProvider } from '@rainbow-me/rainbowkit'
import { WagmiProvider } from 'wagmi'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { config } from '@config'
import { useState } from 'react'

export function Providers({ children }: { children: React.ReactNode }) {
  const [queryClient] = useState(() => new QueryClient())

  return (
    <WagmiProvider config={config}>
      <QueryClientProvider client={queryClient}>
        <RainbowKitProvider>
          {children}
        </RainbowKitProvider>
      </QueryClientProvider>
    </WagmiProvider>
  )
}
```

4. src/app/layout.tsx — корневой layout:

```
import { Providers } from '../providers'

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="ru">
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  )
}
```

5. src/app/page.tsx — главная страница (полный пример):


```
'use client'
```

```
import { ConnectButton } from '@rainbow-me/rainbowkit'
import {
  useConnection,
  useBalance,
  useReadContract,
  useWriteContract,
  useWaitForTransactionReceipt,
} from 'wagmi'
import { erc20Abi, formatEther, parseEther } from 'viem'
import { useState } from 'react'
```

```
// Адрес тестового токена в Sepolia (замените на свой)
const TOKEN_ADDRESS = '0x...' as `0x${string}`
```

```
function DAppContent() {
  const { address, isConnected } = useConnection()
  const [recipient, setRecipient] = useState('')
  const [amount, setAmount] = useState('')

  // 1. Баланс ETH
  const { data: ethBalance } = useBalance({ address })

  // 2. Баланс токена
  const { data: tokenBalance } = useReadContract({
    address: TOKEN_ADDRESS,
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: address ? [address] : undefined,
    query: { enabled: !!address },
  })

  // 3. Отправка токена
  const {
    data: txHash,
    isPending: isWriting,
    writeContract,
  } = useWriteContract()

  // 4. Ожидание подтверждения
  const {
    isLoading: isConfirming,
    isSuccess: isConfirmed,
  } = useWaitForTransactionReceipt({ hash: txHash })

  if (!isConnected) {
    return (
      <div style={{ textAlign: 'center', marginTop: '20vh' }}>
        <h1>Подключите кошелёк</h1>
      </div>
    )
  }
}
```

```

    <ConnectButton />
  </div>
)
}

const handleTransfer = () => {
  if (!recipient || !amount) return
  writeContract({
    address: TOKEN_ADDRESS,
    abi: erc20Abi,
    functionName: 'transfer',
    args: [recipient as `0x${string}`, parseEther(amount)],
  })
}

return (
  <div style={{ maxWidth: 600, margin: '0 auto', padding: 20 }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
      <h1>Мой dApp</h1>
      <ConnectButton />
    </div>

    <div style={{ marginTop: 30 }}>
      <h2>Балансы</h2>
      <p>ETH: {ethBalance ? formatEther(ethBalance.value) : '...'} {ethBalance?.symbol}</p>
      <p>Токен: {tokenBalance ? formatEther(tokenBalance) : '...'}</p>
    </div>

    <div style={{ marginTop: 30 }}>
      <h2>Перевод токенов</h2>
      <input
        placeholder="Адрес получателя (0x...)"
        value={recipient}
        onChange={(e) => setRecipient(e.target.value)}
        style={{ width: '100%', padding: 8, marginBottom: 10 }}
      />
      <input
        placeholder="Количество"
        type="number"
        value={amount}
        onChange={(e) => setAmount(e.target.value)}
        style={{ width: '100%', padding: 8, marginBottom: 10 }}
      />
      <button
        onClick={handleTransfer}
        disabled={isWriting || !recipient || !amount}
        style={{ padding: '10px 20px', cursor: 'pointer' }}
      >

```

```
        {isWriting ? 'Подтвердите в кошельке...' : 'Отправить'}
      </button>
    </div>

    {txHash && (
      <div style={{ marginTop: 20, wordBreak: 'break-all' }}>
        <p>Хеш транзакции: {txHash}</p>
        {isConfirming && <p> Ожидание подтверждения...</p>}
        {isConfirmed && <p> Транзакция подтверждена!</p>}
      </div>
    )}
  </div>
)
}

export default function Home() {
  return <DAppContent />
}
```

Блок-схема flow: от клика до confirmed

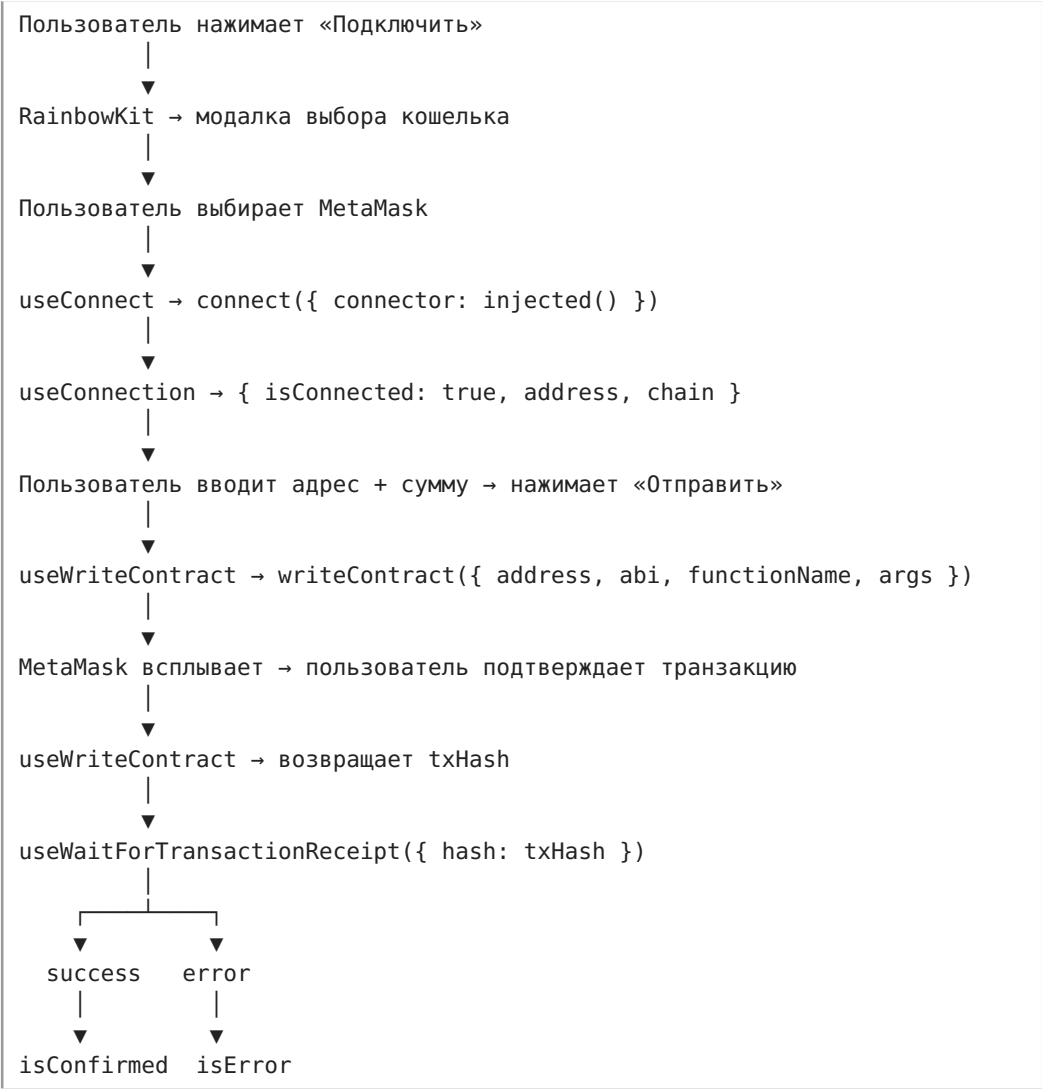


Таблица миграции: wagmi v2 → v3

v2 (старый)	v3 (текущий)
useAccount()	useConnection()
useConnect({ connector })	useConnect() → { connect }, connect({ connector })
useContractRead({ ... })	useReadContract({ ... })
useContractWrite({ ... })	useWriteContract() → writeContract({ ... })
useWaitForTransaction({ hash })	useWaitForTransactionReceipt({ hash })
Прямой вызов write()	.mutate() или деструктуризация { writeContract }
Не нужен TanStack Query	Обязателен QueryClientProvider

Проверяйте версию: `npm list wagmi`. Если v2 — мигрируйте на v3, в v2 больше нет смысла для новых проектов.

Best Practices (из опыта)

1. **query.enabled: !!address** — всегда отключайте запросы к контракту, пока нет адреса. Избегает лишних RPC-вызовов.
2. **useWaitForTransactionReceipt + confirmations: 2** — ждите минимум 2 блока для надёжности.
3. **useSimulateContract перед useWriteContract** — симулируйте вызов до отправки, чтобы поймать ошибки раньше (экономит газ).
4. **Кастомные RPC для продакшена** — публичные RPC троттлят. Alchemy, Infura, QuickNode.
5. **ssr: true в getDefaultConfig** если Next.js — избегает hydration mismatch.
6. **Не смешивайте wagmi и прямой viem для одного и того же** — wagmi кэширует запросы через TanStack Query, прямой вызов viem не попадёт в кэш.
7. **ERC-20 ABI из viem** — используйте erc20Abi из viem вместо ручного написания ABI.
8. **Обработка ошибок:** всегда показывайте isError и error.message — транзакции могут реджектиться по газу, nonce, slippage.

Частые ошибки

Ошибка	Причина	Решение
Connector not found	Не настроены connectors в config	Добавьте injected(), walletConnect()
TypeError: address is undefined	Хук вызван до подключения кошелька	Добавьте enabled: !! address
Missing projectId	WalletConnect требует projectId	Получите на cloud.walletconnect.com
Hydration mismatch (Next.js)	SSR не настроен	ssr: true в getDefaultConfig
window.ethereum is undefined	Нет установленного кошелька	Проверьте, что MetaMask установлен
Cannot read properties of undefined (reading 'call')	ABI не совпадает с контрактом	Проверьте ABI и адрес контракта

Связанное

- [wiki/Сравнение-ethers-viem-wagmi](#) — сравнение ethers.js, viem и wagmi
- [wiki/Solidity-основы](#) — пишем смарт-контракты, к которым подключается фронтенд
- [wiki/Главная](#) — дорожная карта изучения web3

Ссылки

- [wagmi.sh](#) — документация wagmi
- [rainbowkit.com](#) — документация RainbowKit
- [viem.sh](#) — документация viem
- [WalletConnect Cloud](#) — получить projectId
- [Alchemy](#) — RPC-провайдер

Сравнение ethers.js, viem, wagmi

Сравнение ethers.js, viem, wagmi

Три основных инструмента для фронтенда, чтобы общаться с блокчейном. Какой и когда использовать?

ethers.js v6

Что это: низкоуровневая библиотека для взаимодействия с Ethereum. Прямой аналог web3.js, но современнее и легче.

Что умеет:

- Подключаться к нодам через JSON-RPC
- Читать данные из контрактов (contract.balanceOf())

- Отправлять транзакции (`contract.transfer()`)
- Слушать события (`contract.on("Transfer", ...)`)
- Работать с кошельками, подписывать сообщения

Плюсы:

- Зрелая, самая популярная библиотека
- Огромное количество примеров и туториалов
- Полный контроль над всем

Минусы:

- Тяжёлая (~500 KB бандл)
- Не tree-shakeable
- API местами неинтуитивно (особенно `BigNumber` в v5, в v6 улучшили)

Когда использовать: если нужен полный контроль и не пугает большой бандл.

viem

Что это: современная альтернатива `ethers.js` от создателей `wagmi`.

Что умеет: то же самое, что `ethers.js`, но:

Плюсы:

- Лёгкая (~50 KB, tree-shakeable)
- TypeScript-first (типы из коробки, без доп. пакетов)
- Современное API: compose-able, предсказуемое
- Быстрее развивается

Минусы:

- Меньше туториалов и примеров (но быстро догоняет)
- API отличается от `ethers.js` — нужно переучиваться

Когда использовать: новые проекты 2024+. Рекомендуемый выбор.

wagmi

Что это: React-хуки поверх `viem` (работает с `viem` v2, раньше работала с `ethers.js`).

Что умеет:

- `useAccount()` — подключён ли кошелёк, какой адрес
- `useConnect()` — диалог «подключить MetaMask»
- `useReadContract()` — прочитать данные из контракта
- `useWriteContract()` — отправить транзакцию
- `useWaitForTransactionReceipt()` — ждать подтверждения
- `useWatchContractEvent()` — слушать события

Плюсы:

- Декларативный React-подход — меньше бойлерплейта

- Автоматический refetch при смене аккаунта/сети
- Кэширование, инвалидация запросов
- Интеграция с RainbowKit (UI для connect wallet)

Минусы:

- Абстракция — сложнее понять, что под капотом
- Зависимость от React

Когда использовать: всегда, если пишем на React. wagmi + viem = золотой стандарт 2024+.

Как они соотносятся

Уровень абстракции:

wagmi	← React-хуки (самый высокий уровень)
↓	
viem	← низкоуровневые вызовы (легковесный)
↓	
ethers.js	← низкоуровневые вызовы (тяжёлый)

Иерархия:

wagmi	использует viem
viem	— самостоятельная библиотека
ethers.js	— самостоятельная библиотека

Рекомендация для пет-проекта

React + TypeScript

+ wagmi	(хуки: connect, read, write)
+ viem	(идет внутри wagmi, редко напрямую)
+ RainbowKit	(UI для кнопки "Connect Wallet")

Не рекомендуем ethers.js для новых проектов — viem легче, быстрее, и wagmi перешёл на viem.

Минимальный пример (wagmi + viem)

```
import { useAccount, useReadContract, useWriteContract } from 'wagmi';

function TokenBalance() {
  const { address } = useAccount();

  const { data: balance } = useReadContract({
    address: '0x...', // адрес токена
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: [address],
  });

  > **Как читать `useReadContract({ address: '0x...', abi: erc20Abi,
  functionName: 'balanceOf', args: [address] })`:** «React-хуком прочитай б
  аланс ERC-20 токена: вызови view-функцию `balanceOf` у контракта по адре
  су, с типовым ABI стандартного токена, передав адрес пользователя». Мнем
  оника: адрес + ABI + функция + аргументы = один декларативный запрос,
  без бойлерплейта.

  const { writeContract } = useWriteContract();

  return (
    <div>
      <p>Баланс: {balance?.toString()}</p>
      <button onClick={() => writeContract({
        address: '0x...',
        abi: erc20Abi,
        functionName: 'transfer',
        args: ['0xRecipient...', 100n],
      })}>
        Отправить 100 токенов
      </button>
    </div>
  );
}
```

Как читать `writeContract({ address: '0x...', abi: erc20Abi, functionName: 'transfer', args: ['0xRecipient...', 100n] })`: «пошли транзакцию перевода токенов на смарт-контракт: вызови *transfer*, передав адрес получателя и количество в минимальных единицах (`BigInt` с суффиксом *n*)». Мнемоника: *writeContract* = мутация блокчейна; жди подпись в кошельке, потом отслеживай через хеш.

The Graph / Subgraph — индексация ончейн-данных

The Graph — это децентрализованный протокол для индексации и запросов данных из блокчейна. Subgraph (сабграф) — твой код, который говорит The Graph: какие данные доставать из контракта, как их обработать и в каком виде отдавать через GraphQL. Готовый API из сырых ончейн-данных.

Актуальность: июль 2026. 60+ поддерживаемых сетей, graph-cli последней версии, Studio + децентрализованная сеть. Данные проверены по официальной документации thegraph.com/docs.

Зачем нужен Subgraph (прямой вызов контракта медленный)

Представь: ты фронтендер, делаешь dApp — дашборд токена ERC-20. Нужно показать:

- Баланс пользователя — `balanceOf(address)`
- Все его переводы за месяц — события `Transfer`
- Топ-100 держателей — нужны все `Transfer` события

Проблема прямых вызовов (RPC)

```
// Так не делают в продакшене:
const provider = new JsonRpcProvider("https://eth-mainnet.g.alchemy.com/v2/KEY")

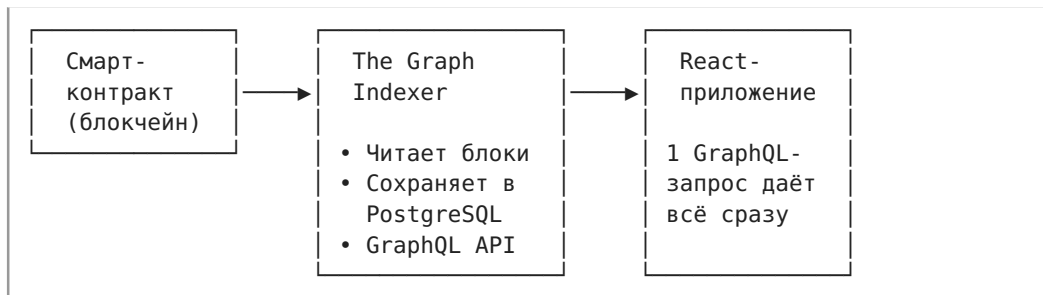
// 1 запрос = 1 RPC-вызов — быстро
const balance = await contract.balanceOf(address)

// Но 100 запросов = 100 RPC-вызовов — медленно
const balances = await Promise.all(
  holders.map(h => contract.balanceOf(h))
)
// ~3-5 секунд, плюс rate-limit провайдера

// А чтобы собрать все Transfer за месяц?
// Нужно тянуть блок за блоком — тысячи запросов, минуты ожидания
```

Прямые RPC-вызовы не предназначены для агрегации и фильтрации данных — блокчейн не база данных. Каждый вызов `balanceOf` ходит в отдельный блок. Каждый `eth_getLogs` тянет сырые логи, которые нужно парсить.

Решение — Subgraph



Subgraph индексирует блокчейн один раз и сохраняет обработанные данные в обычную базу (PostgreSQL). Ты получаешь быстрый GraphQL API:

```
# 1 запрос — всё что нужно!
{
  transfers(first: 100, orderBy: timestamp, orderDirection: desc) {
    from
    to
    value
    timestamp
  }
}
```

Как читать `{ transfers(first: 100, orderBy: timestamp, orderDirection: desc) { from to value timestamp } }`: «запроси у сабграфа последние 100 переводов: отсортируй по времени от новых к старым, для каждого верни отправителя, получателя, сумму и метку времени — одним запросом». Мнемоника: GraphQL к сабграфу = SQL к индексированному блокчейну; *first/skip* = LIMIT/OFFSET.

Как работает The Graph

Архитектура протокола

```
graph TD
  A[Блокчейн] -->|сырые блоки| B[Graph Node]
  B -->|индексация| C[PostgreSQL]
  C -->|GraphQL| D[Твоё dApp]

  E[Subgraph Manifest] -->|subgraph.yaml| B
  F[Mappings] -->|AssemblyScript| B
  G[Schema] -->|schema.graphql| C
```

Ключевые компоненты:

- **Graph Node** — нода, которая слушает блокчейн, запускает маппинги и сохраняет данные в БД. Конечный пользователь не запускает ноду

- сам — использует хостинг (Subgraph Studio) или децентрализованную сеть.
- **Indexer** (индексатор) — оператор ноды в децентрализованной сети. Получает GRT-токены за индексацию и обработку запросов.
 - **Curator** (куратор) — сигнализирует токенами GRT, какие сабграфы полезны/качественны. Indexer'ы индексируют в первую очередь то, на что есть сигнал.
 - **Delegator** (делегатор) — делегирует GRT индексатору и получает долю от наград.

Для фронтендера важны только Indexer и твой Subgraph.
Остальные роли — это экономика протокола. Ты просто деплоишь Subgraph и получаешь GraphQL-эндпоинт.

Два способа хостинга

Способ	Описание	Бесплатно?
Subgraph Studio	Тестирование и разработка. Твой сабграф индексируется одним Upgrade Indexer. Rate-limited, не для продакшена.	Да
The Graph Network	Децентрализованная сеть. Indexer'ы соревнуются за твой сабграф. Публичный, production-ready. 100K запросов/мес бесплатно.	Фримиум

Архитектура Subgraph

Сабграф состоит из трёх файлов — это и есть твой код:

```
my-subgraph/  
├─ subgraph.yaml      # Манифест: ЧТО индексируем  
├─ schema.graphql    # Схема: КАК выглядят данные  
├─ src/  
│  └─ mapping.ts     # Маппинги: КАК обрабатываем  
└─ node_modules/
```

1. Manifest — subgraph.yaml

Указывает, какой контракт слушать, с какого блока, какие события обрабатывать:

```

specVersion: 1.0.0
schema:
  file: ./schema.graphql
dataSources:
  - kind: ethereum
    name: MyToken
    network: mainnet
    source:
      address: "0x...ТВОЙ_КОНТРАКТ..."
      abi: MyToken
      startBlock: 18000000    # Блок деплоя контракта
    mapping:
      kind: ethereum/events
      apiVersion: 0.0.7
      language: wasm/assemblyscript
      entities:
        - Transfer
        - Account
      abis:
        - name: MyToken
          file: ./abis/MyToken.json
      eventHandlers:
        - event: Transfer(indexed address,indexed address,uint256)
          handler: handleTransfer
      file: ./src/mapping.ts

```

Как читать секцию *dataSources* в *subgraph.yaml*: «опиши The Graph, какой контракт слушать: вот его адрес, вот ABI для декодирования, начинай с этого блока; *eventHandlers* связывают события контракта (например, *Transfer(indexed address,indexed address,uint256)*) с функциями-обработчиками в *mapping.ts*». Мнемоника: *subgraph.yaml* = конфиг-слушатель: с какого контракта, с какого блока, на какие события реагировать.

Что здесь важно:

- *startBlock* — блок деплоя контракта. Не ставь 0, сэкономишь дни индексации.
- *eventHandlers* — на каждое событие контракта свой *handler*-функция в *mapping.ts*.
- *abis* — ABI контракта нужен для декодирования событий.

2. Schema — *schema.graphql*

Описывает структуру данных, которые ты будешь запрашивать. Это GraphQL-схема (не Solidity!):

```

# Тип для аккаунта (держателя токенов)
type Account @entity {
  id: ID!                                # адрес — уникальный идентификатор
  address: Bytes!
  balance: BigInt!
  transfersFrom: [Transfer!]! @derivedFrom(field: "from")
  transfersTo: [Transfer!]! @derivedFrom(field: "to")
}

# Тип для перевода
type Transfer @entity {
  id: ID!                                # txHash-logIndex
  from: Account!
  to: Account!
  value: BigInt!
  timestamp: BigInt!
  blockNumber: BigInt!
  transactionHash: Bytes!
}

```

Как читать `type Transfer @entity { id: ID!; from: Account!; to: Account!; value: BigInt! }`: «опиши GraphQL-тип, который The Graph сохранит в PostgreSQL: `@entity` — этот тип станет таблицей; `ID!` — обязательный первичный ключ; `BigInt` — потому что `uint256` не влезает в обычный `Int`; `@derivedFrom` — виртуальная обратная связь (one-to-many), не хранится, вычисляется при запросе». Мнемоника: `schema.graphql` = проектируешь БД на GraphQL; `@entity` = таблица, `@derivedFrom` = внешний ключ наоборот.

Правила:

- `@entity` — тип сохраняется в БД. Без `@entity` нельзя.
- `ID!` — обязательный уникальный идентификатор. Обычно адрес, `txHash`, или составной ключ.
- `BigInt` для чисел (не `Int!` `uint256` не влезает в 32 бита).
- `Bytes` для адресов и хешей.
- `@derivedFrom` — обратная связь (one-to-many), не хранится в БД, вычисляется при запросе.

3. Mappings — `mapping.ts (AssemblyScript)`

Код, который реагирует на события блокчейна и сохраняет данные согласно схеме:

```

import { Transfer as TransferEvent } from '../generated/MyToken/MyToken'
import { Transfer, Account } from '../generated/schema'
import { BigInt, Bytes } from '@graphprotocol/graph-ts'

export function handleTransfer(event: TransferEvent): void {
  // --- Создаём/обновляем запись Transfer ---
  let transfer = new Transfer(
    event.transaction.hash.toHexString() + '-' + event.logIndex.toString()
  )
  transfer.from = event.params.from
  transfer.to = event.params.to
  transfer.value = event.params.value
  transfer.timestamp = event.block.timestamp
  transfer.blockNumber = event.block.number
  transfer.transactionHash = event.transaction.hash
  transfer.save()

  // --- Обновляем баланс отправителя ---
  let fromAccount = Account.load(event.params.from)
  if (fromAccount == null) {
    fromAccount = new Account(event.params.from)
    fromAccount.balance = BigInt.fromI32(0)
  }
  fromAccount.balance = fromAccount.balance.minus(event.params.value)
  fromAccount.save()

  // --- Обновляем баланс получателя ---
  let toAccount = Account.load(event.params.to)
  if (toAccount == null) {
    toAccount = new Account(event.params.to)
    toAccount.balance = BigInt.fromI32(0)
  }
  toAccount.balance = toAccount.balance.plus(event.params.value)
  toAccount.save()
}

```

Как читать связку `new Transfer(id)` + заполнение полей + `.save()` в AssemblyScript-маппинге: «создай новую запись в БД сабграфа с уникальным ключом (обычно txHash-logIndex), заполни поля значениями из события блокчейна и закоммить вызовом `.save()`; для существующих записей — `Entity.load(id)` вернёт объект, который можно изменить и снова сохранить». Мнемоника: `new Entity(id) = INSERT`, `.save() = COMMIT`, `Entity.load(id) = SELECT` для UPDATE.

Важно понимать про AssemblyScript:

- Это **не TypeScript!** Строгий статически типизированный язык, компилируется в WASM.
- Нет `console.log` на каждый чих — используй `log.info()` (логи

появляются в Studio).

- Все импорты — из @graphprotocol/graph-ts. Нет доступа к fs, http, обычному JS.
- BigInt — беззнаковый. Нет отрицательных значений. Вычитание может упасть с overflow.
- Записи в store — new Entity(id) + .save() = INSERT или UPDATE. Под капотом PostgreSQL.

Создание своего Subgraph для ERC-20

Пошаговый гайд

Шаг 1. Установка Graph CLI

```
npm install -g @graphprotocol/graph-cli@latest
graph --version    # проверяем
```

Шаг 2. Инициализация из контракта

```
graph init
```

CLI задаст вопросы:

- **Protocol:** ethereum
- **Subgraph slug:** my-token-mainnet (уникальное имя)
- **Directory:** my-token-subgraph
- **Network:** mainnet (или sepolia для тестов)
- **Contract address:** 0x... (адрес твоего ERC-20)
- **ABI:** путь к JSON-файлу (или автоопределение из Etherscan)
- **Start block:** номер блока деплоя контракта
- **Index events as entities:** true (автоматически создаст маппинги на все события)

CLI создаст структуру:

```
my-token-subgraph/
├── subgraph.yaml
├── schema.graphql
├── src/mapping.ts
├── abis/MyToken.json
└── package.json
```

Шаг 3. Правим схему под ERC-20

Замени автосгенерированную схему на осмысленную:


```
type Account @entity {  
  id: ID!  
  address: Bytes!  
  balance: BigInt!  
  transferCount: Int!  
  transfersFrom: [Transfer!]! @derivedFrom(field: "from")  
  transfersTo: [Transfer!]! @derivedFrom(field: "to")  
}
```

```
type Transfer @entity {  
  id: ID!  
  from: Account!  
  to: Account!  
  value: BigInt!  
  timestamp: BigInt!  
  blockNumber: BigInt!  
  transactionHash: Bytes!  
}
```

Шаг 4. Правим маппинг

```
import { Transfer as TransferEvent } from '../generated/MyToken/MyToken'
import { Transfer, Account } from '../generated/schema'
import { BigInt } from '@graphprotocol/graph-ts'

export function handleTransfer(event: TransferEvent): void {
  // --- Transfer entity ---
  let transfer = new Transfer(
    event.transaction.hash.toHex() + '-' + event.logIndex.toString()
  )
  transfer.from = event.params.from
  transfer.to = event.params.to
  transfer.value = event.params.value
  transfer.timestamp = event.block.timestamp
  transfer.blockNumber = event.block.number
  transfer.transactionHash = event.transaction.hash
  transfer.save()

  // --- From account ---
  let from = Account.load(event.params.from)
  if (!from) {
    from = new Account(event.params.from)
    from.address = event.params.from
    from.balance = BigInt.fromI32(0)
    from.transferCount = 0
  }
  from.balance = from.balance.minus(event.params.value)
  from.transferCount += 1
  from.save()

  // --- To account ---
  let to = Account.load(event.params.to)
  if (!to) {
    to = new Account(event.params.to)
    to.address = event.params.to
    to.balance = BigInt.fromI32(0)
    to.transferCount = 0
  }
  to.balance = to.balance.plus(event.params.value)
  to.transferCount += 1
  to.save()
}
```

Шаг 5. Генерируем код и собираем

```
graph codegen  # генерирует типы из schema.graphql + ABI
graph build    # компилирует AssemblyScript → WASM
```

graph codegen создаст папку generated/ с TypeScript-типами. Их нельзя редактировать — они пересоздаются при каждом codegen.

Шаг 6. Деплой в Subgraph Studio

1. Иди на thegraph.com/studio, подключи кошелёк
2. Нажми «**Create a Subgraph**», назови MyToken Mainnet
3. Скопируй **Deploy Key** со страницы сабграфа
4. В терминале:

```
graph auth <DEPLOY_KEY>
graph deploy <SUBGRAPH_SLUG>
```

Deploy Key ≠ **API Key**. Deploy Key — для деплоя (записи). API Key создаётся отдельно для запросов (чтения).

Шаг 7. Жди индексации

Индексация может занять от 5 минут до нескольких часов — зависит от количества событий с startBlock. В Studio вкладка **Logs** показывает прогресс:

```
Processing block #18000000
Applying 2 event handlers
Synced 15.2%
```

Когда статус сменится на **Synced 100%** — сабграф готов к запросам.

Шаг 8. Публикация в децентрализованную сеть

Для продакшена публикуй сабграф в The Graph Network:

```
graph publish
# или через Studio: кнопка Publish → подпись в кошельке
```

После публикации твой сабграф появляется в [Graph Explorer](https://thegraph.com/explorer), доступен без rate-limit'ов и индексируется децентрализованными Indexer'ами.

Рекомендуется добавить кураторский сигнал (3 000+ GRT) при публикации — это стимулирует Indexer'ов начать индексацию твоего сабграфа.

Запросы из React-приложения

Способ 1. Apollo Client

```
npm install @apollo/client graphql
```

```
// graphql/client.ts
import { ApolloClient, InMemoryCache } from '@apollo/client'

export const client = new ApolloClient({
  uri: 'https://api.studio.thegraph.com/query/YOUR_ID/my-token-mainnet',
  cache: new InMemoryCache(),
})
```

```
// components/TransferList.tsx
import { useQuery, gql } from '@apollo/client'

const GET_TRANSFERS = gql`
  query GetTransfers($first: Int!, $skip: Int!) {
    transfers(
      first: $first,
      skip: $skip,
      orderBy: timestamp,
      orderDirection: desc
    ) {
      id
      from { address }
      to { address }
      value
      timestamp
      transactionHash
    }
  }
`

function TransferList() {
  const { loading, data, fetchMore } = useQuery(GET_TRANSFERS, {
    variables: { first: 20, skip: 0 },
  })

  if (loading) return <div>Загрузка...</div>

  return data.transfers.map(t => (
    <div key={t.id}>
      {t.from.address} → {t.to.address}: {t.value.toString()} wei
    </div>
  ))
}
```

Способ 2. urql (легче, чем Apollo)

```
npm install urql graphql
```

```
// graphql/client.ts
import { Client, cacheExchange, fetchExchange } from 'urql'

export const client = new Client({
  url: 'https://api.studio.thegraph.com/query/YOUR_ID/my-token-mainnet',
  exchanges: [cacheExchange, fetchExchange],
})

// В корне приложения:
// <Provider value={client}><App /></Provider>
```

```
// hooks/useTransfers.ts
import { useQuery } from 'urql'

const TransfersQuery = `
  query ($first: Int!, $skip: Int!) {
    transfers(first: $first, skip: $skip, orderBy: timestamp,
orderDirection: desc) {
      id
      from { id }
      to { id }
      value
      timestamp
    }
  }
`

function useTransfers(first = 20) {
  const [result] = useQuery({ query: TransfersQuery, variables: { first,
skip: 0 } })
  return result
}
```

Способ 3. Простой fetch (без библиотек)

```
const query = `
  {
    transfers(first: 10, orderBy: timestamp, orderDirection: desc) {
      id
      from { id }
      to { id }
      value
    }
  }
`

const res = await fetch('https://api.studio.thegraph.com/query/YOUR_ID/my-token-mainnet', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ query }),
})
const { data } = await res.json()
```

Любой GraphQL-клиент работает — The Graph отдаёт стандартный GraphQL-эндпоинт. Разницы нет, Apollo, urql или fetch.

Пагинация

```
query ($first: Int!, $skip: Int!) {
  transfers(first: $first, skip: $skip, orderBy: timestamp, orderDirection: desc) {
    id
    value
  }
}
```

```
// «Загрузить ещё»:
fetchMore({ variables: { first: 20, skip: data.transfers.length } })
```

Типичный API Key flow

С июля 2024 Subgraph Studio требует API Key для запросов:

1. Создай API Key в Studio → вкладка **API Keys**
2. Добавь в заголовки:

```
const client = new ApolloClient({
  uri: '...',
  cache: new InMemoryCache(),
  headers: {
    Authorization: 'Bearer YOUR_API_KEY',
  },
})
```

Без ключа — 429 ошибка. Бесплатный тир: 100K запросов/мес.

Goldsky — альтернатива The Graph

Goldsky — это коммерческий хостинг для сабграфов. Совместим со стандартным форматом The Graph, но предлагает:

Сравнение The Graph vs Goldsky

Критерий	The Graph (децентрализованная сеть)	Goldsky
Модель	Децентрализованная (Indexer'ы)	Централизованная (Goldsky)
Бесплатный тип	100K запросов/мес	Есть (ограниченный)
Совместимость	Стандартный формат subgraph	Тот же формат! Можно деплоить те же файлы
Realtime	~несколько блоков задержки	Substreams + real-time (< 1 сек)
Webhooks		(JSON webhooks на события)
Direct DB access		Mirror в свою БД
Mirror pipelines		Можно зеркалировать данные в свою БД
Установка	graph-cli → graph deploy	goldsky CLI → goldsky subgraph deploy
Сложность	Проще для старта (Studio бесплатен)	Нужен аккаунт, но больше фич

Когда Goldsky

- Нужен **real-time** (< 1 сек задержки) — Goldsky использует Substreams
- Нужны **webhooks** на ончейн-события
- Нужна **потокковая передача** данных в свою инфраструктуру
- Команда готова **платить** за инфраструктуру

Когда The Graph

- Бесплатный старт через Studio
- Децентрализация принципиальна
- Стандартный индекс-и-запрос без специфических требований
- Хочешь публичный сабграф для сообщества

На практике: многие проекты начинают на The Graph Studio (бесплатно), а для продакшена выбирают между публикацией в децентрализованную сеть The Graph и Goldsky — в зависимости от требований к задержке и дополнительным фичам.

Типичные ошибки

1. AssemblyScript — это не TypeScript

```
// Не работает:
console.log(event.params.value)           // нет console
const x = event.params.value as number    // нет as-приведения для BigInt

// Правильно:
import { log } from '@graphprotocol/graph-ts'
log.info('Transfer value: {}', [event.params.value.toString()])
```

2. BigInt переполнение

```
// Опасно: вычитание из нуля
fromAccount.balance = fromAccount.balance.minus(event.params.value)
// Если баланс был 0, а value > 0 — overflow, сабграф крашится

// Защита:
if (fromAccount.balance.ge(event.params.value)) {
  fromAccount.balance = fromAccount.balance.minus(event.params.value)
} else {
  fromAccount.balance = BigInt.fromI32(0)
}
```

3. Отсутствие id у @entity

```
# Без id не заведётся
type Token @entity {
  name: String!
  symbol: String!
}

# Обязательно поле id: ID!
type Token @entity {
  id: ID!
  name: String!
  symbol: String!
}
```

4. startBlock = 0

```
source:
  startBlock: 0 # Будет индексироваться неделями
  startBlock: 18000000 # Блок деплоя контракта
```

5. GraphQL query без first

```
# Может вернуть 1000+ записей и упасть по таймауту
{ transfers { id } }

# Всегда first + пагинация
{ transfers(first: 100, skip: 0) { id } }
```


Когда НЕ нужен Subgraph

- **Просто прочитать** `balanceOf` **один раз** — RPC достаточно
- **Данные нужны мгновенно** (после транзакции) — Subgraph отстаёт на несколько блоков, используйте события контракта + `wagmi useWatchContractEvent`
- **Данные на один экран** без истории/агрегации — Subgraph избыточен

Правило большого пальца: **Subgraph нужен, когда запрос требует агрегации, фильтрации или истории.** Иначе — прямой RPC.

Связанное

- [wiki/wagmi-RainbowKit-фронтенд](#) — как читать данные из контракта напрямую (альтернатива)
- [wiki/ERC-20-стандарт-токенов](#) — стандарт токенов, события Transfer
- [wiki/Главная](#) — дорожная карта web3
- [wiki/web3-фронтендер-план-трудоустройства](#) — план трудоустройства

Паттерны транзакций в React

Паттерны работы с транзакциями в React

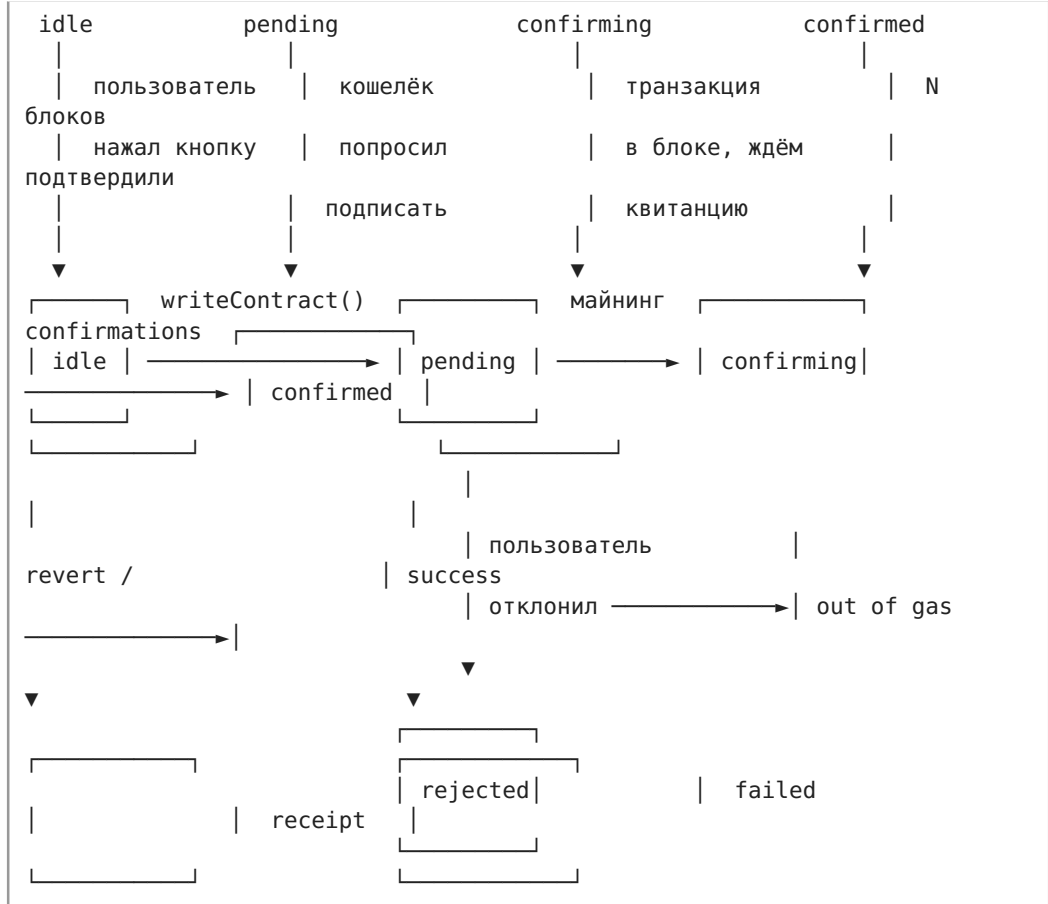
Практическое руководство по обработке транзакций в React-приложениях с wagmi/viem. Все примеры — реальные боевые паттерны, которые пишут в продакшене. Минимум теории блокчейна, максимум кода.

Актуальность: июль 2026, wagmi v3, viem v2.x. API проверен по официальной документации.

Связанные страницы: [wiki/wagmi-RainbowKit-фронтенд](#), [wiki/web3-фронтендер-план-трудоустройства](#)

Жизненный цикл транзакции

Транзакция проходит 4 состояния. Ваш UI должен обрабатывать **каждое**:



Ключевые хуки wagmi, соответствующие состояниям:

Состояние	Хук	Что даёт
idle → pending	useWriteContract / useSendTransaction	writeContract(), isPending, data: hash
pending → confirming	useWaitForTransactionReceipt	isLoading, isSuccess
confirming → confirmed	useWaitForTransactionReceipt с confirmations	Ждёт N блоков
Ошибка	error из хуков + decodeErrorResult	Расшифровка revert

useWriteContract — пишем в смарт-контракт

Базовая механика: useWriteContract возвращает мутацию TanStack Query. Вызываете writeContract() → кошелёк просит подписать → получаете hash.

```

import { useWriteContract, useWaitForTransactionReceipt } from 'wagmi'
import { erc20Abi, parseEther } from 'viem'

function TransferToken() {
  const {
    data: hash,
    isPending,
    error,
    writeContract,
  } = useWriteContract()

  const {
    isLoading: isConfirming,
    isSuccess: isConfirmed,
  } = useWaitForTransactionReceipt({ hash })

  const handleTransfer = () => {
    writeContract({
      address: '0x6B175474E89094C44Da98b954EedeAC495271d0F', // DAI
      abi: erc20Abi,
      functionName: 'transfer',
      args: ['0xRecipient...', parseEther('10')], // 10 DAI
    })
  }

  return (
    <div>
      <button disabled={isPending} onClick={handleTransfer}>
        {isPending ? 'Подтвердите в кошельке...' : 'Отправить 10 DAI'}
      </button>

      {hash && <div>Tx: {hash.slice(0, 10)}...{hash.slice(-8)}</div>}
      {isConfirming && <div> Ожидание подтверждения...</div>}
      {isConfirmed && <div> Транзакция подтверждена!</div>}
    </div>
  )
}

```

Что важно знать о useWriteContract (wagmi v3):

- Это **TanStack Query mutation**. Все поля: data, isPending, isSuccess, isError, error, status, reset.
- mutate() = writeContract() — вызываете для отправки.
- mutateAsync() = writeContractAsync() — возвращает Promise<0x\${string}> (хеш).
- data содержит хеш транзакции (0x\${string}) — **сразу** после подписи в кошельке, до майнинга.
- isPending === true пока кошелёк не подписал (или пользователь не отклонил).

Как читать `writeContract({ address, abi, functionName, args })`:

«отправь транзакцию в контракт: вот адрес, вот ABI-интерфейс, вот имя функции, вот аргументы — кошелёк попросит подпись, а в ответ ты получишь хеш». Мнемоника: `writeContract` = запись в блокчейн, платно, не мгновенно, требует `confirmations`.

Паттерн: передаёте `hash` из `useWriteContract` в `useWaitForTransactionReceipt` — и получаете два независимых состояния: «подписывается» и «майнится».

`useSendTransaction` — отправка нативного ETH

Для простых переводов ETH (не вызов контракта) — `useSendTransaction`:

```
import { useSendTransaction, useWaitForTransactionReceipt } from 'wagmi'
import { parseEther } from 'viem'

function SendEth() {
  const {
    data: hash,
    isPending,
    sendTransaction,
  } = useSendTransaction()

  const { isLoading: isConfirming, isSuccess: isConfirmed } =
    useWaitForTransactionReceipt({ hash })

  const handleSend = () => {
    sendTransaction({
      to: '0xRecipientAddress...',
      value: parseEther('0.05'), // 0.05 ETH
    })
  }

  return (
    <button disabled={isPending} onClick={handleSend}>
      {isPending ? 'Подтвердите...' : 'Отправить 0.05 ETH'}
    </button>
  )
}
```

Отличия от `useWriteContract`:

- `useSendTransaction` — для `transfer` ETH, не требует ABI
- `useWriteContract` — для вызова функций контракта (`transfer`, `mint`, `approve...`)
- Оба возвращают одинаковую структуру (`{ data: hash, isPending, ... }`)

useWaitForTransactionReceipt — ожидание квитанции

Самый важный хук для transaction flow. Ждёт, пока транзакция попадёт в блок, и возвращает **receipt** (статус, gas used, логи).

Базовое использование

```
const { data: hash, writeContract } = useWriteContract()

const {
  data: receipt,
  isLoading: isConfirming,
  isSuccess: isConfirmed,
  isError: isFailed,
  error: receiptError,
} = useWaitForTransactionReceipt({ hash })
```

Возвращаемые поля:

Поле	Тип	Когда
data	TransactionReceipt \ undefined	Транзакция в блоке. Содержит status, blockNumber, gasUsed, logs
isLoading	boolean	Ожидание включения в блок
isSuccess	boolean	Транзакция подтверждена
isError	boolean	Транзакция revert / ошибка
error	WaitForTransactionReceiptErrorType	Детали ошибки

Как читать useWaitForTransactionReceipt({ hash }): «следи за хешем транзакции: пока она майнится — *isLoading: true*, когда блокчейн принял её навсегда — *isSuccess: true* и отдаёт квитанцию с *status, blockNumber, gasUsed*». Мнемоника: подключи хеш — получи квитанцию; без хеша хук молчит.

Параметр confirmations — ждать N блоков

По умолчанию хук резолвится после **1 confirmation** (транзакция в блоке). Для безопасности ждите больше:

```
const { isLoading, isSuccess } = useWaitForTransactionReceipt({
  hash,
  confirmations: 3, // ждём 3 блока после включения
})
```

Когда сколько ждать:

- 1 confirmation — достаточно для тестов и небольших сумм
- 3-5 confirmations — продакшен для средних сумм
- 12+ confirmations — крупные суммы / финализация на L1 (вероятность реорга ~0)

Параметр onReplaced — детект замены транзакции

Если пользователь ускорил (speed up) или отменил транзакцию — хук сообщит:

```
const { isLoading, isSuccess } = useWaitForTransactionReceipt({
  hash,
  onReplaced: (replacement) => {
    console.log('Tx заменена:', replacement.reason)
    // reason: 'replaced' | 'repriced' | 'cancelled'
    if (replacement.reason === 'cancelled') {
      // транзакция отменена пользователем
    }
  },
})
```

Параметр pollingInterval

Частота опроса (мс). По умолчанию — из config. Для быстрых сетей можно уменьшить:

```
useWaitForTransactionReceipt({
  hash,
  pollingInterval: 1_000, // опрос каждую секунду
})
```

Обработка revert: decodeErrorResult

Когда смарт-контракт делает revert, вы получаете ошибку с закодированными данными. **decodeErrorResult** из viem расшифровывает причину.

Паттерн: перехват и расшифровка

```

import { type BaseError, useWriteContract } from 'wagmi'
import { decodeErrorResult } from 'viem'
import { contractAbi } from './abi'

function MintWithErrorHandling() {
  const { data: hash, isPending, error, writeContract } = useWriteContract()

  const getRevertReason = (err: Error | null): string => {
    if (!err) return 'Неизвестная ошибка'

    // User rejected – пользователь отклонил в кошельке
    if (err.name === 'UserRejectedRequestError') {
      return 'Вы отклонили транзакцию в кошельке'
    }

    // Пытаемся декодировать revert причину из контракта
    const baseError = err as BaseError
    const revertError = baseError.walk(
      (e) => (e as any)?.data?.data !== undefined
    )

    if (revertError) {
      try {
        const decoded = decodeErrorResult({
          abi: contractAbi,
          data: (revertError as any).data.data,
        })
        return `Контракт вернул ошибку: ${decoded.errorName} (${decoded.args})`
      } catch {
        // Не удалось декодировать – показываем сырое сообщение
      }
    }

    return baseError.shortMessage || err.message
  }

  return (
    <div>
      <button disabled={isPending} onClick={() => writeContract({ ... })}>
        Mint
      </button>
      {error && <div style={{ color: 'red' }}> {getRevertReason(error)}
    </div>
  )
}

```


Ключевой приём — `baseError.walk()`:

Ошибки в `viem/wagmi` — это цепочка `BaseError`. `walk()` идёт по цепочке и находит вложенную ошибку с нужными данными. В примере выше мы ищем ошибку, у которой есть `data.data` — это закодированный `revert` от контракта.

Как читать `baseError.walk(e => условие)`: «пройдишь по цепочке вложенных ошибок и найди ту, которая удовлетворяет условию — например, содержит закодированный `revert` от контракта в `data.data`».

Мнемоника: `walk` = прогулка по матрёшке ошибок, ищешь самую глубокую с полезными данными.

Как читать `decodeErrorResult({ abi, data })`: «возьми ABI контракта и сырые байты ошибки — расшифруй, какое имя ошибки (например `InsufficientBalance`) и с какими аргументами контракт сделал `revert`». Мнемоника: `decodeErrorResult` = переводчик с языка байтов `revert`-ошибки на человеческий «контракт сказал: недостаточно средств».

Типичные ошибки и их `.name`

error.name	Значение	Действие
UserRejectedRequestError	Пользователь отклонил в кошельке	Показать «Отклонено», не пугать
ContractFunctionRevertedError	Контракт сделал <code>revert</code>	Декодировать <code>decodeErrorResult</code>
EstimateGasExecutionError	Не удалось оценить газ (скорее всего <code>revert</code>)	Показать причину до отправки
InsufficientFundsError	Не хватает ETH на газ	Показать «Недостаточно средств»
TransactionExecutionError	Ошибка при отправке	Показать <code>shortMessage</code>

Gas estimation: почему падает и как чинить

Проблема: `useWriteContract` внутри вызывает `estimateGas`, чтобы рассчитать лимит газа. `estimateGas` симулирует транзакцию — если симуляция делает `revert`, вы получаете ошибку **до** отправки в кошелёк.

Причины падения `estimateGas`

- 1. **Контракт реально делает `revert`** — неправильные аргументы, недостаточно баланса, не тот `allowance`
- 2. **Зависимость от состояния** — между вызовом `estimateGas` и реальной отправкой состояние изменилось (другой пользователь опередил)

3. **Сложная логика газа** — контракт с циклами/ветвлениями, газ зависит от входных данных
4. **Недостаточно средств** — на кошельке нет ETH для оплаты газа

Решение 1: Ручная оценка газа с запасом

```
import { useWriteContract } from 'wagmi'
import { usePublicClient } from 'wagmi'
import { erc20Abi } from 'viem'

function SafeTransfer() {
  const publicClient = usePublicClient()
  const { writeContract } = useWriteContract()

  const handleTransfer = async () => {
    if (!publicClient) return

    // 1. Пробуем оценить газ с запасом
    let gasLimit: bigint
    try {
      const estimated = await publicClient.estimateGas({
        account: '0xUserAddress...',
        to: '0xTokenAddress...',
        data: encodeFunctionData({
          abi: erc20Abi,
          functionName: 'transfer',
          args: ['0xRecipient...', parseEther('10')],
        }),
      })
      // Добавляем 20% запас
      gasLimit = (estimated * 120n) / 100n
    } catch {
      // Оценка не удалась — ставим щедрый дефолт
      gasLimit = 300_000n
    }

    // 2. Отправляем с явным gas limit
    writeContract({
      address: '0xTokenAddress...',
      abi: erc20Abi,
      functionName: 'transfer',
      args: ['0xRecipient...', parseEther('10')],
      gas: gasLimit, // ← форсируем лимит
    })
  }
}
```

Решение 2: Предварительная проверка через useSimulateContract

wagmi v3 предоставляет useSimulateContract — выполняет сухую симуляцию до вызова кошелька:

```

import { useSimulateContract, useWriteContract } from 'wagmi'
import { erc20Abi, parseEther } from 'viem'

function SafeTransferV2() {
  const { data: simulation } = useSimulateContract({
    address: '0xTokenAddress...',
    abi: erc20Abi,
    functionName: 'transfer',
    args: ['0xRecipient...', parseEther('10')],
    query: { enabled: true },
  })

  const { writeContract, isPending } = useWriteContract()

  const isSimulating = simulation === undefined
  const willFail = simulation?.error !== undefined

  return (
    <div>
      {isSimulating && <div> Проверяем транзакцию...</div>}
      {willFail && (
        <div style={{ color: 'red' }}>
          Транзакция не пройдёт: {simulation.error.message}
        </div>
      )}
      <button
        disabled={isPending || willFail || isSimulating}
        onClick={() => writeContract({
          address: '0xTokenAddress...',
          abi: erc20Abi,
          functionName: 'transfer',
          args: ['0xRecipient...', parseEther('10')],
        })}
      >
        Отправить
      </button>
    </div>
  )
}

```

Паттерн: *useSimulateContract* → проверяем *error* → блокируем кнопку
если *revert* → вызываем *writeContract* только если симуляция успешна.

Решение 3: Надбавка газа для нетривиальных контрактов

```
// Для контрактов с переменным потреблением газа (NFT mint, свопы)
const GAS_BUFFER_MULTIPLIER = 1.5 // 50% запас

writeContract({
  address: contractAddress,
  abi: contractAbi,
  functionName: 'mint',
  args: [quantity],
  gas: estimatedGas
    ? BigInt(Math.ceil(Number(estimatedGas) * GAS_BUFFER_MULTIPLIER))
    : 1_000_000n, // fallback для сложных операций
})
```

Замена транзакции: speed up и cancel

В Ethereum транзакцию можно заменить, отправив **новую с тем же nonce** но более высоким газом.

Speed up (ускорение)

Отправить ту же транзакцию, но с повышенной ценой газа:

```
import { useWriteContract } from 'wagmi'

function SpeedUpButton({ oldHash }: { oldHash: `0x${string}` }) {
  const { writeContract } = useWriteContract()

  const speedUp = () => {
    writeContract({
      address: '0xContract...',
      abi: contractAbi,
      functionName: 'transfer',
      args: ['0xRecipient...', parseEther('10')],
      // Форсируем повышенный газ — кошелек сам подставит правильный
      nonce:
        maxFeePerGas: parseGwei('50'), // было 20 → ставим 50
        maxPriorityFeePerGas: parseGwei('3'), // было 1 → ставим 3
    })
  }

  return <button onClick={speedUp}> Ускорить</button>
}
```

Cancel (отмена)

Отправить транзакцию на свой же адрес с 0 ETH и тем же nonce:

```

import { useSendTransaction } from 'wagmi'
import { useConnection } from 'wagmi'
import { parseGwei } from 'viem'

function CancelButton({ oldNonce }: { oldNonce: number }) {
  const { address } = useConnection()
  const { sendTransaction } = useSendTransaction()

  const cancel = () => {
    if (!address) return
    sendTransaction({
      to: address,           // отправляем себе
      value: 0n,             // 0 ETH.
      nonce: oldNonce,       // тот же nonce
      maxFeePerGas: parseGwei('50'), // повышенный газ
      maxPriorityFeePerGas: parseGwei('3'),
    })
  }

  return <button onClick={cancel}>  Отменить</button>
}

```

Отслеживание замены через onReplaced

useWaitForTransactionReceipt сам обнаруживает замену:

```

const { isLoading } = useWaitForTransactionReceipt({
  hash,
  onReplaced: ({ reason, transaction, transactionReceipt }) => {
    switch (reason) {
      case 'replaced':
        toast.success('Транзакция ускорена!')
        // Используйте новый hash из transaction.hash
        break
      case 'cancelled':
        toast.info('Транзакция отменена')
        break
    }
  },
})

```

Toast-уведомления: полноценный хук

Собираем всё в один переиспользуемый хук, который показывает toast на каждом этапе:

```

import { useState, useEffect, useCallback } from 'react'
import {
  useWriteContract,
  useWaitForTransactionReceipt,
  type BaseError,
} from 'wagmi'
import { decodeErrorResult } from 'viem'
import { toast } from 'react-hot-toast' // или sonner, или ваш toaster

type TxLifecycle = {
  status: 'idle' | 'pending' | 'confirming' | 'confirmed' | 'failed'
  hash?: `0x${string}`
  error?: string
  receipt?: { blockNumber: bigint; gasUsed: bigint }
}

export function useTransactionWithToast(abi: any) {
  const [lifecycle, setLifecycle] = useState<TxLifecycle>({ status: 'idle' })

  const {
    data: hash,
    isPending,
    error: writeError,
    writeContract,
  } = useWriteContract()

  const {
    isLoading: isConfirming,
    isSuccess,
    error: receiptError,
  } = useWaitForTransactionReceipt({ hash, confirmations: 2 })

  // === Эффекты для toast ===

  // pending: открываем toast при отправке
  useEffect(() => {
    if (isPending) {
      setLifecycle({ status: 'pending' })
      toast.loading('    Подтвердите транзакцию в кошельке...', {
        id: 'tx-toast',
      })
    }
  }, [isPending])

  // hash получен: обновляем toast
  useEffect(() => {
    if (hash && !isConfirming && !isSuccess) {
      setLifecycle({ status: 'confirming', hash })
      toast.loading('    Транзакция отправлена. Ожидаем')
    }
  }, [hash, isConfirming, isSuccess])
}

```

```

подтверждения...', {
  id: 'tx-toast',
})
}
}, [hash, isConfirming, isSuccess])

// confirmed
useEffect(() => {
  if (isSuccess) {
    setLifecycle({ status: 'confirmed', hash })
    toast.success('Транзакция подтверждена!', { id: 'tx-toast' })
  }
}, [isSuccess, hash])

// ошибка на этапе отправки
useEffect(() => {
  if (writeError) {
    const message = getReadableError(writeError)
    setLifecycle({ status: 'failed', error: message })
    toast.error(` ${message}`, { id: 'tx-toast' })
  }
}, [writeError])

// ошибка на этапе подтверждения (revert)
useEffect(() => {
  if (receiptError) {
    const message = getReadableError(receiptError)
    setLifecycle({ status: 'failed', error: message })
    toast.error(` ${message}`, { id: 'tx-toast' })
  }
}, [receiptError])

// Сброс
const reset = useCallback(() => {
  setLifecycle({ status: 'idle' })
  toast.dismiss('tx-toast')
}, [])

return {
  ...lifecycle,
  isPending,
  isConfirming,
  writeContract,
  reset,
}
}

```

```

// Вспомогательная: человекопонятный текст ошибки
function getReadableError(err: Error): string {
  if (err.name === 'UserRejectedRequestError') {

```

```
    return 'Вы отклонили транзакцию'
  }
  const baseErr = err as BaseError
  if (baseErr.shortMessage?.includes('insufficient funds')) {
    return 'Недостаточно средств для газа'
  }
  return baseErr.shortMessage || err.message.slice(0, 100)
}
```

Использование в компоненте:

```
function MyMintButton() {
  const { status, writeContract, isPending } = useTransactionWithToast(a
bi)

  return (
    <button
      disabled={status === 'pending' || status === 'confirming'}
      onClick={() =>
        writeContract({
          address: '0x...',
          abi,
          functionName: 'mint',
          args: [1n],
        })
      }
    >
      {status === 'idle' && 'Mint NFT'}
      {status === 'pending' && 'Подпишите...'}
      {status === 'confirming' && 'Майнинг...'}
      {status === 'confirmed' && 'Готово'}
      {status === 'failed' && 'Попробовать снова'}
    </button>
  )
}
```

Transaction History: хранение и отображение

Стратегия хранения

Транзакции уходят в блокчейн навсегда, но локально их нужно где-то хранить для истории пользователя. Варианты:

Стратегия	Плюсы	Минусы
localStorage	Просто, переживает перезагрузку	Нет синхронизации между вкладками
React state + localStorage	Мгновенный UI + персистентность	Ручная синхронизация
Subgraph / The Graph	Полная история, индексация	Инфраструктура, задержка
Свой бэкэнд + БД	Полный контроль	Нужен сервер

Реализация: localStorage + React Context

```

// types.ts
type TxRecord = {
  hash: `0x${string}`
  chainId: number
  type: 'send' | 'contract'
  description: string // "Transfer 10 DAI to 0x1234..."
  status: 'pending' | 'confirmed' | 'failed'
  timestamp: number
  blockNumber?: bigint
}

// TxHistoryContext.tsx
import { createContext, useContext, useState, useEffect, useCallback } from 'react'

const STORAGE_KEY = 'tx-history'

function loadHistory(): TxRecord[] {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]')
  } catch {
    return []
  }
}

function saveHistory(records: TxRecord[]) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(records.slice(0, 100)))
}

const TxHistoryContext = createContext<{
  history: TxRecord[]
  addTx: (tx: TxRecord) => void
  updateTx: (hash: string, update: Partial<TxRecord>) => void
}>({ history: [], addTx: () => {}, updateTx: () => {} })

export function TxHistoryProvider({ children }: { children: React.ReactNode }) {
  const [history, setHistory] = useState<TxRecord[]>(loadHistory())

  useEffect(() => { saveHistory(history) }, [history])

  const addTx = useCallback((tx: TxRecord) => {
    setHistory((prev) => [tx, ...prev])
  }, [])

  const updateTx = useCallback((hash: string, update: Partial<TxRecord>) => {
    setHistory((prev) =>
      prev.map((tx) => (tx.hash === hash ? { ...tx, ...update } : tx))
    )
  })
}

```

```

    )
  }, [])

  return (
    <TxHistoryContext.Provider value={{ history, addTx, updateTx }}>
      {children}
    </TxHistoryContext.Provider>
  )
}

export const useTxHistory = () => useContext(TxHistoryContext)

```

Интеграция с хуком транзакции:

```

function TransferWithHistory() {
  const { addTx, updateTx } = useTxHistory()
  const { writeContract, isPending } = useWriteContract()

  const { isLoading: isConfirming, isSuccess } =
    useWaitForTransactionReceipt({
      hash,
      confirmations: 1,
    })

  // При получении hash – добавляем в историю как pending
  useEffect(() => {
    if (hash) {
      addTx({
        hash,
        chainId: chainId!,
        type: 'contract',
        description: `Transfer ${amount} DAI to ${to.slice(0, 6)}...`,
        status: 'pending',
        timestamp: Date.now(),
      })
    }
  }, [hash])

  // При подтверждении – обновляем статус
  useEffect(() => {
    if (isSuccess && hash) {
      updateTx(hash, { status: 'confirmed' })
    }
  }, [isSuccess, hash])
}

```

Компонент истории

```

function TxHistoryList() {
  const { history } = useTxHistory()

  if (history.length === 0) {
    return <div style={{ color: '#888' }}>История транзакций пуста</div>
  }

  return (
    <div>
      <h3>История транзакций</h3>
      {history.map((tx) => (
        <div
          key={tx.hash}
          style={{
            padding: '12px',
            border: '1px solid #333',
            borderRadius: '8px',
            marginBottom: '8px',
          }}
        >
          <div style={{ display: 'flex', justifyContent: 'space-between' }}>
            <strong>{tx.description}</strong>
            <TxStatusBadge status={tx.status} />
          </div>
          <div style={{ fontSize: '0.85em', color: '#888' }}>
            {tx.hash.slice(0, 10)}...{tx.hash.slice(-8)}
            {' · '}
            {new Date(tx.timestamp).toLocaleString()}
          </div>
          <a
            href={`https://etherscan.io/tx/${tx.hash}`}
            target="_blank"
            rel="noopener noreferrer"
            style={{ fontSize: '0.85em' }}
          >
            Etherscan ↗
          </a>
        </div>
      )))
    </div>
  )
}

function TxStatusBadge({ status }: { status: TxRecord['status'] }) {
  const colors = {
    pending: { bg: '#332b00', text: '#ffd700' },
    confirmed: { bg: '#00331a', text: '#00ff88' },
    failed: { bg: '#330000', text: '#ff4444' },
  }
}

```

```

const c = colors[status]
return (
  <span
    style={{
      background: c.bg,
      color: c.text,
      padding: '2px 8px',
      borderRadius: '12px',
      fontSize: '0.8em',
    }}
  >
    {status === 'pending' && ' Pending'}
    {status === 'confirmed' && ' Done'}
    {status === 'failed' && ' Failed'}
  </span>
)
}

```

Боевой компонент: всё вместе

Собираем все паттерны в один продакшен-компонент для transfer ERC-20:

```

import { useState, useEffect } from 'react'
import {
  useWriteContract,
  useWaitForTransactionReceipt,
  useSimulateContract,
  useConnection,
  type BaseError,
} from 'wagmi'
import { erc20Abi, parseEther, decodeErrorResult, formatEther } from 'viem'
import { toast } from 'react-hot-toast'
import { useTxHistory } from './TxHistoryContext'

type TransferState =
  | { step: 'idle' }
  | { step: 'simulating' }
  | { step: 'ready'; gasEstimate: bigint }
  | { step: 'pending_sign' }
  | { step: 'mining'; hash: `0x${string}` }
  | { step: 'confirmed'; hash: `0x${string}` }
  | { step: 'error'; message: string }

export function TransferForm({
  tokenAddress,
  tokenDecimals,
}: {
  tokenAddress: `0x${string}`
  tokenDecimals: number
}) {
  const { address } = useConnection()
  const [to, setTo] = useState('')
  const [amount, setAmount] = useState('')
  const [state, setState] = useState<TransferState>({ step: 'idle' })
  const { addTx, updateTx } = useTxHistory()

  // Проверяем невалидный адрес
  const isValidAddress = /^0x[0-9a-fA-F]{40}$/.test(to)
  const isValidAmount = amount && !isNaN(Number(amount)) &&
    Number(amount) > 0
  const canEstimate = isValidAddress && isValidAmount

  // === Симуляция ===
  const { data: simulation, isLoading: isSimulating } = useSimulateContract({
    address: tokenAddress,
    abi: erc20Abi,
    functionName: 'transfer',
    args: canEstimate
      ? [to as `0x${string}`, parseEther(amount)]
      : undefined,
  })

```

```

    query: { enabled: canEstimate },
  })

  const willFail = simulation?.error !== undefined

  // === Отправка ===
  const {
    data: hash,
    isPending,
    error: writeError,
    writeContract,
  } = useWriteContract()

  // === Ожидание ===
  const {
    isLoading: isConfirming,
    isSuccess,
    error: receiptError,
  } = useWaitForTransactionReceipt({
    hash,
    confirmations: 2,
  })

  // === Эффекты ===
  useEffect(() => {
    if (isSimulating) setState({ step: 'simulating' })
    else if (simulation && !willFail) {
      setState({ step: 'ready', gasEstimate: simulation.gasEstimate })
    } else if (willFail) {
      setState({ step: 'error', message:
getHumanError(simulation!.error!) })
    }
  }, [isSimulating, simulation, willFail])

  useEffect(() => {
    if (isPending) setState({ step: 'pending_sign' })
  }, [isPending])

  useEffect(() => {
    if (hash) {
      setState({ step: 'mining', hash })
      addTx({
        hash,
        chainId: 1,
        type: 'contract',
        description: `Transfer ${amount} tokens to ${to.slice(0, 6)}...`,
        status: 'pending',
        timestamp: Date.now(),
      })
      toast.loading('Транзакция в мемпуле...', { id: 'tx' })
    }
  })

```



```

    }
    }, [hash])

    useEffect(() => {
      if (isConfirming) {
        toast.loading(`Майнинг... (1/2 confirmations)`, { id: 'tx' })
      }
    }, [isConfirming])

    useEffect(() => {
      if (isSuccess && hash) {
        setState({ step: 'confirmed', hash })
        updateTx(hash, { status: 'confirmed' })
        toast.success('Транзакция подтверждена!', { id: 'tx' })
      }
    }, [isSuccess, hash])

    useEffect(() => {
      if (writeError || receiptError) {
        const err = (writeError || receiptError)!
        setState({ step: 'error', message: getHumanError(err) })
        if (hash) updateTx(hash, { status: 'failed' })
        toast.error(getHumanError(err), { id: 'tx' })
      }
    }, [writeError, receiptError])

    // === Обработчик отправки ===
    const handleTransfer = () => {
      setState({ step: 'pending_sign' })
      writeContract({
        address: tokenAddress,
        abi: erc20Abi,
        functionName: 'transfer',
        args: [to as `0x${string}`, parseEther(amount)],
      })
    }

    return (
      <div style={{ maxWidth: 480, margin: '0 auto' }}>
        <h2>Отправить токены</h2>

        <input
          placeholder="Адрес получателя 0x..."
          value={to}
          onChange={(e) => setTo(e.target.value)}
          disabled={state.step !== 'idle' && state.step !== 'ready' && state.step !== 'error'}
          style={{ width: '100%', marginBottom: 8, padding: 8 }}
        />

```

```

<input
  placeholder="Сумма"
  value={amount}
  onChange={(e) => setAmount(e.target.value)}
  disabled={state.step !== 'idle' && state.step !== 'ready' && state.step !== 'error'}
  style={{ width: '100%', marginBottom: 8, padding: 8 }}
/>

{/* Стейты кнопки */}
{state.step === 'idle' && (
  <button disabled style={{ width: '100%', padding: 10 }}>
    Введите адрес и сумму
  </button>
)}
{state.step === 'simulating' && (
  <button disabled style={{ width: '100%', padding: 10 }}>
    Проверяем транзакцию...
  </button>
)}
{state.step === 'ready' && (
  <button onClick={handleTransfer} style={{ width: '100%', padding: 10 }}>
    Отправить ({formatEther(state.gasEstimate)} ETH gas)
  </button>
)}
{state.step === 'pending_sign' && (
  <button disabled style={{ width: '100%', padding: 10 }}>
    Подтвердите в кошельке...
  </button>
)}
{state.step === 'mining' && (
  <button disabled style={{ width: '100%', padding: 10 }}>
    Майнинг... {state.hash.slice(0, 10)}...
  </button>
)}
{state.step === 'confirmed' && (
  <button
    onClick={() => setState({ step: 'idle' })}
    style={{ width: '100%', padding: 10, background: '#00aa55', color: '#fff' }}
  >
    Готово! Отправить ещё
  </button>
)}

{/* Ошибка */}
{state.step === 'error' && (
  <div
    style={{

```

```

        * marginTop: 12,
          padding: 12,
          background: '#330000',
          color: '#ff6666',
          borderRadius: 8,
        }}
      >
        {state.message}
      <button
        onClick={() => setState({ step: 'idle' })}
        style={{ marginLeft: 12 }}
      >
        Попробовать снова
      </button>
    </div>
  )}
</div>
)
}

// Человекопонятная ошибка
function getHumanError(err: Error): string {
  if (err.name === 'UserRejectedRequestError') return 'Вы отклонили транзакцию'
  const baseErr = err as BaseError
  if (baseErr.shortMessage?.includes('insufficient funds'))
    return 'Недостаточно средств'
  return baseErr.shortMessage || err.message.slice(0, 150)
}

```

Шпаргалка: все хуки для транзакций

Хук	Назначение	Ключевые поля
useWriteContract	Вызов write-функции контракта	writeContract(), data: hash, isPending
useSendTransaction	Отправка ETH	sendTransaction(), data: hash, isPending
useWaitForTransactionReceipt	Ожидание квитанции	isLoading, isSuccess, data: receipt, confirmations
useSimulateContract	Сухая симуляция до отправки	data: { result, gasEstimate, error }
useTransactionReceipt	Получить receipt по hash	data: receipt
useTransactionConfirmations	Количество подтверждений	data: number

Чек-лист: что проверить перед продакшеном

- [] Все 4 состояния транзакции отображаются в UI (idle → pending → confirming → confirmed)
- [] Кнопка блокируется (disabled) на время pending и confirming
- [] Ошибка UserRejectedRequestError обрабатывается отдельно (не пугает пользователя)
- [] Revert причина декодируется через decodeErrorResult
- [] Gas limit устанавливается с запасом ($\geq 20\%$)
- [] Используется confirmations ≥ 2 для значимых транзакций
- [] Toast-уведомления на каждом этапе
- [] Хеш транзакции сохраняется в историю (localStorage минимум)
- [] Ссылка на Etherscan для каждой транзакции
- [] onReplaced обрабатывает speed up / cancel

Дополнительные ресурсы

- **wagmi docs:** <https://wagmi.sh/react/guides/write-to-contract>
- **viem decodeErrorResult:** <https://viem.sh/docs/contract/decodeErrorResult>
- **viem estimateGas:** <https://viem.sh/docs/actions/public/estimateGas>
- **viem ошибки (BaseError):** <https://viem.sh/docs/glossary/errors>
- **Связанная wiki:** [wiki/wagmi-RainbowKit-фронтенд](https://github.com/wagmi-dev/wagmi/wiki/ERC-20-стандарт-токенов), [wiki/Сравнение-ethers-viem-wagmi](https://github.com/wagmi-dev/wagmi/wiki/ERC-20-стандарт-токенов)

DeFi для фронтендера

DeFi для фронтендера: что нужно знать

DeFi (Decentralized Finance) — финансы без банков. Вместо банка код (смарт-контракты). Вместо менеджера — математика.

Цель этого гайда: понимать DeFi-продукты на уровне «могу объяснить на собеседовании и написать к ним фронтенд».

Связанное: [wiki/ERC-20-стандарт-токенов](https://github.com/wagmi-dev/wagmi/wiki/ERC-20-стандарт-токенов), [wiki/web3-фронтендер-план-трудоустройства](https://github.com/wagmi-dev/wagmi/wiki/web3-фронтендер-план-трудоустройства), [wiki/Вопросы-web3-собеседование](https://github.com/wagmi-dev/wagmi/wiki/Вопросы-web3-собеседование)

Uniswap: децентрализованная биржа (DEX)

Уровень 1. Для пятилетнего ребёнка

Представь автомат по обмену валюты, только без хозяина. Ты кладёшь яблоки, забираешь бананы. Автомат сам считает курс: если яблок внутри стало больше — курс падает, если меньше — растёт. Никто не

может этот автомат выключить или обмануть — он работает по жёстким математическим правилам. И любой может поставить свой ящик с яблоками и бананами в автомат и получать долю от комиссий за обмены.

Уровень 2. Для новичка в web3

Uniswap — крупнейшая децентрализованная биржа (DEX) на Ethereum и L2-сетях. В отличие от централизованных бирж (Binance, Coinbase), Uniswap не хранит твои средства и не требует регистрации. Ты просто подключаешь кошелек и меняешь токены напрямую через смарт-контракт.

Ключевое отличие от классической биржи с ордербуком (order book): в Uniswap нет продавцов и покупателей, выставляющих заявки. Вместо этого работает модель **АММ — Automated Market Maker** (автоматический маркет-мейкер).

Уровень 3. Для разработчика

АММ и формула $x \cdot y = k$

Сердце Uniswap — формула $x \cdot y = k$, где:

- **x** — количество токена А в пуле
- **y** — количество токена В в пуле
- **k** — константа, которая **всегда остаётся неизменной** при свопах

Пример:

В пуле ETH/USDC лежит 10 ETH и 20 000 USDC

$k = 10 \times 20\,000 = 200\,000$

Я хочу купить 1 ETH. Сколько USDC заплачу?

Новое количество ETH в пуле: $10 - 1 = 9$

Новое количество USDC должно быть: $200\,000 / 9 = 22\,222\text{ USDC}$

Значит, я заплачу: $22\,222 - 20\,000 = 2\,222\text{ USDC}$

Проверка: $9 \times 22\,222 \approx 200\,000 \checkmark$

Из-за умножения курс **меняется нелинейно**: чем больше покупаешь — тем дороже каждая следующая единица. Это и есть **slippage**.

Как читать формулу $x \cdot y = k$: «сколько в пуле токена X умножить на сколько в пуле токена Y — это число никогда не меняется при обмене; если ты забираешь X, протокол добывает Y так, чтобы произведение осталось прежним». Мнемоника: произведение — константа, курс — производная, slippage — плата за сдвиг.

Пулы ликвидности (Liquidity Pools)

Пул — это просто смарт-контракт, в котором лежат два токена. Любой может стать **поставщиком ликвидности** (LP — Liquidity Provider): внести в пул оба токена в текущей пропорции и получить **LP-токены** (долю в пуле). За каждый своп в пуле взимается комиссия (0.05%–1% в зависимости от версии и пула), которая распределяется между LP пропорционально их доле.

Для фронтендера: при разработке UI добавления ликвидности нужно показывать пользователю:

- Текущее соотношение токенов в пуле
- Сколько LP-токенов он получит
- Его долю в пуле (pool share %)
- Ожидаемый доход от комиссий (APR)

Slippage (проскальзывание)

Разница между ожидаемой ценой и реальной ценой исполнения. Возникает по двум причинам:

1. **Ценовой удар (price impact)** — твоя сделка настолько большая, что сама двигает цену в пуле (см. формулу выше)
2. **Фронترаннинг** — кто-то вставил свою транзакцию перед твоей (см. раздел MEV)

Для фронтендера: в интерфейсе свопа критически важно показывать:

- Ожидаемое количество получаемых токенов (expected output)
- Минимальное количество (minimum received) с учётом slippage tolerance
- Предупреждение при высоком price impact (>2% — жёлтое, >5% — красное)
- Ползунок/поле для настройки slippage tolerance (обычно 0.1%–1%)

Impermanent Loss (непостоянная потеря)

Самое важное, что нужно знать LP. Если ты внёс токены в пул, а их цена изменилась относительно друг друга — ты получишь **меньше денег, чем если бы просто держал токены в кошельке.**

Пример (утрировано):

Внёс в пул ETH/DAI: 1 ETH (\$2000) + 2000 DAI (\$2000) = \$4000

ETH вырос до \$3000.

Если бы держал: 1 ETH (\$3000) + 2000 DAI = \$5000 (+\$1000)

В пуле из-за арбитражников: ~0.816 ETH + ~2449 DAI = ~\$4899 (+\$899)

Потеря: \$101 — это и есть *impermanent loss*

Почему «непостоянная»? Если цена вернётся обратно — потеря исчезнет. Но если ты выведешь ликвидность в момент расхождения — потеря станет постоянной.

Для фронтендера: в интерфейсе пулов обязательно показывать:

- Текущий IL относительно холда (HODL-сравнение)
- График: комиссии vs IL — перекрывают ли комиссии потерю
- Предупреждение для волатильных пар: «Высокий риск *impermanent loss*»

Версии Uniswap (v2, v3, v4)

Версия	Ключевая фича	Для фронтендера
v2	$x*y=k$, ликвидность на всём диапазоне цен ($0 \rightarrow \infty$)	Самый простой API, классические пулы
v3	Концентрированная ликвидность — LP выбирают диапазон цен	Нужен UI для выбора min/max цены, график диапазона, расчёт неактивных позиций
v4	Хуки (hooks) — кастомная логика при свобах/добавлении ликвидности	Появились пулы с разной логикой, нужно учитывать хук-контракты

Aave: кредитование и займы

Уровень 1. Для пятилетнего ребёнка

Представь банк-автомат, в который одни люди кладут деньги под проценты (как вклад), а другие берут деньги займы (как кредит). Но чтобы взять кредит, нужно оставить что-то ценное в залог — например, положить в автомат золотую монету, а взять серебряные. Если золото подешевеет слишком сильно — автомат продаст твоё золото и погасит кредит автоматически. Никаких менеджеров, всё по правилам.

Уровень 2. Для новичка в web3

Aave — крупнейший протокол кредитования в DeFi. Работает на Ethereum и L2-сетях (Arbitrum, Optimism, Polygon). Позволяет:

- **Deposit (вклад):** положить токены в протокол и получать проценты (supply APY)
- **Borrow (заём):** взять токены в долг под залог своего депозита

Ключевое правило: **стоимость залога всегда должна быть больше стоимости займа.**

Уровень 3. Для разработчика

Как это работает

1. Пользователь вносит ETH как **collateral (залог)**
2. Протокол рассчитывает, сколько можно занять: $\text{maxBorrow} = \text{collateralValue} \times \text{LTV}$
3. **LTV (Loan-to-Value)** — максимальный процент от залога, который можно занять. Например, для ETH $\text{LTV} = 80\%$ (можно занять до 80% от стоимости залога)
4. Пользователь берёт заём в USDC (или другом токене)
5. На занятую сумму начисляются проценты (borrow APY)

Пример:

Внёс 10 ETH (\$20 000) как залог

LTV для ETH = 80%

Можно занять: $\$20\,000 \times 80\% = \$16\,000$ (например, 16 000 USDC)

Залог: \$20 000, Долг: \$16 000 — безопасно

Health Factor (фактор здоровья)

Главная метрика безопасности позиции. Показывает, насколько залоговая стоимость превышает порог ликвидации.

$$\text{healthFactor} = (\text{collateralValue} \times \text{liquidationThreshold}) / \text{borrowedValue}$$

Как читать $\text{healthFactor} = (\text{collateralValue} \times \text{liquidationThreshold}) / \text{borrowedValue}$: «посчитай, во сколько раз твой залог с учётом порога ликвидации покрывает занятую сумму». Мнемоника: упало ниже 1 — позицию ликвидируют, твой залог уйдёт с молотка.

- **Health Factor > 2:** безопасно, позиция в порядке
- **Health Factor 1.0-2.0:** умеренный риск, стоит мониторить
- **Health Factor < 1.0:** ликвидация! Позиция может быть ликвидирована

Liquidation Threshold — порог, после которого позиция считается рискованной (обычно чуть выше LTV, например 82.5% для ETH).

Ликвидация

Если стоимость залога падает (или стоимость занятого токена растёт) и health factor опускается ниже 1.0, **любой желающий** может ликвидировать позицию:

1. Ликвидатор погашает часть долга заёмщика
2. Забирает залог с **бонусом** (liquidation bonus, обычно 5–10%)
3. Заёмщик теряет часть залога

Это держит протокол платёжеспособным: долги всегда обеспечены залогом.

Для фронтендера: в интерфейсе Aave критически важно:

- **Dashboard с позициями:** какие активы внесены, какие заняты, net worth
- **Health Factor:** цветовая индикация (зелёный/жёлтый/красный), график изменения
- **Калькулятор LTV:** сколько ещё можно занять до опасного уровня
- **Предупреждения:** «Health Factor ниже 1.5 — рекомендуем добавить залог или погасить часть долга»
- **APY-дисплей:** текущая ставка для каждого актива (supply + borrow), меняется динамически в зависимости от utilisation rate (насколько пул занят)

aTokens и долговые токены

Когда пользователь вносит депозит, он получает **aTokens** (aETH, aUSDC и т.д.) — это процентные токены, которые автоматически растут в количестве (rebasing). Когда забирает депозит обратно — сжигает aTokens.

Для займа выпускаются **долговые токены** (debt tokens), которые отслеживают сколько должен пользователь.

Стейкинг (Staking)

Уровень 1. Для пятилетнего ребёнка

Представь, что у тебя есть акции компании, и ты говоришь: «Я не буду их продавать, оставлю у вас надолго». За это компания платит тебе новые акции. Так ты помогаешь сети быть безопаснее, а она тебя за это благодарит.

Уровень 2. Для новичка в web3

Стейкинг — это блокировка токенов в протоколе для получения вознаграждения. Работает в двух основных вариантах:

1. **Стейкинг в Proof-of-Stake (PoS):** блокируешь ETH (или другой PoS-токен) для участия в валидации блоков. Получаешь награду за каждый предложенный/подтверждённый блок. Фундаментальный механизм безопасности Ethereum.
2. **DeFi-стейкинг (yield farming):** вносишь LP-токены или другие токены в протокол, чтобы получать дополнительные токены проекта. Маркетинговый механизм: проекты раздают токены за использование протокола.

Уровень 3. Для разработчика

Как работает стейкинг в PoS

Валидатор блокирует 32 ETH (или кратно 32). Сеть случайно выбирает валидаторов для предложения и подтверждения блоков. За честную работу — награда (~3–5% годовых). За попытку обмана — **slashing**: потеря части (или всего) стека.

Жидкий стейкинг (Liquid Staking): протоколы вроде **Lido** позволяют стейкать любую сумму (не обязательно 32 ETH) и получать взамен **stETH** — токен, который можно использовать в DeFi (обменивать, давать в залог). Это ключевая инновация, которая сделала стейкинг массовым.

DeFi-стейкинг / Yield Farming

Протокол выделяет пул своих governance-токенов и распределяет их между пользователями, которые:

- Предоставляют ликвидность в определённые пулы (LP-стейкинг)
- Держат токены в стейкинг-контракте (single-sided staking)
- Выполняют определённые действия (например, занимают на Aave)

Награда обычно измеряется в **APR** (годовой процент без сложного процента) или **APY** (со сложным процентом).

Для фронтендера:

- Показывать APR/APY для каждого пула, с разбивкой: базовая награда + бонусы
- Таймеры окончания программ стейкинга (часто ограничены по времени)

- Калькулятор дохода: «застейкаешь X → получишь Y через месяц»
 - Lock-периоды: некоторые стейкинг-пулы требуют блокировки на неделю/месяц
 - Unstaking cooldown: время между запросом на вывод и фактическим получением токенов
 - Compound/Claim кнопки: реинвестировать награду или забрать
-

Layer 2: Arbitrum и Optimism

Уровень 1. Для пятилетнего ребёнка

Ethereum — это как одна очень медленная касса в супермаркете, где очередь стоит часами, а чек стоит \$10. Layer 2 — это когда ту же кассу выносят в быстрый коридор: все покупки там считают быстро и дёшево, а потом приносят чек в главную кассу и говорят: «Всё честно, проверь». Главная касса проверяет итог, а не каждую покупку.

Уровень 2. Для новичка в web3

Layer 2 (L2) — это сеть поверх Ethereum (Layer 1), которая обрабатывает транзакции быстрее и дешевле, но использует безопасность Ethereum. Основные L2-решения:

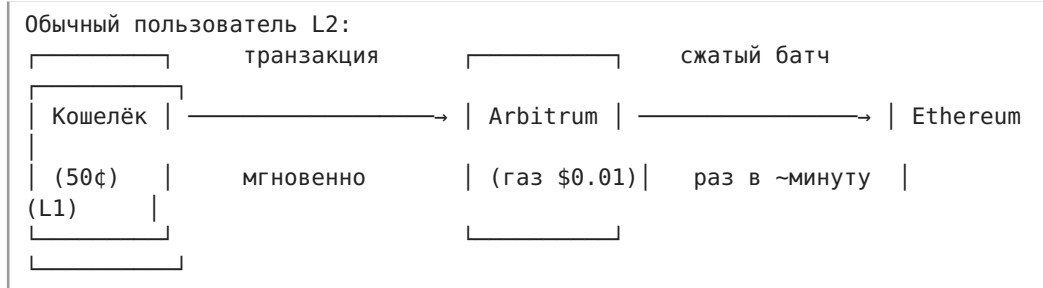
- **Arbitrum** — крупнейший L2 по TVL (>\$15 млрд)
- **Optimism** — второй по величине, часть Superchain-экосистемы
- **Base** — L2 от Coinbase, построен на стеке Optimism

Обе сети используют **Optimistic Rollups** — технологию, при которой транзакции «оптимистично» считаются верными, если никто не докажет обратное.

Уровень 3. Для разработчика

Как работают Optimistic Rollups

1. Пользователи отправляют транзакции в L2-сеть (Arbitrum/Optimism)
2. **Секвенсор** (sequencer) собирает транзакции в батчи, исполняет их и публикует сжатые данные на Ethereum
3. Есть **период оспаривания** (challenge period) — обычно 7 дней — в течение которых любой может доказать fraud proof (доказательство мошенничества)
4. Если никто не оспорил — батч считается финальным на L1



Arbitrum vs Optimism: отличия для фронтендера

Аспект	Arbitrum	Optimism
RPC URL	https://arb1.arbitrum.io/rpc	https://mainnet.optimism.io
Chain ID	42161	10
Блок-эксплорер	arbiscan.io	optimistic.etherscan.io
Нативный токен	ETH (как на L1)	ETH (как на L1)
Блоки	~0.25 сек	2 сек
Финализация на L1	~7 дней (challenge period)	~7 дней (challenge period)
Отличия на уровне контракта	block.number возвращает L2-номер; есть ARB_SYS прекомпилы	Аналогично; есть свои прекомпилы

Что меняется для фронтендера при работе с L2:

- Другой Chain ID** — нужно добавить сеть в кошелёк пользователя (или использовать `wallet_addEthereumChain`)
- Другой RPC-эндпоинт** — в `createPublicClient / createWalletClient` (`viem/wagmi`) указывается RPC L2
- Газ в ЕТН, но копеечный** — транзакция стоит \$0.01-\$0.10 вместо \$5-\$50 на L1. Для UX это значит, что можно показывать газ без страха напугать пользователя
- Адреса контрактов разные** — Uniswap на Ethereum и на Arbitrum имеют разные адреса. Всегда проверяй адреса в документации конкретного L2
- Бриджинг (мост)** — перевод токенов между L1 и L2. Занимает время:
- L1 → L2: ~10-30 минут (ждём финализации батча)
- L2 → L1: ~7 дней (challenge period) — если не использовать быстрые мосты (Across, Hop)

Практический пример (wagmi + Arbitrum):

```
import { arbitrum } from 'wagmi/chains';
import { createConfig, http } from 'wagmi';

const config = createConfig({
  chains: [arbitrum],
  transports: {
    [arbitrum.id]: http('https://arb1.arbitrum.io/rpc'),
  },
});
```

Как читать `createConfig({ chains, transports })`: «собери конфигурацию wagmi: на каких сетях работаем и через какие RPC-эндпоинты к ним подключаемся». Мнемоника: *chains* — куда ходим, *transports* — как добираемся.

Всё остальное (вызовы контрактов, подпись, работа с токенами) — **абсолютно такое же**, как на L1. Именно поэтому L2 так удобны для фронтендера: те же инструменты, та же экосистема, но в 100 раз дешевле.

Зачем нужны L2

- **Масштабирование:** Ethereum обрабатывает ~15 TPS, L2 суммарно — тысячи TPS
- **Цена газа:** на L1 своп стоит \$5–30, на L2 — центы
- **Доступность:** пользователи с маленькими суммами могут пользоваться DeFi без потери всего на газе
- **Экосистема:** Uniswap, Aave, все основные протоколы развёрнуты на Arbitrum и Optimism

MEV: Miner/Maximal Extractable Value

Уровень 1. Для пятилетнего ребёнка

Представь очередь в магазине. Ты стоишь с мороженым за \$2. Кто-то видит, что ты готов заплатить \$5, забегают вперёд тебя, скупают всё мороженое за \$2 и тут же продаёт его тебе за \$5. Ты получаешь своё мороженое, но платишь больше. Этот «кто-то» заработал \$3 просто потому, что увидел твою покупку и успел встать перед тобой в очереди.

Уровень 2. Для новичка в web3

MEV — это прибыль, которую можно извлечь из знания о pending-транзакциях в мемпуле (mempool — «зал ожидания», где транзакции ждут включения в блок). Название расшифровывается как **Maximal Extractable Value** (раньше — Miner Extractable Value).

В блокчейне все транзакции сначала видны в мемпуле до того, как попадут в блок. Боты (MEV searchers) сканируют мемпул в поисках выгодных возможностей — и вставляют свои транзакции до/после целевой.

Уровень 3. Для разработчика

Sandwich Attack (сэндвич-атака)

Самый распространённый вид MEV, напрямую влияющий на UX рядового пользователя.

Мемпул:

1. Твоя транзакция: SWAP 5 ETH → USDC
(видна всем в мемпуле!)

Бот видит это и делает сэндвич:

Бот: SWAP ETH → USDC (газ выше твоего)	← frontrun: бот покупает USDC, двигая цену вверх
Ты: SWAP ETH → USDC	← твоя сделка проходит по худшей цене
Бот: SWAP USDC → ETH	← backrun: бот продаёт USDC обратно, фиксируя прибыль

Итог: ты получил меньше USDC, чем ожидал. Бот заработал на разнице.

Потери пользователя от сэндвича: обычно 0.1%–2% от суммы сделки. На крупных суммах это тысячи долларов.

Другие виды MEV

- **Арбитраж:** токен стоит по-разному на Uniswap и SushiSwap → бот покупает дёшево, продаёт дорого (это полезное MEV — выравнивает цены)
- **Ликвидации:** бот видит, что чья-то позиция на Aave упала ниже health factor 1.0 → первым вызывает ликвидацию и получает бонус (тоже полезное — защищает протокол)
- **Backrunning:** бот видит крупную сделку и сразу после неё делает арбитраж между пулами

Как MEV влияет на UX и что с этим делать

Проблемы для фронтендера и пользователя:

- Сделка проходит по худшей цене, чем ожидалось
- Транзакция может не пройти (revert), если slippage tolerance слишком мал, а бот уже сдвинул цену
- Пользователь винит интерфейс: «Я же поставил slippage 0.5%, почему сделка упала?»

Защита на уровне фронтенда:

1. Разумный slippage tolerance по умолчанию:

2. Стабильные пары (USDC/DAI): 0.1%
3. ETH/USDC: 0.5%
4. Волатильные пары: 1%+
5. Не ставь slippage 0.05% — транзакция почти наверняка упадёт

6. Показывать minimumReceived явно:

"Вы получите не менее 4.85 ETH (ожидаемо 4.87 ETH)"

Пользователь должен видеть гарантированный минимум.

7. Flashbots / MEV-Protection:

8. Отправлять транзакцию не в публичный мемпул, а напрямую блок-строителям (Flashbots Protect, MEV Blocker)
9. Wagmi поддерживает это через privateKeyRpc или кастомные transport'ы

10. Если ты делаешь серьёзный DeFi-фронтенд — интегрируй MEV-защиту

11. Информативность:

12. Предупреждение при большом объёме сделки: «Крупные сделки могут привлечь MEV-ботов»
13. Показывать реальную цену исполнения vs ожидаемую — если разница большая, пользователь понимает, что это не баг интерфейса

Flashbots для фронтендера (практика)

```
// Отправка транзакции через Flashbots (protect.flashbots.net)
import { createPublicClient, createWalletClient, http } from 'viem';
import { mainnet } from 'viem/chains';

// Flashbots Protect RPC – транзакция не попадает в публичный мемпул
const transport = http('https://rpc.flashbots.net');

const client = createWalletClient({
  chain: mainnet,
  transport,
});
// Теперь sendTransaction отправит через Flashbots, защищая от сэндвичей
```

Как читать `createWalletClient({ chain, transport: http('https://rpc.flashbots.net') })`: «создай кошелёчного клиента viem, который отправляет транзакции не в публичный мемпул, а напрямую строителям блоков через Flashbots». Мнемоника: обычный RPC = все видят твою транзакцию, Flashbots RPC = приватная отправка.

Что спрашивают на собеседовании по DeFi (для фронтендера)

Эти вопросы реально задают. От тебя не ждут глубины Solidity-разработчика, но базовое понимание продукта обязательно.

Uniswap

- Что такое АММ и чем отличается от order book?
- Объясни формулу $x*y=k$ на пальцах
- Что такое slippage и как его правильно выставить в UI?
- Что такое impermanent loss? Может ли LP потерять все деньги?
- Чем отличаются Uniswap v2, v3, v4?

Aave

- Как работает lending/borrowing в Aave?
- Что такое Health Factor и что происходит, когда он падает ниже 1.0?
- Что такое LTV и liquidation threshold?
- Как показывать пользователю его риски в интерфейсе?

Стейкинг

- Зачем нужен стейкинг в PoS?
- Что такое liquid staking (Lido, stETH)?
- Чем APR отличается от APY?

Layer 2

- Что такое optimistic rollups и зачем они нужны?
- Какие chain ID у Arbitrum и Optimism?
- Что меняется в коде фронтенда при переходе с L1 на L2?
- Почему вывод из L2 в L1 занимает 7 дней?

MEV

- Что такое MEV и sandwich attack?
- Как MEV-боты влияют на UX обычного пользователя?
- Как защитить пользователя от сэндвича на уровне фронтенда?
- Что такое Flashbots?

Чек-лист: что должен уметь DeFi-фронтендер

- [] Подключить кошелёк и определить сеть (Ethereum / Arbitrum / Optimism)
- [] Реализовать интерфейс свопа: выбор токенов, ввод суммы, расчёт курса, slippage
- [] Показывать user-friendly ошибки: slippage exceeded, insufficient balance, gas estimation failed
- [] Сверстать дашборд позиций: депозиты, займы, health factor
- [] Отображать APR/APY для пулов ликвидности и стейкинга
- [] Добавить сеть (Arbitrum/Optimism) в кошелёк пользователя программно
- [] Понимать, как читать цены из on-chain оракулов (Chainlink) или вычислять из пулов
- [] Интегрировать MEV-защиту (Flashbots RPC) — опционально, но выделяет кандидата

Вопросы web3-собеседования

Вопросы web3-собеседования: фронтендер

Развёрнутые ответы на реальные вопросы, которые задают React-фронтендерам на web3-позициях. 30 вопросов с уровнями сложности — от Junior до Senior.

Связанное: [wiki/web3-фронтендер-план-трудоустройства](#), [wiki/DeFi-для-фронтендера](#), [wiki/wagmi-RainbowKit-фронтенд](#), [wiki/Паттерны-транзакций-React](#)

Блокчейн-основы

1. Объясни, как работает блокчейн на примере Ethereum

Уровень: Junior/Middle

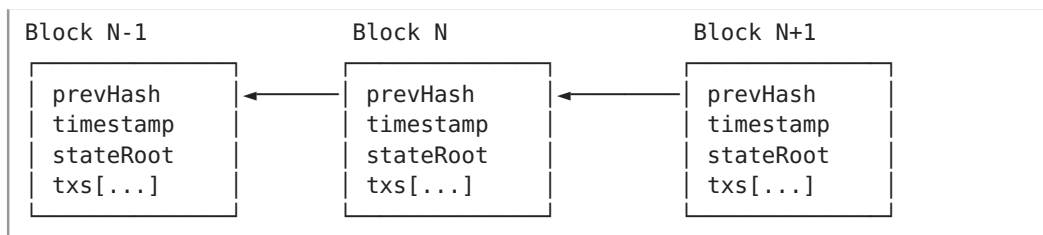
Что спрашивают на самом деле

Проверяют не просто «заучил определение», а понимаешь ли ты фундаментальные механизмы, которые влияют на твою работу фронтендера: почему транзакции не мгновенные, что такое финальность, почему газ дорогой.

Развёрнутый ответ

Блокчейн Ethereum — это распределённая машина состояний. Представь глобальный компьютер, который все участники сети вычисляют синхронно.

Структура:



Ключевые концепции:

1. **Блок** — набор транзакций + хеш предыдущего блока. Измени хоть бит в блоке N-1 → все последующие блоки станут невалидными. Это и есть «неизменяемость».
2. **Консенсус (Proof of Stake)** — валидаторы ставят ETH (stake) и по очереди предлагают блоки. Если валидатор жульничает — его stake сжигается (slashing). Финальность наступает через 2 эпохи (~12.8 минут).
3. **Состояние (State)** — не просто «балансы», а полное дерево всех аккаунтов, контрактов и их storage. State root в заголовке блока — это как «снимок всего».
4. **EVM (Ethereum Virtual Machine)** — среда выполнения смарт-контрактов. Каждая нода исполняет транзакции и получает одинаковый результат. Детерминированность — ключевое свойство.

Почему это важно фронтендеру:

- Транзакция в мемпуле ≠ транзакция в блоке. UI должен показывать pending → confirming → confirmed.
- После отправки транзакции данные на фронте не меняются мгновенно — нужен либо polling, либо события, либо Subgraph.

- Газ стоит денег → каждая `writeContract()` должна быть оправдана.
- Чтение (`call/readContract`) — бесплатно.

Что хотят услышать

- Блоки, хеши, связь между блоками (цепочка)
- Консенсус (PoS), валидаторы, финальность
- EVM и детерминированность
- Связь с фронтендом: почему транзакции не мгновенные, почему нужен Subgraph
- «Блокчейн — это распределённая база данных» (слишком поверхностно)

2. Что такое газ (gas)? Почему одна транзакция стоит \$1, а другая \$50?

Уровень: Junior

Что спрашивают на самом деле

Понимаешь ли ты, что каждая строчка кода в контракте стоит денег, и как это влияет на UX.

Развёрнутый ответ

Газ — единица измерения вычислительной работы в Ethereum. Каждая операция EVM имеет фиксированную стоимость в газе:

Операция	Газ
ADD (сложение)	3
SSTORE (запись в storage)	20 000 (новый слот) / 2 900 (обновление)
SLOAD (чтение из storage)	2 100 (холодное) / 100 (тёплое)
CALL (вызов другого контракта)	от 2 600
Базовая стоимость транзакции	21 000

Формула стоимости:

Стоимость = `gasUsed` × `gasPrice`

- **gasUsed** — сколько газа реально потратила транзакция (зависит от сложности контракта)
- **gasPrice** — цена за единицу газа в gwei (рыночная — зависит от загрузки сети)

Почему цены разные:

- Простой перевод ETH: ~21 000 газа → даже при 100 gwei = ~\$5
- Свop через Uniswap V3: 150 000–300 000 газа → при 100 gwei = ~\$35–70
- Минт NFT с загрузкой в storage: 100 000+ газа

- Сложная композитруемая операция (multicall + flash loan): 500 000+ газа

EIP-1559 (с августа 2021) изменил модель:

- **baseFee** — сжигается, адаптивно растёт/падает от загрузки
- **priorityFee** (чаевые) — валидатору за включение в блок

```
gasPrice = baseFee + priorityFee
```

Для фронтендера:

- Показывай **оценку газа перед отправкой**: estimateGas() из viem/ethers
- Дай пользователю выбрать priority fee (slow/medium/fast)
- Показывай экономию при L2 (Arbitrum: газ в 10-50x дешевле)

Что хотят услышать

- Газ = плата за вычисления, каждая операция EVM стоит газ
- Разница между gasUsed (сложность) и gasPrice (рынок)
- EIP-1559: baseFee + priorityFee
- Почему storage-операции дорогие
- Связь с фронтендом: estimateGas, выбор комиссии, UX

3. Чем отличается EOA от Contract Account?

Уровень: Junior/Middle

Развёрнутый ответ

В Ethereum два типа аккаунтов:

	EOA	Contract Account
Контролируется	Приватным ключом	Кодом (байткодом)
Может инициировать транзакцию	Да	Нет (только в ответ на вызов)
Имеет баланс ETH		
Имеет storage		
Имеет код		
Газ	21 000 база	21 000 + выполнение кода
Создание	Генерация ключей (бесплатно)	Деплой транзакцией (платно)

EOA (Externally Owned Account):

- Твой кошелёк MetaMask
- Адрес = последние 20 байт от кесак256(publicKey)
- Может отправлять ETH и вызывать контракты
- Не имеет storage — нельзя хранить данные «внутри» EOA

Contract Account:

- Адрес = хеш от (адрес создателя + nonce) — CREATE
- CREATE2: адрес = хеш от (0xFF + sender + salt + bytecodeHash) — предсказуемый адрес!
- Имеет storage (persistent key-value хранилище)
- Код неизменяем после деплоя (кроме прокси-паттернов)

Для фронтендера важно:

- `msg.sender` в контракте — это всегда адрес того, кто инициировал транзакцию. Если контракт А вызывает контракт В, то `msg.sender` в В = адрес контракта А (а не оригинального пользователя). Это критично для безопасности.
- Чтобы проверить, контракт ли адрес: `getCode(address) !== '0x'` в `viem`.

Что хотят услышать

EOA = ключ, Contract = код
Только EOA может инициировать транзакцию
Contract имеет storage, EOA — нет
`msg.sender` и цепочка вызовов
Как отличить на фронте: `getCode()`

4. Что такое nonce и зачем он нужен?

Уровень: Middle

Развёрнутый ответ

Nonce — счётчик транзакций для каждого EOA (аккаунта). Начинается с 0 и увеличивается на 1 с каждой подтверждённой транзакцией.

Две функции nonce:

1. **Порядок транзакций** — гарантирует, что транзакции от одного адреса выполняются строго последовательно. Нельзя отправить транзакцию с nonce 5, пока не подтверждены 0-4.
2. **Защита от повторной отправки (replay protection)** — одну и ту же подписанную транзакцию нельзя включить в блокчейн дважды. После включения nonce «занят».

Пример проблемы:

Отправил swap (nonce=3) → ещё не попала в блок
Отправил approve (nonce=4) → застряла, ждёт nonce 3!

Как это влияет на фронтенд:

```
// wagmi получает nonce автоматически через viem
const { writeContract } = useWriteContract()

// Но если нужно управлять вручную – возможны коллизии:
// Пользователь в двух вкладках отправил транзакции с одинаковым nonce.
// Одна пройдёт, вторая – revert «nonce too low» или «replaced».
```

Боевые ситуации:

- **Застрявшая транзакция (stuck):** nonce 3 pending, а nonce 4 не может выполняться. Решение: отправить «пустую» транзакцию с nonce 3 и высоким газом (speed up / cancel в MetaMask).
- **Замена транзакции:** отправить новую с тем же nonce и газом выше на 10%+ — первая будет вытеснена (replaced).

Что хотят услышать

- Nonce = счётчик транзакций аккаунта
- Гарантирует порядок и защиту от повторов
- Проблема застрявших транзакций
- Как ускорять/отменять (speed up / cancel)
- Как фронтенд получает nonce (автоматически в wagmi/viem)

5. Как работает Proof of Stake в Ethereum?

Уровень: Middle

Развёрнутый ответ

Proof of Stake (PoS) — механизм консенсуса Ethereum с сентября 2022 (The Merge). Заменяет Proof of Work (майнинг).

Как работает:

1. **Валидаторы** ставят 32 ETH в депозитный контракт. Сейчас ~1 млн валидаторов.
2. **Слоты и эпохи:**
3. Слот = 12 секунд. В каждом слоте один случайный валидатор предлагает блок.
4. Эпоха = 32 слота (~6.4 минуты). Каждую эпоху валидаторы голосуют за блоки (attestations).
5. **Комитеты:** валидаторы разбиваются на комитеты (≥ 128 валидаторов). Один комитет на слот.
6. **Финальность (finality):** блок становится «оправданным» (justified) после голосования 2/3 комитета. После двух последовательных justified-эпох наступает финальность — блок нельзя откатить без сжигания $> 1/3$ всего stake.

Наказания:

- **Пропуск слота:** небольшой штраф (~\$1)
- **Двойное предложение / двойное голосование:** slashing — потеря части stake (минимум 1 ETH) и принудительный выход
- **Попытка атаки 51%:** потеря всего stake

Для фронтендера:

- Транзакция подтверждена (в блоке) ≠ финализирована. Финальность — через ~12.8 минут.
- Для большинства dApp достаточно 1-2 подтверждений. Для крупных сумм (>\$100K) ждут финальности.
- L2 (Arbitrum, Optimism) наследуют безопасность Ethereum, но имеют свою финальность.

Что хотят услышать

- Валидаторы ставят ETH, предлагают блоки
- Слоты (12 сек), эпохи, комитеты
- Финальность через 2 эпохи (~12.8 мин)
- Slashing за жульничество
- Практическое значение: сколько подтверждений ждать

Смарт-контракты

6. Что такое ABI и зачем он нужен фронтендеру?

Уровень: Junior

Что спрашивают на самом деле

Это самый практичный вопрос для фронтендера. Без ABI ты не можешь вызвать ни одну функцию контракта. Проверяют, понимаешь ли ты, откуда берётся ABI и как его использовать.

Развёрнутый ответ

ABI (Application Binary Interface) — JSON-описание интерфейса смарт-контракта. Это как «TypeScript-типы» для контракта, только в JSON.

Как выглядит ABI:

```
[
  {
    "type": "function",
    "name": "balanceOf",
    "inputs": [{"name": "owner", "type": "address"}],
    "outputs": [{"name": "", "type": "uint256"}],
    "stateMutability": "view"
  },
  {
    "type": "function",
    "name": "transfer",
    "inputs": [
      {"name": "to", "type": "address"},
      {"name": "amount", "type": "uint256"}
    ],
    "outputs": [{"name": "", "type": "bool"}],
    "stateMutability": "nonpayable"
  },
  {
    "type": "event",
    "name": "Transfer",
    "inputs": [
      {"name": "from", "type": "address", "indexed": true},
      {"name": "to", "type": "address", "indexed": true},
      {"name": "value", "type": "uint256", "indexed": false}
    ]
  }
]
```

Зачем ABI фронтендеру:

1. **Кодирование вызовов** — `balanceOf(address)` → `0x70a0823100000000000000000000000000000000000000000000000000000000abc...` (4 байта селектора + 32 байта аргумента)
2. **Декодирование ответов** — сырые байты → понятные значения
3. **Парсинг событий** — сырые логи → `{ from, to, value }`

Где взять ABI:

- Из Hardhat/Foundry после компиляции (`artifacts/contracts/Token.json`)
- Из Etherscan (вкладка Contract → Code → Contract ABI)
- Из npm-пакетов (`@uniswap/v3-core`)
- **Важно:** верифицировать ABI! Несовпадение ABI и реального байткода → непредсказуемое поведение.

Использование в коде:


```
// viem — типобезопасно через Human Readable ABI
const balance = await publicClient.readContract({
  address: '0x...',
  abi: erc20Abi,
  functionName: 'balanceOf',
  args: ['0x...'],
})

// wagmi — через хук
const { data: balance } = useReadContract({
  address: tokenAddress,
  abi: erc20Abi,
  functionName: 'balanceOf',
  args: [userAddress],
})
```

Что хотят услышать

- ABI = JSON-описание функций и событий контракта
- Нужен для кодирования вызовов и декодирования ответов
- Где брать ABI (компиляция, Etherscan, пакеты)
- Как использовать в viem/wagmi
- Связь с type safety

7. ERC-20 vs ERC-721: в чём разница?

Уровень: Junior/Middle

Развёрнутый ответ

ERC-20 — стандарт взаимозаменяемых (fungible) токенов.

ERC-721 — стандарт невзаимозаменяемых (non-fungible) токенов — NFT.

	ERC-20	ERC-721
Взаимозаменяемость	1 USDC = любой другой 1 USDC	Каждый токен уникален
Баланс	balanceOf(address) → число	balanceOf(address) → количество (какие именно — отдельно)
Перевод	transfer(to, amount)	transferFrom(from, to, tokenId)
Информация о токене	name(), symbol(), decimals()	Дополнительно: tokenURI(tokenId) → метаданные
Approval	approve(spender, amount) — на сумму	approve(to, tokenId) — на конкретный токен
Владелец	Баланс (безличный)	ownerOf(tokenId) → конкретный адрес

Интерфейс ERC-20:

```

function totalSupply() view returns (uint256)
function balanceOf(address) view returns (uint256)
function transfer(address to, uint256 amount) returns (bool)
function approve(address spender, uint256 amount) returns (bool)
function transferFrom(address from, address to, uint256 amount) returns (bool)
function allowance(address owner, address spender) view returns (uint256)

event Transfer(address from, address to, uint256 value)
event Approval(address owner, address spender, uint256 value)

```

Интерфейс ERC-721 (дополнительно к ERC-165):

```

function ownerOf(uint256 tokenId) view returns (address)
function tokenURI(uint256 tokenId) view returns (string)
function safeTransferFrom(address from, address to, uint256 tokenId)

event Transfer(address from, address to, uint256 tokenId) // value = tokenId!
event Approval(address owner, address approved, uint256 tokenId)
event ApprovalForAll(address owner, address approved operator, bool approved)

```

Для фронтендера:

ERC-20:

```

// Показать баланс пользователя
const { data: balance } = useReadContract({
  abi: erc20Abi,
  functionName: 'balanceOf',
  args: [address],
})
// balance = 1500000000000000000n → форматируем: 1.5 USDC

// Отправить токены
const { writeContract } = useWriteContract()
writeContract({ abi: erc20Abi, functionName: 'transfer', args: [to, amount] })

```

ERC-721:

```
// Получить все NFT пользователя – сложнее!
// Вариант 1: Subgraph (индексирует Transfer события)
// Вариант 2: Alchemy / Moralis NFT API
// Вариант 3: balanceOf + tokenOfOwnerByIndex (если есть enumerable
extension)

// Показать картинку NFT
const { data: tokenURI } = useReadContract({
  abi: erc721Abi,
  functionName: 'tokenURI',
  args: [tokenId],
})
// tokenURI = "ipfs://Qm.../123.json" или "https://metadata.nft.com/123"
```

Вариации:

- **ERC-1155** — мультитокен: один контракт управляет и fungible, и non-fungible токенами. Часто используется в играх.

➤ **ERC-721A** (Azuki) — газ-оптимизированный мент нескольких NFT в одной транзакции.

➤ **Soulbound (SBT)** — NFT, который нельзя передать (нет transfer).

Что хотят услышать

➤ Fungible vs non-fungible, ключевые отличия интерфейсов

balanceOf возвращает число vs количество

tokenURI — отдельный метод только у ERC-721

Как показывать NFT на фронте (Subgraph/Alchemy/
tokenOfOwnerByIndex)

Упомянуть ERC-1155 и Soulbound как бонус

8. События (events) в Solidity: как их слушать на фронте?

Уровень: Middle

Развёрнутый ответ

Events — это логи, которые смарт-контракт испускает при выполнении. Они хранятся в блокчейне вечно, но **недоступны изнутри смарт-контракта** (контракт не может прочитать свои же события).

Объявление в Solidity:

```
event Transfer(address indexed from, address indexed to, uint256 value);

function transfer(address to, uint256 amount) public returns (bool) {
    balances[msg.sender] -= amount;
    balances[to] += amount;
    emit Transfer(msg.sender, to, amount); // испускаем событие
    return true;
}
```

- indexed — до 3 параметров можно пометить как indexed → по ним можно фильтровать
- Не-indexed параметры хранятся в data (не фильтруются, но читаются)

Чтение событий на фронтенде:

Способ 1: getLogs (исторические события)

```
import { publicClient } from './config'

const logs = await publicClient.getLogs({
    address: tokenAddress,
    event: parseAbiItem('event Transfer(address indexed from, address indexed to, uint256 value)'),
    args: { from: userAddress }, // фильтр по from
    fromBlock: 19000000n,
    toBlock: 'latest',
})
```

Способ 2: watchEvent (реальное время через WebSocket)

```
const unwatch = publicClient.watchEvent({
    address: tokenAddress,
    event: parseAbiItem('event Transfer(address indexed from, address indexed to, uint256 value)'),
    onLogs: (logs) => {
        for (const log of logs) {
            console.log(`${log.args.from} → ${log.args.to}: ${log.args.value}`)
        }
    },
})
// unwatch() — отписаться
```

Как читать watchEvent({ event: parseAbiItem('event Transfer(...)'), onLogs }): «подпишись на событие Transfer контракта: parseAbiItem превращает сигнатуру события в фильтр, onLogs — колбэк, который срабатывает каждый раз когда блокчейн рождает новый Transfer. unwatch() отписывает». Мнемоника: watchEvent = WebSocket-подписка на блокчейн-события: «кто-то перевёл токен — покажи мне прямо сейчас».

Способ 3: wagmi хук useWatchContractEvent

```
import { useWatchContractEvent } from 'wagmi'
```

```
useWatchContractEvent({
  address: tokenAddress,
  abi: erc20Abi,
  eventName: 'Transfer',
  onLogs(logs) {
    // logs – массив событий
  },
})
```

Способ 4: Subgraph (для сложных запросов)

```
{
  transfers(first: 10, where: { from: "0x..." }, orderBy: timestamp, orderDirection: desc) {
    id
    from
    to
    value
    timestamp
  }
}
```

Когда что использовать:

Нужно	Инструмент
История операций пользователя	Subgraph (индексация)
Реальное время (live feed)	watchEvent (WebSocket)
Разовый запрос данных	getLogs
Статистика/аналитика	Dune Analytics (над Subgraph)

Что хотят услышать

Events = логи в блокчейне, контракт их не читает

indexed параметры = фильтруемые

Три способа чтения: getLogs, watchEvent, Subgraph

Когда что использовать (история vs real-time vs аналитика)

Понимание, что события дешевле storage и часто используются для «истории»

9. Storage vs Memory vs Calldata в Solidity

Уровень: Middle/Senior

Развёрнутый ответ

Три области данных в Solidity. Понимать их критически важно для оптимизации газа и безопасности.

	Storage	Memory	Calldata
Где хранится	В блокчейне (persistent)	В памяти EVM (временная)	В данных транзакции (read-only)
Время жизни	Между транзакциями	Только во время вызова	Только во время вызова
Стоимость	ОЧЕНЬ дорого	Умеренно (растёт с размером)	Дёшево
Изменяемость	mutable	mutable	read-only
Где используется	Переменные состояния	Локальные переменные	Аргументы external-функций

Пример кода:

```
contract Example {
    // STORAGE – хранится в блокчейне, дорого!
    uint256[] public storedArray; // storage

    function example(
        uint256[] calldata _input // calldata – read-only, дёшево
    ) external {
        uint256[] memory temp = new uint256[](3); // memory – временно

        // Чтение из storage – SLOAD (2100 холодное / 100 тёплое)
        uint256 x = storedArray[0];

        // Запись в storage – SSTORE (20000 новый / 2900 обновление)
        storedArray.push(x);

        // memory → storage – дорого (копирование каждого элемента)
        storedArray = temp; //

        // calldata → memory – дёшево
        uint256[] memory copy = _input; //
    }
}
```

Правила и подводные камни:

1. **External-функции** — используй calldata для массивов и строк вместо memory. Экономия газа 5–10x.
2. **Storage-указатели опасны:**

```
function bad() public {
    MyStruct storage s = storedStructs[0]; // s – УКАЗАТЕЛЬ, не копия!
    s.value = 100; // меняет storedStructs[0] в storage!
}
```

1. **Чтение из storage в цикле — антипаттерн:**

```
// Плохо: SLOAD на каждой итерации
for (uint i = 0; i < storedArray.length; i++) { ... }
```

```
// Хорошо: один SLOAD в memory
uint256 len = storedArray.length;
for (uint i = 0; i < len; i++) { ... }
```

Для фронтендера: эти знания нужны, чтобы:

- Понимать, почему одни функции стоят дороже других (пишут в storage vs читают)
- Читать код контракта и оценивать газ
- Понимать отчёты о газе в Hardhat/Foundry (gas reporter)

Что хотят услышать

Storage = блокчейн, memory = RAM EVM, calldata = входные данные
Разница в стоимости: SSTORE дорогой, calldata дешёвый
Storage-указатели и почему они опасны
SLOAD в цикле — антипаттерн
calldata для external-функций

10. Как работает approve + transferFrom в ERC-20?

Уровень: Middle

Что спрашивают на самом деле

Это фундаментальный паттерн DeFi. Каждый своп на Uniswap начинается с approve. Проверяют, понимаешь ли ты двухшаговый процесс и UX-последствия.

Развёрнутый ответ

Проблема: я хочу, чтобы контракт Uniswap свопнул мои USDC на ETH. Но transfer может вызвать только владелец токенов (я). Как дать право Uniswap перевести мои токены?

Решение: approve + transferFrom

Шаг 1: approve
Пользователь → ERC-20 контракт: «Разрешаю Uniswap потратить 1000 USDC с моего адреса»
approve(uniswapRouter, 1000e18)

Шаг 2: transferFrom
Uniswap → ERC-20 контракт: «Перевожу 1000 USDC от пользователя на пул»
transferFrom(user, pool, 1000e18)

Как читать approve(spender, amount) → transferFrom(owner, to, amount): «шаг 1: ты говоришь токену "разрешаю контракту spender потратить до amount моих токенов" — это как подписать чек с

лимитом. Шаг 2: spender говорит токену "переведи amount от owner на to" — контракт проверяет чек и выполняет перевод». Мнемоника: approve = выписал чек, transferFrom = кто-то предъявил чек к оплате; два шага, две транзакции, два газа.

Solidity (как это внутри ERC-20):

```
mapping(address => mapping(address => uint256)) private _allowances;

function approve(address spender, uint256 amount) public returns (bool) {
    _allowances[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}

function transferFrom(address from, address to, uint256 amount) public returns (bool) {
    // Проверяем, что msg.sender (spender) имеет разрешение
    uint256 currentAllowance = _allowances[from][msg.sender];
    require(currentAllowance >= amount, "insufficient allowance");

    _allowances[from][msg.sender] = currentAllowance - amount;
    _transfer(from, to, amount);
    return true;
}
```

Фронтенд — двухшаговый UX:


```

function SwapButton() {
  // Шаг 1: Проверяем allowance
  const { data: allowance } = useReadContract({
    abi: erc20Abi,
    functionName: 'allowance',
    args: [userAddress, uniswapRouter],
  })

  const needApproval = allowance < amountToSwap

  // Шаг 2a: Если allowance недостаточен — approve
  const { writeContract: approve } = useWriteContract()

  // Шаг 2b: Если allowance достаточен — swap
  const { writeContract: swap } = useWriteContract()

  if (needApproval) {
    return <button onClick={() => approve({
      abi: erc20Abi,
      address: tokenAddress,
      functionName: 'approve',
      args: [uniswapRouter, maxUint256], // infinite approval!
    })}>Approve USDC</button>
  }

  return <button onClick={() => swap({
    abi: uniswapRouterAbi,
    address: uniswapRouter,
    functionName: 'swapExactTokensForTokens',
    args: [amountIn, amountOutMin, path, userAddress, deadline],
  })}>Swap</button>
}

```

Важные нюансы:

- **Infinite approval** — `approve(spender, type(uint256).max)` — чтобы не делать approve каждый раз. Но риск: если контракт взломан — могут украсть весь allowance.
- **Race condition** — если изменить allowance с 100 на 200, пока транзакция в мемпуле, `spender` может успеть потратить и старый (100), и новый (200) allowance. Решение: сначала сбросить в 0, потом установить новое значение. Или использовать `increaseAllowance/decreaseAllowance`.
- **Permit (EIP-2612)** — `gasless approve` через подпись. Пользователь подписывает сообщение, контракт верифицирует подпись и устанавливает allowance. Одна транзакция вместо двух!

Что хотят услышать

Двухшаговый процесс: approve → transferFrom

_allowances mapping: owner → spender → amount

Как реализовать UX: проверить allowance → кнопка Approve или Swap

Infinite approval: плюсы и риски

Race condition в approve и как избежать

Упомянуть Permit (EIP-2612) — бонус

11. Как прочитать и вызвать смарт-контракт из React?

Уровень: Junior/Middle

Развёрнутый ответ

Чтение (read) — бесплатно, мгновенно (с точки зрения пользователя):

```
import { useReadContract } from 'wagmi'
import { erc20Abi } from './abis/erc20'

function TokenBalance({ userAddress }: { userAddress: string }) {
  const { data: balance, isLoading, error } = useReadContract({
    address: '0xdAC17F958D2ee523a2206206994597C13D831ec7', // USDT
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: [userAddress],
  })

  // balance = 15000000000n (BigInt)
  const formatted = balance ? formatUnits(balance, 6) : '0' // USDT = 6
  decimals

  if (isLoading) return <div>Загрузка баланса...</div>
  return <div>Баланс: {formatted} USDT</div>
}
```

Запись (write) — платно, требует подтверждения:

```

import { useWriteContract, useWaitForTransactionReceipt } from 'wagmi'

function SendToken({ to }: { to: string }) {
  const { writeContract, data: hash, isPending } = useWriteContract()

  const { isLoading: isConfirming, isSuccess } = useWaitForTransactionRe
ceipt({
    hash,
  })

  const send = () => {
    writeContract({
      address: '0xdAC17F958D2ee523a2206206994597C13D831ec7',
      abi: erc20Abi,
      functionName: 'transfer',
      args: [to, parseUnits('10', 6)], // 10 USDT
    })
  }

  return (
    <div>
      <button onClick={send} disabled={isPending}>
        {isPending ? 'Подтвердите в кошельке...' : 'Отправить 10 USDT'}
      </button>
      {hash && <div>Tx: {hash}</div>}
      {isConfirming && <div>Подтверждается...</div>}
      {isSuccess && <div>    Отправлено!</div>}
    </div>
  )
}

```

Мульти-контрактное чтение (multicall) — для производительности:

```

import { useReadContracts } from 'wagmi'

function TokenInfo({ tokens }: { tokens: string[] }) {
  const { data } = useReadContracts({
    contracts: tokens.flatMap(addr => [
      { address: addr, abi: erc20Abi, functionName: 'name' },
      { address: addr, abi: erc20Abi, functionName: 'symbol' },
      { address: addr, abi: erc20Abi, functionName: 'decimals' },
    ]),
  })
  // data = [('Tether', 'USDT', 6), ('USD Coin', 'USDC', 6), ...]
}

```

Как читать `useReadContracts({ contracts: tokens.flatMap(...)}):`

«возьми массив адресов токенов, для каждого размножь в три запроса (name, symbol, decimals) через flatMap — и получи все ответы

одним батчем в том же порядке». Мнемоника: `useReadContracts` = мультитул для чтения: много контрактов × много функций = один вызов, один массив ответов.

Без wagmi — напрямую через viem:

```
import { createPublicClient, http } from 'viem'
import { mainnet } from 'viem/chains'

const publicClient = createPublicClient({
  chain: mainnet,
  transport: http(),
})

const balance = await publicClient.readContract({
  address: '0xdAC17F958D2ee523a2206206994597C13D831ec7',
  abi: erc20Abi,
  functionName: 'balanceOf',
  args: ['0x...'],
})
```

Что хотят услышать

- Чтение (`readContract` / `useReadContract`) — бесплатно, RPC-нода
- Запись (`writeContract` / `useWriteContract`) — платно, цепочка состояний
- Жизненный цикл: `idle` → `pending` (кошелёк) → `confirming` (майнинг) → `confirmed`
 - `useReadContracts` для мульти-чтения (`multicall`)
 - `formatUnits` / `parseUnits` для работы с decimals

dApp-фронтенд

12. Как подключить кошелёк пользователя к dApp?

Уровень: Junior

Что спрашивают на самом деле

Самый частый вопрос. Проверяют, знаешь ли ты современный стек (RainbowKit, wagmi, WalletConnect) и понимаешь ли весь flow от кнопки до подписанной транзакции.

Развёрнутый ответ

Современный способ: RainbowKit + wagmi + viem

```
// 1. Установка: npm i @rainbow-me/rainbowkit wagmi viem @tanstack/react-
query

// 2. Конфигурация (config.ts)
import { getDefaultConfig } from '@rainbow-me/rainbowkit'
import { mainnet, polygon, arbitrum } from 'wagmi/chains'

export const config = getDefaultConfig({
  appName: 'My dApp',
  projectId: 'YOUR_WALLETCONNECT_PROJECT_ID', // из https://
cloud.walletconnect.com
  chains: [mainnet, polygon, arbitrum],
  ssr: false,
})

// 3. Провайдеры (main.tsx)
import { RainbowKitProvider } from '@rainbow-me/rainbowkit'
import { WagmiProvider } from 'wagmi'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import '@rainbow-me/rainbowkit/styles.css'

const queryClient = new QueryClient()

function App() {
  return (
    <WagmiProvider config={config}>
      <QueryClientProvider client={queryClient}>
        <RainbowKitProvider>
          {/* Ваше приложение */}
        </RainbowKitProvider>
      </QueryClientProvider>
    </WagmiProvider>
  )
}

// 4. Кнопка Connect (любой компонент)
import { ConnectButton } from '@rainbow-me/rainbowkit'

function Header() {
  return <ConnectButton />
  // ГОТОВО: кнопка, модалка выбора кошелька, смена сети, баланс, адрес —
  всё из коробки!
}
```

Что происходит при нажатии «Connect»:

Пользователь → ConnectButton → Модалка RainbowKit → Выбор кошелька

лучей

MetaMask (injected)
window.ethereum
.request({
 method:
 'eth_requestAccounts'
})

WalletConnect
QR-код
(для мобильных
кошельков)
Сканирование → пара к

Аккаунт подключён
wagmi сохраняет:
- address
- chainId
- connector (MetaMask/WalletConnect/...)

Без RainbowKit — вручную через wagmi:

```
import { useConnect, useAccount, useDisconnect } from 'wagmi'
import { injected } from 'wagmi/connectors'

function ManualConnect() {
  const { connect } = useConnect()
  const { address, isConnected } = useAccount()
  const { disconnect } = useDisconnect()

  if (isConnected) {
    return (
      <div>
        {address}
        <button onClick={() => disconnect()}>Disconnect</button>
      </div>
    )
  }

  return (
    <button onClick={() => connect({ connector: injected() })}>
      Connect MetaMask
    </button>
  )
}
```

Важные нюансы:

- **Смена сети:** автоматически через RainbowKit, вручную — useSwitchChain
- **Disconnect:** не отключает MetaMask глобально, только ваш dApp «забывает» сессию

- **WalletConnect v2:** требует projectId из WalletConnect Cloud
- **SSR:** Next.js требует `ssr: false` в конфиге wagmi, потому что MetaMask — browser-only

Что хотят услышать

- RainbowKit + wagmi + viem как стандартный стек
- Полный flow: конфиг → провайдеры → ConnectButton
- injected (MetaMask) vs WalletConnect (QR)
- Как работает под капотом: `eth_requestAccounts`
- Смена сети, `disconnect`
- SSR-нюансы с Next.js

13. Жизненный цикл транзакции: как обработать все состояния в UI?

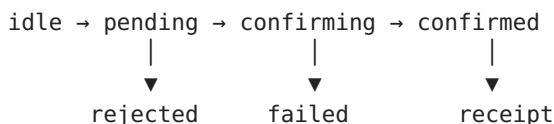
Уровень: Middle/Senior

Что спрашивают на самом деле

Это проверка твоего опыта. Проблема: пользователь нажал кнопку, прошло 30 секунд, ничего не происходит. Что показывать? Как не потерять транзакцию при перезагрузке? Как понять, что `revert`?

Развёрнутый ответ

Пять состояний транзакции:



Полная реализация компонента:

```

function TransactionButton() {
  const [status, setStatus] = useState<'idle' | 'pending' | 'confirming'
  | 'confirmed' | 'failed'>('idle')
  const [txHash, setTxHash] = useState<string | null>(null)
  const [error, setError] = useState<string | null>(null)

  const { writeContractAsync } = useWriteContract()
  const { refetch: refetchBalance } = useReadContract({ /* ... */ })

  const send = async () => {
    try {
      setStatus('pending')
      setError(null)

      // 1. Отправка – MetaMask показывает окно подтверждения
      const hash = await writeContractAsync({
        address: tokenAddress,
        abi: erc20Abi,
        functionName: 'transfer',
        args: [to, amount],
      })
      setTxHash(hash)
      setStatus('confirming')

      // 2. Ждём квитанцию
      const receipt = await publicClient.waitForTransactionReceipt({
hash })

> **Как читать writeContractAsync(...) → waitForTransactionReceipt({
hash }):** «вызови функцию контракта и жди Promise с хешем (кошелёк попро-
сит подпись), затем передай хеш в ожидание квитанции – второй Promise р-
азрешится когда транзакция попадёт в блок». Мнемоника: *writeContractAsy-
nc = «отправь и дай хеш», waitForTransactionReceipt = «жди пока блокчейн
примет»; связка двух await'ов.*

      // 3. Проверяем статус
      if (receipt.status === 'success') {
        setStatus('confirmed')
        await refetchBalance() // Обновляем баланс
      } else {
        setStatus('failed')
        setError('Транзакция вернула revert')
      }
    } catch (err: any) {
      // Пользователь отклонил в MetaMask
      if (err?.code === 'ACTION_REJECTED' || err?.message?.includes('re-
ected')) {
        setStatus('idle')
        setError('Вы отклонили транзакцию')
      } else {

```



```

        setStatus('failed')
        setError(parseTxError(err))
    }
}
}

return (
  <div>
    <button onClick={send} disabled={status !== 'idle'}>
      {status === 'idle' && 'Отправить'}
      {status === 'pending' && 'Подтвердите в кошельке...'}
      {status === 'confirming' && 'Транзакция обрабатывается...'}
      {status === 'confirmed' && 'Успешно!'}
      {status === 'failed' && 'Ошибка'}
    </button>

    {txHash && (
      <a href={`https://etherscan.io/tx/${txHash}`} target="_blank">
        Посмотреть на Etherscan ↗
      </a>
    )}

    {error && <div className="error">{error}</div>}

    {/* Прогресс-бар подтверждений */}
    {status === 'confirming' && <ConfirmationProgress hash={txHash!} r
equiredConfirmations={2} />}
  </div>
)
}

// Компонент прогресса подтверждений
function ConfirmationProgress({ hash, requiredConfirmations }: { hash: s
tring; requiredConfirmations: number }) {
  const { data: receipt } = useWaitForTransactionReceipt({ hash, confirm
ations: requiredConfirmations })

  // useWaitForTransactionReceipt уже ждёт N подтверждений.
  // Пока ждёт – isLoading = true.
  // Если нужен прогресс – опрашиваем вручную.
}

```

Критические UX-требования:

- 1. Не блокировать UI на время транзакции.** Пользователь может переключиться на другую вкладку. Сохраняя txHash в localStorage и восстанавливая состояние при возвращении.
- 2. «Застрявшая» транзакция:** если pending дольше 5 минут → показать кнопку «Ускорить» (speed up) или «Отменить».

3. **Revert с причиной:** расшифровать через `decodeErrorResult` и показать человеческое сообщение.

```
function parseTxError(error: any): string {
  if (error?.walk) {
    // viem: можно пройти по цепочке ошибок
    const decoded = error.walk(e => e?.data?.args?.[0])
    if (decoded) return decoded
  }
  // Ручной маппинг известных ошибок
  if (error?.message?.includes('insufficient funds')) return 'Недостаточ
но средств для оплаты газа'
  if (error?.message?.includes('transfer amount exceeds balance'))
return 'Недостаточно токенов'
  return 'Транзакция не удалась'
}
```

1. **Тост-уведомления:** не модалки! Пользователь не должен ждать.
- Показывай toast с хешем и статусом.

Что хотят услышать

- 5 состояний: idle → pending → confirming → confirmed/failed
- writeContract → хеш → waitForTransactionReceipt
- Обработка ошибок: rejected, revert, out of gas
- Persistence txHash в localStorage для восстановления при перезагрузке
- Ссылка на Etherscan
- Speed up / cancel для застрявших транзакций

14. wagmi vs ethers.js vs viem — что выбрать?

Уровень: Middle

Развёрнутый ответ

	ethers.js v6	viem v2	wagmi v3
Слой	Низкоуровневый	Низкоуровневый	React-хуки
Размер	~500KB	~50KB (tree-shakeable)	~100KB + viem
TypeScript	Родной, но исторически сложный	First-class, выведенные типы	First-class
Производительность	Средняя	Высокая	Высокая (viem под капотом)
React-интеграция	Ручная	Ручная	Из коробки
Документация	Обширная (старая)	Отличная	Отличная
Поддержка	Активная	Активная (та же команда wevm)	Активная

Рекомендация:

Если React-приложение:

→ wagmi + viem (viem уже идёт как зависимость wagmi)

Если Node.js / скрипты / CLI:

→ viem

Если legacy проект на ethers.js:

→ можно не переписывать, но новые проекты — viem

Почему viem быстрее и меньше:

- Tree-shakeable: импортируешь только нужные функции
- Нет классов — чистые функции
- Нативные BigInt (ethers.js долго использовал BigNumber)
- Оптимизированная сериализация RPC-запросов

Пример чтения баланса на всех трёх:

```
// ethers.js v6
import { ethers } from 'ethers'
const provider = new ethers.JsonRpcProvider(rpcUrl)
const balance = await provider.getBalance(address)

// viem
import { createPublicClient, http } from 'viem'
const publicClient = createPublicClient({ chain: mainnet, transport:
http() })
const balance = await publicClient.getBalance({ address })

// wagmi (React)
import { useBalance } from 'wagmi'
const { data: balance } = useBalance({ address })
```

Что хотят услышать

viem — современный низкоуровневый (лёгкий, быстрый, tree-shakeable)

wagmi — React-хуки поверх viem

ethers.js — legacy, но всё ещё много где используется

Разница в размере бандла, производительности, TypeScript

Практическая рекомендация: wagmi + viem для React

15. Как отображать «ждущие» транзакции и обновлять данные после подтверждения?

Уровень: Middle

Развёрнутый ответ

Это продолжение вопроса 13, но акцент на мульти-транзакционном UX и инвалидации кеша.

Проблема: пользователь отправил токены, баланс должен обновиться, но React Query кеширует старый ответ RPC.

Решение: инвалидация после подтверждения.

```
import { useQueryClient } from '@tanstack/react-query'
import { useWriteContract, useWaitForTransactionReceipt,
useReadContract } from 'wagmi'

function SendWithBalanceUpdate() {
  const queryClient = useQueryClient()
  const { writeContractAsync } = useWriteContract()

  const send = async () => {
    const hash = await writeContractAsync({
      address: tokenAddress,
      abi: erc20Abi,
      functionName: 'transfer',
      args: [to, amount],
    })

    // Ждём квитанцию
    const receipt = await publicClient.waitForTransactionReceipt({ hash })

    if (receipt.status === 'success') {
      // Инвалидируем ВСЕ запросы, связанные с этим контрактом и адресом
      queryClient.invalidateQueries({
        queryKey: useReadContract.getQueryKey({
          address: tokenAddress,
          functionName: 'balanceOf',
          args: [senderAddress],
        }),
      })
    }
    // Пользователь увидит обновлённый баланс автоматически!
  }
}
```

Мульти-транзакционный UX (очередь транзакций):

```

interface PendingTx {
  id: string
  hash: string
  description: string
  status: 'pending' | 'confirming' | 'confirmed' | 'failed'
  timestamp: number
}

function TransactionQueue() {
  const [pendingTxs, setPendingTxs] = useState<PendingTx[]>([])

  const addTx = (hash: string, description: string) => {
    const tx: PendingTx = {
      id: hash,
      hash,
      description,
      status: 'confirming',
      timestamp: Date.now(),
    }
    setPendingTxs(prev => [tx, ...prev])

    // Ждём подтверждения
    publicClient.waitForTransactionReceipt({ hash }).then(receipt => {
      setPendingTxs(prev =>
        prev.map(t => t.id === hash
          ? { ...t, status: receipt.status === 'success' ? 'confirmed' :
'failed' }
          : t
        )
      )
    })
  }

  return (
    <div className="tx-queue">
      {pendingTxs.map(tx => (
        <div key={tx.id} className={`tx-item tx-${tx.status}`}>
          <span className="tx-desc">{tx.description}</span>
          <span className="tx-status">
            {tx.status === 'confirming' && <Loader />}
            {tx.status === 'confirmed' && ' '}
            {tx.status === 'failed' && ' '}
          </span>
          <a href={`https://etherscan.io/tx/${tx.hash}`}>Tx </a>
        </div>
      )
    )}
    </div>
  )
}

```

Persistent queue (localStorage):

```
// Сохраняем при добавлении
localStorage.setItem('pendingTxs', JSON.stringify(pendingTxs))

// Восстанавливаем при загрузке
useEffect(() => {
  const saved = localStorage.getItem('pendingTxs')
  if (saved) {
    const txs: PendingTx[] = JSON.parse(saved)
    // Для каждой незавершённой — проверяем статус заново
    txs.filter(t => t.status === 'confirming').forEach(tx => {
      checkTxStatus(tx.hash)
    })
  }
}, [])
```

Что хотят услышать

- Инвалидация кеша React Query после подтверждения транзакции
- Очередь транзакций (массив pendingTxs)
- Persistent queue в localStorage
- Автоматическая проверка статуса незавершённых транзакций при загрузке
- Тост-уведомления вместо блокирующих модалок

16. Как обрабатывать ошибки транзакций и показывать понятные сообщения?

Уровень: Middle

Развёрнутый ответ

Транзакция может упасть по десятку причин. Задача фронтендера — превратить `0x08c379a0...` в «У вас недостаточно токенов USDC».

Типы ошибок:

1. **Отклонение пользователем** (user rejected)
2. **Revert смарт-контракта** (логика контракта)
3. **Недостаточно газа** (out of gas)
4. **Nonce-проблемы** (слишком низкий / повтор)
5. **Сетевые ошибки** (RPC недоступен, таймаут)

Полная функция парсинга:

```

import { decodeErrorResult, BaseError, ContractFunctionRevertedError } from 'viem'

function parseTransactionError(error: unknown): string {
  // 1. Пользователь отклонил
  if (typeof error === 'object' && error !== null) {
    const err = error as any
    if (
      err.code === 4001 || // MetaMask
      err.code === 'ACTION_REJECTED' || // ethers
      err.message?.includes('rejected') || // общее
      err.message?.includes('denied') // WalletConnect
    ) {
      return 'Вы отклонили транзакцию в кошельке'
    }
  }

  // 2. Revert контракта – пробуем расшифровать
  if (error instanceof BaseError) {
    // viem: цепочка ошибок
    const revertError = error.walk(
      (e) => e instanceof ContractFunctionRevertedError
    )

    if (revertError instanceof ContractFunctionRevertedError) {
      const errorName = revertError.data?.errorName ?? ''
      // Кастомные ошибки контракта
      const errorMessages: Record<string, string> = {
        'ERC20InsufficientBalance': 'Недостаточно токенов на балансе',
        'ERC20InsufficientAllowance': 'Недостаточно разрешения (approve)',
        'OwnableUnauthorizedAccount': 'Вы не владелец контракта',
        'ReentrancyGuardReentrantCall': 'Повторный вызов заблокирован',
      }
      if (errorMessages[errorName]) return errorMessages[errorName]

      // Если есть revert reason строка
      if (revertError.reason) return revertError.reason
    }
  }

  // 3. Общие ошибки
  const msg = (error as any)?.message ?? String(error)
  if (msg.includes('insufficient funds')) return 'Недостаточно ETH для оплаты газа'
  if (msg.includes('nonce too low')) return 'Nonce слишком низкий. Попробуйте сбросить активность MetaMask'
  if (msg.includes('nonce too high')) return 'Nonce слишком высокий. Возможно, есть pending-транзакция'
  if (msg.includes('gas required exceeds allowance')) return 'Газ

```

```

превышает лимит блока'
    if (msg.includes('timeout') || msg.includes('TIMEOUT')) return 'RPC-
нода не отвечает. Попробуйте позже'
    if (msg.includes('network')) return 'Ошибка сети. Проверьте
подключение'

    // 4. Fallback — сырая ошибка
    return `Ошибка транзакции: ${msg.slice(0, 100)}\`
}

```

Использование в компоненте:

```

function SendButton() {
  const [error, setError] = useState<string | null>(null)
  const { writeContractAsync } = useWriteContract()

  const handleSend = async () => {
    setError(null)
    try {
      const hash = await writeContractAsync({ /* ... */ })
      // успех
    } catch (err) {
      setError(parseTransactionError(err))
    }
  }

  return (
    <div>
      <button onClick={handleSend}>Отправить</button>
      {error && <div className="tx-error">{error}</div>}
    </div>
  )
}

```

Лучшие практики:

- **Никогда не показывай сырой стек ошибки пользователю** — это пугает и неинформативно
- **Логируй полную ошибку в консоль/Sentry** для отладки
- **Для кастомных ошибок контракта** определи mapping `errorName` → человеческое сообщение
- **Revert reason** в виде строки — устарело, но ещё встречается. Новые контракты используют кастомные ошибки (gas-эффективнее).

Что хотят услышать

Различать типы ошибок: `rejected`, `revert`, `gas`, `nonce`, `network`
`decodeErrorResult` / `ContractFunctionRevertedError` в `viem`

Маппинг кастомных ошибок на человеческие сообщения

User rejected обрабатывать отдельно (это не ошибка, а отмена)

Не показывать сырые ошибки пользователю

17. Как слушать события смарт-контракта в реальном времени в React?

Уровень: Middle/Senior

Развёрнутый ответ

Способ 1: wagmi useWatchContractEvent

```
import { useWatchContractEvent } from 'wagmi'
import { parseAbiItem } from 'viem'

function LiveTransfers({ tokenAddress }: { tokenAddress: string }) {
  const [transfers, setTransfers] = useState<Transfer[]>([])

  useWatchContractEvent({
    address: tokenAddress,
    abi: erc20Abi,
    eventName: 'Transfer',
    onLogs(logs) {
      for (const log of logs) {
        setTransfers(prev => [{
          hash: log.transactionHash,
          from: log.args.from!,
          to: log.args.to!,
          value: log.args.value!,
        }, ...prev].slice(0, 50)) // храним последние 50
      }
    },
  })

  return (
    <div>
      <h3>Live Transfers</h3>
      {transfers.map((t, i) => (
        <div key={i}>
          {t.from.slice(0, 6)} → {t.to.slice(0, 6)}:
          {formatUnits(t.value, 18)}
        </div>
      ))}
    </div>
  )
}
```

Способ 2: viem watchEvent + useEffect (больше контроля)

```
function useLiveEvents(tokenAddress: string) {
  const [events, setEvents] = useState<any[]>([])

  useEffect(() => {
    const unwatch = publicClient.watchEvent({
      address: tokenAddress,
      event: parseAbiItem('event Transfer(address indexed from, address indexed to, uint256 value)'),
      onLogs: (logs) => {
        setEvents(prev => [...logs, ...prev].slice(0, 100))
      },
      // Фильтр: только новые блоки
      fromBlock: 'latest',
    })

    return () => {
      unwatch() // очистка при размонтировании!
    }
  }, [tokenAddress])

  return events
}
```

Способ 3: WebSocket провайдер (для максимальной производительности)

```
// viem автоматически использует WebSocket, если URL ws://
const publicClient = createPublicClient({
  chain: mainnet,
  transport: websocket('wss://mainnet.infura.io/ws/v3/YOUR_KEY'),
})
```

Важно: WebSocket vs HTTP polling

	WebSocket	HTTP Polling
Задержка	~1 сек (мгновенно при новом блоке)	~12 сек (интервал между блоками)
Нагрузка	Только при событиях	Каждые N секунд запрос
Сложность	Переподключение при обрыве	Проще
Поддержка RPC	Не все провайдеры	Все

Практические проблемы:

- **Переподключение:** WebSocket рвётся. Нужен reconnect с backoff.
- **Пропущенные блоки:** при разрыве — запросить getLogs за пропущенный диапазон.
- **Реконнект в React:** использовать useEffect cleanup + ref для хранения последнего обработанного блока.

```

function useRobustEventWatch(tokenAddress: string) {
  const lastBlockRef = useRef<bigint>(0n)

  useEffect(() => {
    let unwatch: () => void
    let retries = 0

    const startWatching = () => {
      unwatch = publicClient.watchEvent({
        address: tokenAddress,
        event: parseAbiItem('event Transfer(...)'),
        fromBlock: lastBlockRef.current || 'latest',
        onLogs: (logs) => {
          // обновляем lastBlock при каждом событии
          if (logs.length > 0) {
            const maxBlock = logs.reduce((max, l) => l.blockNumber >
max ? l.blockNumber : max, 0n)
            lastBlockRef.current = maxBlock
          }
          // обрабатываем логи...
        },
        onError: (error) => {
          console.error('Watch error:', error)
          retries++
          if (retries < 5) {
            setTimeout(startWatching, Math.min(1000 * 2 ** retries,
30000))
          }
        },
      })
    }

    startWatching()
    return () => unwatch?.()
  }, [tokenAddress])
}

```

Что хотят услышать

useWatchContractEvent из wagmi

watchEvent из viem для ручного контроля

WebSocket vs HTTP polling

Очистка подписки в useEffect cleanup

Переподключение при обрыве и восстановление пропущенных событий

18. Что такое multicall и зачем он на фронтенде?

Уровень: Middle/Senior

Развёрнутый ответ

Multicall — контракт, который выполняет несколько `eth_call` в одной RPC-транзакции. Экономит сетевые запросы.

Проблема без multicall:

```
Запрос 1: balanceOf(user, tokenA) → 200ms
Запрос 2: balanceOf(user, tokenB) → 200ms
Запрос 3: balanceOf(user, tokenC) → 200ms
...
Запрос 20: name(tokenA) → 200ms
Итого: 20 запросов × 200ms = 4 секунды!
```

С multicall:

```
1 запрос: multicall.aggregate([
  balanceOf(user, tokenA),
  balanceOf(user, tokenB),
  ...
]) → 200ms
```

Как работает:

Multicall контракт (задеплоен на каждом chain по известному адресу):

1. Принимает массив (`target`, `callData`)
2. Для каждого элемента делает `staticcall` в `target` с `callData`
3. Возвращает массив результатов

wagmi — автоматически через `useReadContracts`:

```

import { useReadContracts } from 'wagmi'

function Portfolio({ tokenList, userAddress }: Props) {
  const { data, isLoading } = useReadContracts({
    contracts: tokenList.flatMap(token => [
      {
        address: token.address,
        abi: erc20Abi,
        functionName: 'balanceOf',
        args: [userAddress],
      },
      {
        address: token.address,
        abi: erc20Abi,
        functionName: 'symbol',
      },
    ]),
  })

  // data = [balanceA, symbolA, balanceB, symbolB, ...]

  if (isLoading) return <div>Загрузка...</div>

  return (
    <ul>
      {tokenList.map((token, i) => (
        <li key={token.address}>
          {data?.[i * 2 + 1]}: {formatUnits(data?.[i * 2] ?? 0n, token.d
ecimals)}
        </li>
      ))}
    </ul>
  )
}

```

viem — напрямую:

```

import { multicall } from 'viem/actions'

const results = await multicall(publicClient, {
  contracts: [
    { address: tokenA, abi: erc20Abi, functionName: 'balanceOf', args: [u
serAddress] },
    { address: tokenB, abi: erc20Abi, functionName: 'balanceOf', args: [u
serAddress] },
  ],
  multicallAddress: '0xcA11bde05977b3631167028862bE2a173976CA11', //
Multicall3
})

```

Multicall3 — стандартный адрес на всех EVM-сетях:

- Адрес: 0xcA11bde05977b3631167028862bE2a173976CA11
- Деплоен на Ethereum, Arbitrum, Optimism, Polygon, BSC, Avalanche и сотнях других
- Поддерживает aggregate3 — каждая неудачная под-выборка не рушит весь multicall

Когда не нужен multicall:

- Один-два запроса — оверхед multicall не оправдан
- Запросы требуют разных blockNumber (multicall — один блок для всех подзапросов)

Что хотят услышать

- Multicall = много eth_call в одном RPC-запросе
- useReadContracts из wagmi делает это автоматически
- Multicall3: адрес 0xcA11b0..., доступен на всех сетях
- Экономия: 20 запросов → 1 запрос
- Когда НЕ нужен multicall

DeFi

19. Как работает Uniswap? Что такое AMM?

Уровень: Middle

Что спрашивают на самом деле

Понимаешь ли ты DeFi на уровне, достаточном для написания фронтенда свопа? Это не вопрос «расскажи формулу» — ждут понимание всего flow.

Развёрнутый ответ

Uniswap — децентрализованная биржа (DEX), работающая по модели **AMM (Automated Market Maker)**. Вместо книги ордеров (как на Binance) используется математическая формула.

Формула постоянного произведения ($x \times y = k$):

В пуле лежит x токенов А и y токенов В
 $k = x \times y$ — константа, которая НЕ меняется при свопах

Ты покупаешь Δx токенов А. Сколько заплатишь (Δy)?

Новое количество А: $x - \Delta x$

Новое количество В: $k / (x - \Delta x) = y + \Delta y$

$\Delta y = k / (x - \Delta x) - y$

Пример с числами:

Пул ETH/USDC: $10 \text{ ETH} \times 20\,000 \text{ USDC}$, $k = 200\,000$

Покупаю 1 ETH:

Новый баланс ETH: 9

Новый баланс USDC: $200\,000 / 9 \approx 22\,222$

Плачу: $22\,222 - 20\,000 = 2\,222 \text{ USDC}$

Покупаю ещё 1 ETH:

Новый баланс ETH: 8

Новый баланс USDC: $200\,000 / 8 = 25\,000$

Плачу: $25\,000 - 22\,222 = 2\,778 \text{ USDC}$ ← дороже!

Ключевые сущности для фронтендера:

1. **Router** (SwapRouter в V3, UniswapV2Router02 в V2):
2. `swapExactTokensForTokens(amountIn, amountOutMin, path, to, deadline)`
3. `amountOutMin` — минимальное количество, которое ты готов получить (slippage protection)
4. `deadline` — через сколько секунд транзакция станет невалидной
5. **Quoter** (только чтение, off-chain):
6. `quoteExactInputSingle(tokenIn, tokenOut, fee, amountIn, sqrtPriceLimitX96)`
7. Возвращает ожидаемое количество output-токенов
8. Вызывается перед свопом, чтобы показать пользователю курс
9. **Pool** (контракт пары токенов):
10. Хранит резервы и цену
11. `slot0()` в V3: `sqrtPriceX96` (текущая цена), `tick`, `observationIndex`

Frontend flow свопа:

```

function SwapComponent() {
  // 1. Котировка (quote) – читаем, сколько получим
  const { data: quote } = useReadContract({
    address: quoterAddress,
    abi: quoterAbi,
    functionName: 'quoteExactInputSingle',
    args: [tokenIn, tokenOut, fee, amountIn, 0n],
  })
  // quote = 1 850 000 000n (ожидаемое количество USDC)

  // 2. Рассчитываем amountOutMin с учётом slippage (например, 0.5%)
  const slippageBps = 50n // 0.5% = 50 bps
  const amountOutMin = quote
    ? quote - (quote * slippageBps) / 10000n
    : 0n

  // 3. Своп – одна транзакция (если уже есть approve)
  const { writeContract } = useWriteContract()

  const swap = () => {
    writeContract({
      address: swapRouterAddress,
      abi: swapRouterAbi,
      functionName: 'exactInputSingle',
      args: [{
        tokenIn,
        tokenOut,
        fee: 3000, // 0.3%
        recipient: userAddress,
        deadline: BigInt(Math.floor(Date.now() / 1000) + 1200), // 20
        amountIn,
        amountOutMinimum: amountOutMin,
        sqrtPriceLimitX96: 0n, // без лимита цены
      }],
    })
  }
}

```

Uniswap V2 vs V3:

- **V2:** простое $xy=k$, вся ликвидность по всей кривой ($0 \rightarrow \infty$). Низкая эффективность капитала.
- **V3:** концентрированная ликвидность — LP выбирают диапазон цены (tick range). Выше эффективность капитала, но сложнее для LP (impermanent loss выше).
- **V4 (скоро):** хуки (hooks) — кастомная логика на каждом свопе, синглтон-пул (весь V4 в одном контракте).

Что хотят услышать

АММ: $xy=k$, нет книги ордеров

Цена меняется нелинейно (slippage из-за размера сделки)

Router, Quoter, Pool — роли контрактов

Flow свопа: quote → slippage → amountOutMin → swap

V2 (простой) vs V3 (концентрированная ликвидность)

20. Что такое slippage и как его обрабатывать на фронтенде?

Уровень: Middle

Развёрнутый ответ

Slippage (проскальзывание) — разница между ожидаемой ценой и реальной ценой исполнения свопа.

Две причины slippage:

1. **Price impact** — твоя сделка настолько большая относительно пула, что сама двигает цену. Чем больше $\text{amountIn} / \text{poolLiquidity}$, тем выше price impact.
2. **Фронтраннинг/сэндвич** — кто-то вставил свою транзакцию перед твоей и изменил состояние пула.

Формула расчёта price impact (приближённая):

$$\text{priceImpact} = \text{amountIn} / (\text{poolReserve} + \text{amountIn})$$

Если в пуле 100 ETH, а ты покупаешь 10 ETH → price impact ~9%.

Как обрабатывать на фронтенде:

```

function SlippageControl() {
  const [slippage, setSlippage] = useState(0.5) // 0.5% default
  const [isAuto, setIsAuto] = useState(true)

  return (
    <div className="slippage-settings">
      <span>Slippage Tolerance</span>

      <div className="slippage-options">
        <button onClick={() => { setSlippage(0.1); setIsAuto(true) }}>
          >0.1%</button>
        <button onClick={() => { setSlippage(0.5); setIsAuto(true) }}>
          >0.5%</button>
        <button onClick={() => { setSlippage(1.0); setIsAuto(true) }}>
          >1.0%</button>

        <div className="custom">
          <input
            type="number"
            value={isAuto ? '' : slippage}
            onChange={e => {
              setIsAuto(false)
              setSlippage(Number(e.target.value))
            }}
            placeholder="Кастом"
          />
          <span>%</span>
        </div>
      </div>

      {/* Предупреждения */}
      {slippage < 0.1 && (
        <div className="warning"> Слишком низкий slippage: своп может н
          е выполняться</div>
      )}
      {slippage > 5 && (
        <div className="warning error"> Высокий slippage: вы можете по
          терять до {slippage}%!</div>
      )}
    </div>
  )
}

```

Price impact — предупреждения:

```
function PriceImpactWarning({ impact }: { impact: number }) {
  if (impact > 15) return (
    <div className="error-box">
      Price impact {impact.toFixed(2)}% – своп невозможен
    </div>
  )

  if (impact > 5) return (
    <div className="error-box">
      Очень высокий price impact: {impact.toFixed(2)}%
      Вы потеряете значительную сумму. Рекомендуем уменьшить размер сдел
ки.
    </div>
  )

  if (impact > 2) return (
    <div className="warning-box">
      Высокий price impact: {impact.toFixed(2)}%
    </div>
  )

  return null
}
```

Расчёт amountOutMin (защита от slippage):

```
const amountOutMin = expectedAmount - (expectedAmount *
BigInt(Math.floor(slippage * 100))) / 10000n
// slippage 0.5% = (expectedAmount * 50) / 10000

// ВАЖНО: slippage + price impact – это разные вещи!
// - price impact: показывает, насколько цена изменится от твоей сделки
// - slippage tolerance: твой лимит на проскальзывание (включая MEV)
// Если price impact 1%, а slippage 0.5% – транзакция вероятно
ревертнется!
```

Что хотят услышать

Slippage = разница между ожидаемой и реальной ценой

Price impact vs slippage tolerance

amountOutMin для защиты

UI: выбор slippage, предупреждения при высоких значениях

Взаимосвязь: если slippage tolerance < price impact → revert

21. Что такое MEV и sandwich attack?

Уровень: Middle/Senior

Развёрнутый ответ

MEV (Maximal Extractable Value) — прибыль, которую можно извлечь из манипуляции порядком транзакций в блоке.

Sandwich attack (самая распространённая MEV-атака):

Мемпул (публичный):

1. Жертва: swap 10 ETH → USDC (покупает ETH)
2. ...

Атакующий видит транзакцию жертвы и вставляет ДВЕ своих:

Блок:

1. Атакующий: swap USDC → ETH	← frontrun: покупает ETH до жертвы
2. Жертва: swap USDC → ETH	← жертва покупает по худшей цене
3. Атакующий: swap ETH → USDC	← backrun: продаёт ETH после жертвы

Результат:

- Жертва: получила меньше ETH, чем ожидала
- Атакующий: прибыль = разница в курсе - газ

Как защищаться на уровне пользователя:

1. **Slippage tolerance** — установить разумный (0.5–1%). Если слишком низкий — жертва сэндвича. Слишком высокий — теряешь на проскальзывании.
2. **Flashbots Protect** — отправить транзакцию в приватный мемпул, а не публичный.

Как поддержать на фронтенде:

```
// Отправка через Flashbots (приватный мемпул)
import { FlashbotsBundleProvider } from '@flashbots/ethers-provider-bundle'

async function sendPrivateTx(tx: TransactionRequest) {
  // Вариант 1: Flashbots RPC (mev-protect)
  const client = createPublicClient({
    transport: http('https://rpc.mevblocker.io'), // MEV Blocker
  })

  // Вариант 2: Flashbots Bundle
  const flashbots = await FlashbotsBundleProvider.create(provider, authSigner)
  const signedTx = await wallet.signTransaction(tx)
  const bundle = await flashbots.sendBundle([
    { signedTransaction: signedTx },
  ], targetBlock + 5)

  return bundle
}

// UI-опция для пользователя:
<div className="tx-options">
  <label>
    <input type="checkbox" checked={useMEVProtection} onChange={...} />
    MEV-защита (приватная отправка)
  </label>
</div>
```

Для фронтендера важно:

- ▶ Показывать предупреждение при высоком slippage (>1%)
- ▶ Показывать real-time цену и ожидаемый output
- ▶ Объяснять, почему транзакция вернула меньше токенов, чем было в цитате (MEV + price impact)

Что хотят услышать

MEV = прибыль от манипуляции порядком транзакций
 Sandwich: frontrun + жертва + backrun в одном блоке
 Slippage как защита
 Flashbots / приватный мемпул
 Как информировать пользователя на фронтенде

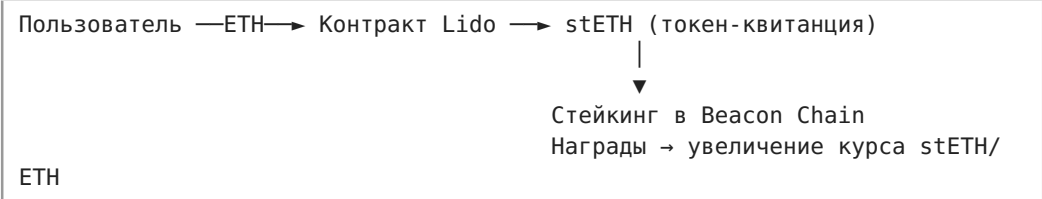
22. Как работает стейкинг с точки зрения фронтендера?

Уровень: Middle

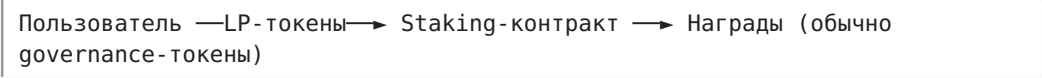
Развёрнутый ответ

Стейкинг — блокировка токенов в смарт-контракте для получения наград. Существует несколько видов:

1. Liquid Staking (Lido, Rocket Pool):



2. DeFi Staking/Yield Farming:



Фронтенд для liquid staking (stake ETH → stETH):

```

function StakeETH() {
  const [amount, setAmount] = useState('')
  const { writeContract, isPending } = useWriteContract()

  const { data: stEthBalance } = useReadContract({
    address: stEthAddress,
    abi: stEthAbi,
    functionName: 'balanceOf',
    args: [userAddress],
  })

  const stake = () => {
    writeContract({
      address: lidoAddress,
      abi: lidoAbi,
      functionName: 'submit',
      args: [referralAddress], // реферал (опционально)
      value: parseEther(amount), // ETH прилагается к транзакции!
    })
  }

  return (
    <div>
      <input
        type="number"
        value={amount}
        onChange={e => setAmount(e.target.value)}
        placeholder="Количество ETH"
      />
      <button onClick={stake} disabled={isPending}>
        {isPending ? 'Подтверждение...' : 'Stake'}
      </button>
      <div>Ваш stETH: {stEthBalance ? formatEther(stEthBalance) : '0'}</div>
    </div>
  )
}

```

DeFi Staking с наградами:

```

function FarmStaking() {
  const { writeContract } = useWriteContract()

  // 1. Stake LP-токены
  const stake = () => {
    writeContract({
      address: farmAddress,
      abi: farmAbi,
      functionName: 'stake',
      args: [amount],
    })
  }

  // 2. Claim награда
  const claim = () => {
    writeContract({
      address: farmAddress,
      abi: farmAbi,
      functionName: 'getReward',
    })
  }

  // 3. Чтение данных
  const { data: stakedBalance } = useReadContract({ /* userInfo */ })
  const { data: earnedRewards } = useReadContract({ /* earned */ })
  const { data: apr } = useReadContract({ /* rewardRate / totalStaked
  */ })

  return (
    <div>
      <div>Застейкано: {formatEther(stakedBalance ?? 0n)} LP</div>
      <div>Награды: {formatEther(earnedRewards ?? 0n)} TOKEN</div>
      <div>APR: {apr}%</div>
      <button onClick={stake}>Stake</button>
      <button onClick={claim}>Claim Rewards</button>
    </div>
  )
}

```

Unstake с задержкой (cooldown):


```

function UnstakeWithCooldown() {
  // Шаг 1: Запросить вывод (начинает cooldown)
  const { writeContract: requestWithdraw } = useWriteContract()

  // Шаг 2: Проверить, можно ли вывести
  const { data: canWithdraw } = useReadContract({
    address: stakingAddress,
    abi: stakingAbi,
    functionName: 'canWithdraw',
    args: [userAddress],
  })

  // Шаг 3: Вывести
  const { writeContract: withdraw } = useWriteContract()

  // Cooldown timer
  const { data: cooldownEnd } = useReadContract({
    address: stakingAddress,
    abi: stakingAbi,
    functionName: 'cooldownEnds',
    args: [userAddress],
  })

  return (
    <div>
      {!canWithdraw && (
        <button onClick={() => requestWithdraw({ /* ... */ })}>
          Request Withdraw
        </button>
      )}
      {canWithdraw && (
        <>
          <div>Cooldown ends: {new Date(Number(cooldownEnd) * 1000).toLocaleString()}</div>
          <button onClick={() => withdraw({ /* ... */ })}>Withdraw</button>
        </>
      )}
    </div>
  )
}

```

Что хотят услышать

Два типа: liquid staking (stETH) и DeFi farming (награды governance-токенами)

Stake → получение токенов-квитанций / начисление наград

Claim rewards, unstake с cooldown

Отображение APR/APY, наград, истории

Unstake с задержкой и UX для этого

23. Что такое Permit (EIP-2612) и как это упрощает UX свопа?

Уровень: Senior

Развёрнутый ответ

Проблема: обычный своп требует двух транзакций:

1. `approve(router, amount)` — разрешить тратить токены
2. `swap(...)` — сам своп

Две транзакции = двойной газ + двойное ожидание. Ужасный UX.

Permit (EIP-2612) — `gasless approve` через подпись. Пользователь подписывает сообщение (бесплатно, мгновенно), и своп выполняется в одной транзакции, которая внутри себя вызывает `permit()` + `transferFrom()`.

Как работает:

Без `permit`:

1. `approve(spender, amount)` ← транзакция (платно)
2. `swap(...)` ← транзакция (платно)

С `permit`:

1. `signTypedData(permitMsg)` ← подпись (бесплатно)
2. `swap(..., permitSignature)` ← ОДНА транзакция!
Внутри свопа: `permit(owner, spender, amount, deadline, v, r, s)`
Затем: `transferFrom(owner, pool, amount)`

Реализация на фронтенде:

```

import { useSignTypedData } from 'wagmi'

function SwapWithPermit() {
  const { signTypedDataAsync } = useSignTypedData()

  const swap = async () => {
    const deadline = BigInt(Math.floor(Date.now() / 1000) + 1200)

    // 1. Получаем nonce для permit (нужен правильный nonce)
    const nonce = await publicClient.readContract({
      address: tokenAddress,
      abi: erc20PermitAbi,
      functionName: 'nonces',
      args: [userAddress],
    })

    // 2. Подписываем permit
    const signature = await signTypedDataAsync({
      domain: {
        name: 'USD Coin',
        version: '1',
        chainId: chainId,
        verifyingContract: tokenAddress,
      },
      types: {
        Permit: [
          { name: 'owner', type: 'address' },
          { name: 'spender', type: 'address' },
          { name: 'value', type: 'uint256' },
          { name: 'nonce', type: 'uint256' },
          { name: 'deadline', type: 'uint256' },
        ],
      },
      primaryType: 'Permit',
      message: {
        owner: userAddress,
        spender: swapRouterAddress,
        value: amountToSwap,
        nonce: nonce,
        deadline: deadline,
      },
    })

    // 3. Расшифровываем подпись в r, s, v
    const { r, s, v } = parseSignature(signature)

    // 4. Свop с permit – ОДНА транзакция!
    writeContract({
      address: swapRouterWithPermit,
      abi: swapRouterAbi,

```

```

    functionName: 'swapWithPermit',
    args: [
        tokenIn, tokenOut, amountIn, amountOutMin,
        deadline, v, r, s, // permit-подпись
    ],
  },
})
}
}
}

```

Dai-style Permit (для токенов, не поддерживающих EIP-2612):

```

// Dai использует permit(holder, spender, nonce, expiry, allowed, v, r,
s)
// Важно: nonce у Dai – не счётчик, а случайное число (для защиты от
replay)

const nonce =
ethers.hexlify(ethers.randomBytes(32)) // уникальный каждый раз!

```

Permit2 (Uniswap) — универсальный permit:

Permit2 — отдельный контракт, через который можно делать permit для ЛЮБОГО ERC-20, даже если он не поддерживает EIP-2612.

1. approve(Permit2, maxUint256) — один раз на все токены
2. signPermit2 — подпись на конкретный трансфер
3. swap с Permit2 — все свопы через единую подпись

Что хотят услышать

Permit заменяет approve-транзакцию подписью (газ-экономия)

EIP-2612: EIP-712 typed data подпись

Flow: nonce → signTypedData → parseSignature → одна tx

Dai-style permit с random nonce

Permit2 от Uniswap — универсальное решение

UX: две транзакции → одна транзакция

Безопасность

24. Какие типичные уязвимости смарт-контрактов ты знаешь?

Уровень: Middle

Развёрнутый ответ

Фронтендер не обязан быть аудитором, но должен знать основные уязвимости, чтобы:

- Не продвигать скам-контракты
- Понимать предупреждения MetaMask/Revoke.cash
- Осмысленно отвечать на собеседовании

1. Reentrancy (повторный вход)

```
// Уязвимый контракт
function withdraw() public {
    uint256 amount = balances[msg.sender];
    (bool success, ) = msg.sender.call{value: amount}(""); // ← вызов ДО
обнуления баланса!
    require(success);
    balances[msg.sender] = 0; // ← слишком поздно!
}

// Атакующий контракт
receive() external payable {
    if (address(victim).balance > 0) {
        victim.withdraw(); // ← повторный вход!
    }
}
// Защита: Check-Effects-Interactions паттерн, ReentrancyGuard
```

2. Integer Overflow/Underflow (в Solidity <0.8)

```
// Solidity 0.7: 255 + 1 = 0 (переполнение)
uint8 x = 255;
x++; // x = 0!
// Solidity 0.8+: встроенная проверка, revert
```

3. Frontrunning

```
// Пользователь отправляет решение викторины
// Атакующий видит в мемпуле → копирует → даёт газ выше
// Решение атакующего выполняется первым → приз уходит ему
```

4. Access Control

```
function setOwner(address newOwner) external {
    // Забыли проверку: onlyOwner!
    owner = newOwner; // любой может стать владельцем
}
```

5. Unchecked External Call

```
function sendEth(address to, uint256 amount) public {
    to.call{value: amount}(""); // ← не проверяем success!
    // Если вызов упал — думаем, что ETH ушли, а они нет
}
```

6. Phishing / Approval Scam

Пользователь заходит на фейковый сайт Uniswap
Подписывает approve(scamContract, maxUint256)
Скам-контракт забирает BCE токены через transferFrom

Для фронтендера — что делать:

- Проверять адрес контракта на Etherscan (verified, не прокси на scam)

- Показывать пользователю, что именно он approve-ит
- Рекомендовать Revoke.cash для проверки и отзыва approvals
- Использовать transaction-simulation перед подписью (Wallet Guard, Pocket Universe)

Что хотят услышать

Reentrancy: причина, пример, защита (CEI, ReentrancyGuard)

Overflow/underflow: проблема в <0.8, фикс в 0.8+

Frontrunning: мемпул, опережение

Access control: забытая проверка onlyOwner

Phishing через approve: как защищаться

25. Как защитить пользователя от фишинговой транзакции на фронтенде?

Уровень: Middle/Senior

Развёрнутый ответ

Уровни защиты:

1. Верификация контрактов (Etherscan)

```
function VerifiedBadge({ address }: { address: string }) {
  const [isVerified, setIsVerified] = useState(false)

  useEffect(() => {
    // Проверяем через Etherscan API
    fetch(`https://api.etherscan.io/api?
module=contract&action=getabi&address=${address}`)
      .then(r => r.json())
      .then(data => setIsVerified(data.status === '1'))
  }, [address])

  if (!isVerified) return <span className="badge-unverified"> Unverifi
ed</span>
  return <span className="badge-verified"> Verified</span>
}
```

2. Симуляция транзакции перед отправкой

```
// Используем Tenderly Simulation API
async function simulateTx(tx: TransactionRequest) {
  const result = await tenderlyClient.simulateTransaction({
    from: tx.from,
    to: tx.to,
    value: tx.value,
    data: tx.data,
    blockNumber: 'latest',
  })

  // Анализируем изменения состояния
  const assetChanges = result.transaction.transactionInfo.assetChanges

  for (const change of assetChanges) {
    if (change.type === 'TRANSFER' && change.from === tx.from) {
      console.log(`    Уходят: ${change.amount} ${change.tokenInfo.symbol}`)
    }
  }

  return result
}
```

3. Предупреждения об известных скам-адресах

```
const SCAM_ADDRESSES = new Set([
  '0x000...scam1',
  '0x000...scam2',
])

function AddressWarning({ address }: { address: string }) {
  if (SCAM_ADDRESSES.has(address.toLowerCase())) {
    return <div className="scam-warning">    Этот адрес в списке мошенников!</div>
  }

  // Проверка через BlockSec / GoPlus API
  const { data: risk } = useQuery({
    queryKey: ['address-risk', address],
    queryFn: () => fetch(`https://api.gopluslabs.io/api/v1/token_security/1?contract_addresses=${address}`),
  })

  return null
}
```

4. Человекочитаемые описания транзакций

```
function TransactionPreview({ tx }: {tx: PreparedTx }) {
  // Парсим calldata и показываем, что именно произойдёт
  const decoded = decodeTxData(tx)

  return (
    <div className="tx-preview">
      <h3>Вы собираетесь:</h3>
      {decoded.type === 'approve' && (
        <div>
          <span className="action">Разрешить</span>
          <span className="address">{decoded.spender}</span>
          <span className="danger">
            тратить {decoded.amount} === 'unlimited' ? 'Всё ваши
токены' : decoded.amount}
          </span>
        </div>
      )}
      {decoded.type === 'swap' && (
        <div>
          <span>Обменять {decoded.amountIn} {decoded.tokenIn}</span>
          <span>на ~{decoded.amountOut} {decoded.tokenOut}</span>
        </div>
      )}
    </div>
  )
}
```

5. Рекомендации по отзыву approvals после взаимодействия

```
function PostSwapReminder() {
  const [showReminder, setShowReminder] = useState(false)

  // Показываем напоминание после свопа
  return (
    <div>
      {showReminder && (
        <div className="reminder">
          <p>Вы дали infinite approval контракту. Рекомендуем отозвать н
енужные разрешения:</p>
          <a href="https://revoke.cash" target="_blank">Revoke.cash -></a>
        </div>
      )}
    </div>
  )
}
```

Что хотят услышать

- Верификация контракта на Etherscan
- Симуляция транзакции (Tenderly)
- Проверка адресов по базам скама

26. Что такое reentrancy и как от неё защищаются?

Уровень: Middle

Развёрнутый ответ

Reentrancy — атака, при которой контракт А вызывает контракт В, а В в ответ вызывает А снова — до того как А завершил первое выполнение.

Классический пример (DAO Hack, 2016):

```
// Уязвимый контракт
contract VulnerableBank {
    mapping(address => uint) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint amount = balances[msg.sender];
        require(amount > 0);

        // Ошибка: отправляем ETH ДО обнуления баланса
        (bool sent, ) = msg.sender.call{value: amount}("");
        require(sent);

        balances[msg.sender] = 0; // срабатывает слишком поздно!
    }
}

// Контракт атакующего
contract Attacker {
    VulnerableBank bank;

    constructor(address _bank) { bank = VulnerableBank(_bank); }

    function attack() external payable {
        bank.deposit{value: 1 ether}();
        bank.withdraw();
    }

    receive() external payable {
        // Пока в банке есть ETH – продолжаем выводить!
        if (address(bank).balance >= 1 ether) {
            bank.withdraw();
        }
    }
}
}
```

Как это работает:

1. Attacker.attack() → deposit(1 ETH) → withdraw()
 2. Bank.withdraw(): проверяет баланс → 1 ETH
 3. Bank отправляет 1 ETH → Attacker.receive()
 4. Attacker.receive() → Bank.withdraw() ← ПОВТОРНЫЙ ВХОД!
 5. Bank.withdraw(): проверяет баланс → всё ещё 1 ETH (не обнулён!)
 6. Bank отправляет ещё 1 ETH...
- ...повторяется, пока не кончатся все ETH в Bank.

Защита 1: Check-Effects-Interactions (CEI)

```
function withdraw() external {
    uint amount = balances[msg.sender];
    require(amount > 0);

    // 1. Check – проверка (уже сделали)
    // 2. Effects – меняем состояние ДО внешнего вызова
    balances[msg.sender] = 0;

    // 3. Interactions – внешний вызов ПОСЛЕДНИМ
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent);
}
```

Защита 2: ReentrancyGuard (OpenZeppelin)

```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    function withdraw() external nonReentrant {
        uint amount = balances[msg.sender];
        require(amount > 0);

        balances[msg.sender] = 0;

        (bool sent, ) = msg.sender.call{value: amount}("");
        require(sent);
    }
}

// nonReentrant модификатор: при повторном входе → revert
```

Защита 3: Pull over Push

```
// Вместо отправки ETH каждому – пусть сами забирают
function withdraw(uint amount) external {
    require(balances[msg.sender] >= amount);
    balances[msg.sender] -= amount;
    payable(msg.sender).transfer(amount); // transfer ограничен 2300 gas
    // – reentrancy невозможна
}
```

Что хотят услышать

Механизм: внешний вызов до обновления состояния → повторный вход

CEI (Check-Effects-Interactions) — главная защита

ReentrancyGuard (OpenZeppelin)

transfer/send (2300 gas limit) — дополнительная защита, но менее гибкая

Исторический пример: DAO Hack (2016)

27. Почему нельзя использовать `Math.random()` или `Date.now()` для рандома в контракте?

Уровень: Middle/Senior

Развёрнутый ответ

Блокчейн **детерминирован**: все ноды должны прийти к одинаковому результату. Если контракт использует случайность — ноды не смогут договориться о состоянии.

Проблема `block.timestamp`:

```
// НЕбезопасно
function random() public view returns (uint) {
    return uint(keccak256(abi.encodePacked(block.timestamp, block.diffic
ulty)));
}
// Майнер/валидатор может манипулировать timestamp в пределах ~900
секунд!
```

Проблема `blockhash`:

```
// Всё ещё небезопасно
function random() public view returns (uint) {
    return uint(keccak256(abi.encodePacked(blockhash(block.number - 1), m
sg.sender)));
}
// Валидатор знает blockhash заранее и может решить — включать транзакцию
или нет
```

Почему это важно фронтендеру:

Когда делаешь фронтенд для лотереи, NFT-drops, гейминга — нужно понимать, что настоящий рандом на блокчейне невозможен без оракула.

Решения:

1. Chainlink VRF (Verifiable Random Function)

```
// Правильный способ: запрос рандома у Chainlink
function requestRandomWords() external returns (uint256 requestId) {
    return COORDINATOR.requestRandomWords(
        keyHash,
        s_subscriptionId,
        requestConfirmations, // ждём N блоков
        callbackGasLimit,
        numWords               // сколько случайных слов
    );
}

function fulfillRandomWords(
    uint256 requestId,
    uint256[] memory randomWords
) internal override {
    // randomWords содержат ДОКАЗУЕМО случайные числа
    uint winner = randomWords[0] % participants.length;
}
```

2. Commit-Reveal схема (без оракула)

1. Commit: все участники отправляют hash(свой_вклад + secret)
2. Reveal: все раскрывают свои вклады
3. Результат: хог всех вкладов → случайное число

Недостаток: последний раскрывший может отказаться → нужен депозит

3. Для некритичных случаев (игры с низкими ставками):

```
// Относительно приемлемо для низких ставок
uint random = uint(keccak256(abi.encodePacked(
    block.prevrandao, // случайное значение от Beacon Chain (раньше
    block.difficulty)
    block.timestamp,
    msg.sender,
    nonce++
)));
```

Что хотят услышать

Блокчейн детерминирован — настоящего рандома нет
 block.timestamp и blockhash манипулируемы валидатором
 Chainlink VRF — стандартное решение
 Commit-Reveal для децентрализованного рандома
 block.prevrandao вместо block.difficulty (после The Merge)

28. Как работает подпись сообщений (EIP-712) и зачем она нужна?

Уровень: Senior

Развёрнутый ответ

EIP-712 — стандарт типизированных подписей. Вместо подписи сырых байтов пользователь подписывает **структурированные данные**, которые MetaMask показывает в человекочитаемом виде.

Без EIP-712:

MetaMask показывает: "Sign this message? 0xabc123def456..."
Пользователь: что я подписываю? Непонятно. → фишинг-риск!

С EIP-712:

MetaMask показывает:
Domain: MyDApp
Action: Swap
Token In: 100 USDC
Token Out: 0.05 ETH
Deadline: 2026-07-19 15:30

Пользователь: ясно, подписываю!

Структура EIP-712:

```

// Контракт – верификация
contract SignatureVerifier {
    bytes32 private constant PERMIT_TYPEHASH = keccak256(
        "Permit(address owner,address spender,uint256 value,uint256
nonce,uint256 deadline)"
    );
    bytes32 private immutable DOMAIN_SEPARATOR;

    constructor() {
        DOMAIN_SEPARATOR = keccak256(abi.encode(
            keccak256("EIP712Domain(string name,string version,uint256
chainId,address verifyingContract)"),
            keccak256(bytes("MyToken")),
            keccak256(bytes("1")),
            block.chainid,
            address(this)
        ));
    }

    function verify(
        address owner, address spender, uint256 value,
        uint256 deadline, uint8 v, bytes32 r, bytes32 s
    ) internal view returns (bool) {
        bytes32 structHash = keccak256(abi.encode(
            PERMIT_TYPEHASH, owner, spender, value, nonces[owner], deadl
ine
        ));
        bytes32 digest = keccak256(abi.encodePacked("\x19\x01", DOMAIN_S
EPARATOR, structHash));
        address signer = ecrecover(digest, v, r, s);
        return signer == owner;
    }
}

```

Фронтенд — подпись EIP-712:

```
import { useSignTypedData } from 'wagmi'

function SignPermit() {
  const { signTypedDataAsync } = useSignTypedData()

  const sign = async () => {
    const signature = await signTypedDataAsync({
      domain: {
        name: 'MyDApp',
        version: '1',
        chainId: await getChainId(),
        verifyingContract: '0x...',
      },
      types: {
        Swap: [
          { name: 'tokenIn', type: 'address' },
          { name: 'tokenOut', type: 'address' },
          { name: 'amountIn', type: 'uint256' },
          { name: 'minAmountOut', type: 'uint256' },
          { name: 'deadline', type: 'uint256' },
        ],
      },
      primaryType: 'Swap',
      message: {
        tokenIn: '0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48',
        tokenOut: '0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2',
        amountIn: 100000000n, // 100 USDC
        minAmountOut: 500000000000000000n, // 0.05 ETH
        deadline: BigInt(Math.floor(Date.now() / 1000) + 1200),
      },
    })
    // signature = "0x..."
  }
}
```

Типичные use-case'ы:

- **Permit (газлесс approve)**
- **Газлесс транзакции** (пользователь подписывает, relayer платит газ)
- **Аутентификация** (доказательство владения адресом: «войти через кошелёк»)
- **Ордера (limit orders)** — пользователь подписывает офчейн, исполнитель платит газ
- **Snapshots/голосования** — подпись вместо транзакции для governance

Верификация на бэкенде (если нужно проверить подпись на сервере):


```
import { verifyTypedData } from 'viem'

const valid = await verifyTypedData({
  address: expectedSigner,
  domain: { /* ... */ },
  types: { /* ... */ },
  primaryType: 'Swap',
  message: { /* ... */ },
  signature,
})
```

Что хотят услышать

- EIP-712 = типизированные структурированные подписи
- MetaMask показывает domain + message в читаемом виде → защита от фишинга
- DOMAIN_SEPARATOR: name, version, chainId, verifyingContract
- signTypedData / signTypedDataAsync в wagmi/viem
- Use-case'ы: Permit, газлесс tx, аутентификация, ордера

Практические задачи

29. Напиши компонент: баланс ERC-20 + отправка токенов

Уровень: Middle (живое кодирование на собеседовании)

Что спрашивают на самом деле

Могут дать эту задачу на live coding. Проверяют: знаешь ли wagmi API, обрабатываешь ли все состояния, валидируешь ли ввод, форматируешь ли decimals. Ждут рабочий код за 15-20 минут.


```

import { useState } from 'react'
import {
  useAccount,
  useReadContract,
  useWriteContract,
  useWaitForTransactionReceipt,
} from 'wagmi'
import { erc20Abi } from 'viem'
import { formatUnits, parseUnits, isAddress } from 'viem'

// Предположим, что decimals известны и токен стандартный ERC-20
const TOKEN_ADDRESS = '0x1234...' as const
const TOKEN_DECIMALS = 18
const TOKEN_SYMBOL = 'TKN'

export function TokenTransfer() {
  const { address, isConnected } = useAccount()
  const [recipient, setRecipient] = useState('')
  const [amount, setAmount] = useState('')
  const [error, setError] = useState<string | null>(null)

  // Чтение баланса
  const { data: balance, refetch: refetchBalance } = useReadContract({
    address: TOKEN_ADDRESS,
    abi: erc20Abi,
    functionName: 'balanceOf',
    args: address ? [address] : undefined,
    query: { enabled: Boolean(address) },
  })

  // Отправка
  const {
    writeContract,
    data: txHash,
    isPending: isWalletPending,
  } = useWriteContract()

  const {
    isLoading: isConfirming,
    isSuccess: isConfirmed,
  } = useWaitForTransactionReceipt({ hash: txHash })

  const handleSend = () => {
    setError(null)

    // Валидация
    if (!isAddress(recipient)) {
      setError('Некорректный адрес получателя')
      return
    }
  }

```

```

const parsedAmount = parseUnits(amount, TOKEN_DECIMALS)
if (parsedAmount <= 0n) {
  setError('Сумма должна быть больше 0')
  return
}

if (balance !== undefined && parsedAmount > balance) {
  setError('Недостаточно токенов')
  return
}

writeContract({
  address: TOKEN_ADDRESS,
  abi: erc20Abi,
  functionName: 'transfer',
  args: [recipient as `0x${string}`, parsedAmount],
})
}

// После успешной отправки – обновляем баланс
if (isConfirmed) {
  // В реальном коде: queryClient.invalidateQueries
  refetchBalance()
}

// Состояния кнопки
const buttonText = (() => {
  if (!isConnected) return 'Подключите кошелёк'
  if (isWalletPending) return 'Подтвердите в кошельке...'
  if (isConfirming) return 'Отправляется...'
  if (isConfirmed) return 'Отправлено!'
  return 'Отправить'
})()

const isDisabled = !isConnected || isWalletPending || isConfirming

const formattedBalance = balance !== undefined
  ? Number(formatUnits(balance, TOKEN_DECIMALS)).toLocaleString(undefi
ned, {
    maximumFractionDigits: 4,
  })
  : '-'

return (
  <div className="token-transfer">
    <h3>Отправка {TOKEN_SYMBOL}</h3>

    <div className="balance">
      Баланс: {formattedBalance} {TOKEN_SYMBOL}

```

```

</div>

<div className="form">
  <input
    type="text"
    placeholder="0x..."
    value={recipient}
    onChange={e => setRecipient(e.target.value)}
    disabled={isDisabled}
  />

  <input
    type="number"
    placeholder="0.0"
    value={amount}
    onChange={e => setAmount(e.target.value)}
    disabled={isDisabled}
    min="0"
    step="any"
  />
  <button
    onClick={() => setAmount(formatUnits(balance ?? 0n, TOKEN_DECIMALS))}
    disabled={isDisabled}
  >
    MAX
  </button>

  <button onClick={handleSend} disabled={isDisabled}>
    {buttonText}
  </button>
</div>

{error && <div className="error">{error}</div>}
{txHash && (
  <div className="tx-link">
    Tx: <a href={`https://etherscan.io/tx/${txHash}`} target="_blank">
      {txHash.slice(0, 10)}...{txHash.slice(-8)}
    </a>
  </div>
)}
</div>
)
}

```

Что хотят увидеть

Правильное использование useReadContract и useWriteContract
formatUnits / parseUnits для работы с decimals

Валидация: адрес, сумма > 0, не больше баланса

Все состояния UI: загрузка, pending, подтверждение, успех, ошибка

MAX-кнопка для удобства

Ссылка на Etherscan

Обработка разных состояний кнопки

30. Как реализовать «Connect Wallet» с нуля без RainbowKit?

Уровень: Middle

Развёрнутый ответ

Иногда нужно кастомное решение (кастомный дизайн, специфичные требования). Показываем, что понимаем, что под капотом.

```

import { useConnect, useAccount, useDisconnect, useBalance, useSwitchChain } from 'wagmi'
import { injected, walletConnect } from 'wagmi/connectors'
import { mainnet, polygon, arbitrum } from 'wagmi/chains'

export function CustomConnectButton() {
  const { connect, connectors, isPending } = useConnect()
  const { address, isConnected, chainId } = useAccount()
  const { disconnect } = useDisconnect()
  const { data: balance } = useBalance({ address })
  const { switchChain } = useSwitchChain()
  const [isOpen, setIsOpen] = useState(false)

  // Отключён – показываем кнопку connect
  if (!isConnected) {
    return (
      <div className="custom-connect">
        <button onClick={() => setIsOpen(!isOpen)} className="connect-trigger">
          Connect Wallet
        </button>

        {isOpen && (
          <div className="connect-modal">
            <h3>Выберите кошелёк</h3>

            { /* MetaMask / Injected */ }
            <button
              onClick={() => {
                connect({ connector: injected() })
                setIsOpen(false)
              }}
              disabled={isPending}
            >
              
              MetaMask
            </button>

            { /* WalletConnect */ }
            <button
              onClick={() => {
                connect({
                  connector: walletConnect({
                    projectId: 'YOUR_PROJECT_ID',
                    showQrModal: true, // WalletConnect сам покажет QR
                  }),
                })
                setIsOpen(false)
              }}
              disabled={isPending}
            >
              
              WalletConnect
            </button>
          </div>
        ) }
      </div>
    )
  }
}

```

```

    >
    
    WalletConnect
  </button>

  {/* Можно добавить Coinbase Wallet, Rabby и т.д. */}
</div>
  })
</div>
)
}

// Подключён — показываем информацию
return (
  <div className="custom-account">
    <div className="network-badge">
      {chainId === mainnet.id && 'Ethereum'}
      {chainId === polygon.id && 'Polygon'}
      {chainId === arbitrum.id && 'Arbitrum'}
    </div>

    <div className="balance">
      {balance ? `${Number(balance.formatted).toFixed(3)} ${balance.symbol}` : '...'}
    </div>

    <button onClick={() => setIsOpen(!isOpen)} className="address-btn">
      {address?.slice(0, 6)}...{address?.slice(-4)}
    </button>

    {isOpen && (
      <div className="account-dropdown">
        <div className="account-address">{address}</div>

        <div className="network-switcher">
          <h4>Сеть</h4>
          {[mainnet, polygon, arbitrum].map(chain => (
            <button
              key={chain.id}
              onClick={() => switchChain({ chainId: chain.id })}
              className={chainId === chain.id ? 'active' : ''}
            >
              {chain.name}
            </button>
          ))}
        </div>

        <button onClick={() => disconnect()} className="disconnect-
btn">
          Disconnect

```



```
        </button>
      </div>
    )}
  </div>
)
```

Что нужно помнить:

- injected() — ищет window.ethereum (MetaMask, Rabby, Brave)
- walletConnect() — для мобильных кошельков, требует projectId
- Состояние подключения сохраняется в localStorage автоматически (wagmi)
- useSwitchChain — смена сети (пользователь может отклонить)
- Авто-реконнект при загрузке страницы — wagmi делает сам

Что хотят услышать

useConnect, useAccount, useDisconnect из wagmi
injected() — MetaMask и подобные
walletConnect() — для мобильных
Dropdown с сетями и сменой через useSwitchChain
Баланс ETH через useBalance

31. Как сделать optimistic UI при отправке транзакции?

Уровень: Senior

Развёрнутый ответ

Optimistic UI — показываем результат сразу, не дожидаясь подтверждения блокчейна. Если транзакция сфейлилась — откатываем.

Почему это важно: 12 секунд на блок + финальность ~13 минут.
Пользователь не должен ждать.

```

function OptimisticStake() {
  const [stakedAmount, setStakedAmount] = useState(0n) // реальный
баланс из контракта
  const [optimisticAmount, setOptimisticAmount] = useState<bigint | null>
(null) // UI-состояние
  const [rollbackTimer, setRollbackTimer] = useState<NodeJS.Timeout | nu
ll>(null)

  const { writeContractAsync } = useWriteContract()
  const queryClient = useQueryClient()

  const displayAmount = optimisticAmount ?? stakedAmount // показываем
оптимистичное!

  const stake = async (amount: bigint) => {
    // 1. Оптимистичное обновление
    setOptimisticAmount(stakedAmount + amount)
    setError(null)

    try {
      // 2. Отправляем транзакцию
      const hash = await writeContractAsync({
        address: stakingAddress,
        abi: stakingAbi,
        functionName: 'stake',
        args: [amount],
      })

      // 3. Ждём подтверждения
      const receipt = await publicClient.waitForTransactionReceipt({
hash })

      if (receipt.status === 'success') {
        // 4. Подтверждено – инвалидируем кеш, получаем реальные данные
        await queryClient.invalidateQueries({ queryKey:
['stakedBalance'] })
        // wagmi перечисляет balance → realAmount обновится
        setOptimisticAmount(null) // убираем оптимистичное состояние
      } else {
        // 5. Revert – откатываем
        setOptimisticAmount(null)
        setError('Транзакция не удалась. Попробуйте снова.')
      }
    } catch (err) {
      // 6. Ошибка отправки или отклонение – откатываем
      setOptimisticAmount(null)
      if (err?.message?.includes('rejected')) {
        setError('Вы отклонили транзакцию')
      } else {
        setError('Ошибка при отправке')
      }
    }
  }
}

```

```

    }
  }
}

return (
  <div>
    <div className="staked-amount">
      {/* Визуальный индикатор optimistic state */}
      Застейкано: {formatEther(displayAmount)} LP
      {optimisticAmount !== null && (
        <span className="optimistic-badge">(ожидает подтверждения)</sp
an>
        )}
    </div>

    <button onClick={() => stake(parseEther('100'))}>
      Stake 100 LP
    </button>

    {error && <div className="error">{error}</div>}
  </div>
)
}

```

Более продвинутый вариант — очередь optimistic-транзакций:

```

interface OptimisticAction {
  id: string
  type: 'stake' | 'unstake' | 'claim'
  delta: bigint // изменение баланса
  txHash?: string // появится после отправки
  status: 'optimistic' | 'pending' | 'confirmed' | 'failed'
}

function useOptimisticBalance(contractBalance: bigint) {
  const [actions, setActions] = useState<OptimisticAction[]>([])

  // Результирующий баланс = реальный + сумма optimistic deltas
  const displayBalance = actions
    .filter(a => a.status !== 'failed')
    .reduce((sum, a) => sum + a.delta, contractBalance)

  const addOptimistic = (action: OptimisticAction) => {
    setActions(prev => [...prev, action])
  }

  const confirmAction = (id: string, txHash: string) => {
    setActions(prev => prev.map(a => a.id === id ? { ...a, txHash,
status: 'pending' } : a))
  }

  const resolveAction = (id: string, success: boolean) => {
    if (success) {
      // Удаляем из optimistic (реальный баланс уже обновлён через
инвалидацию)
      setActions(prev => prev.filter(a => a.id !== id))
    } else {
      setActions(prev => prev.map(a => a.id === id ? { ...a, status: 'fa
iled', delta: 0n } : a))
    }
  }

  return { displayBalance, actions, addOptimistic, confirmAction, resolv
eAction }
}

```

Когда НЕ стоит использовать optimistic UI:

- Крупные суммы (>\$10K) — пользователь должен видеть реальное подтверждение
- Необратимые действия (например, отправка на внешний адрес)
- Если высокая вероятность revert (низкая ликвидность, высокий slippage)

Что хотят услышать

- Концепция: показали результат → подтвердили → убрали optimistic Откат при ошибке/revert
 - Очередь optimistic-экшенов (мульти-транзакции)
 - Когда НЕ использовать optimistic UI (крупные суммы, необратимые действия)
 - Инвалидация React Query после подтверждения
-

32. Подпиши сообщение и верифицируй на бэкенде (EIP-191 + EIP-1271)

Уровень: Middle/Senior

Развёрнутый ответ

EIP-191 (Personal Sign) — стандарт подписи произвольных сообщений. MetaMask показывает: «Sign this message?» + само сообщение.

```

import { useSignMessage } from 'wagmi'
import { verifyMessage } from 'viem'

function SignAndVerify() {
  const { signMessageAsync } = useSignMessage()
  const [signature, setSignature] = useState<string | null>(null)
  const [verified, setVerified] = useState<boolean | null>(null)

  const sign = async () => {
    const sig = await signMessageAsync({
      message: 'Я подтверждаю вход в MyDApp',
    })
    setSignature(sig)

    // Верификация на клиенте (для мгновенной обратной связи)
    const isValid = await verifyMessage({
      address: userAddress!,
      message: 'Я подтверждаю вход в MyDApp',
      signature: sig,
    })
    setVerified(isValid)
  }

  return (
    <div>
      <button onClick={sign}>Подписать</button>
      {signature && <div>Signature: {signature}</div>}
      {verified !== null && (
        <div>{verified ? 'Подпись валидна' : 'Подпись невалидна'}</div>
      )}
    </div>
  )
}

```

Что происходит под капотом EIP-191:

Личное сообщение: "Я подтверждаю вход в MyDApp"

1. Добавляется префикс: "\x19Ethereum Signed Message:\n" + длина сообщения
→ "\x19Ethereum Signed Message:\n29Я подтверждаю вход в MyDApp"
2. Хешируется: кессак256(префикс + сообщение)
3. Подписывается приватным ключом (через MetaMask)
4. Подпись: r, s, v → 65 байт hex

Верификация на бэкенде (Node.js):

```
import { verifyMessage } from 'viem' // работает и в Node.js!

async function verifyAuth(message: string, signature: string, expectedAddress: string) {
  const recoveredAddress = await verifyMessage({
    address: expectedAddress, // ожидаемый адрес
    message,
    signature: signature as `0x${string}`,
  })

  return recoveredAddress // true/false
}
```

EIP-1271 — подписи от смарт-контрактов:

Проблема: контракт не имеет приватного ключа → не может подписать EIP-191.

Но контракт может верифицировать подпись ПО-СВОЕМУ.

EIP-1271: у контракта должна быть функция:

isValidSignature(bytes32 hash, bytes memory signature) returns (bytes4 magicValue)

Если возвращает 0x1626ba7e → подпись валидна.

```
contract MultisigWallet {
  function isValidSignature(bytes32 hash, bytes calldata signature)
    external view returns (bytes4)
  {
    // Проверяем, что подписали минимум 2 из 3 владельцев
    address[] memory signers = recoverSigners(hash, signature);
    uint validCount;
    for (uint i = 0; i < signers.length; i++) {
      if (isOwner[signers[i]]) validCount++;
    }
    if (validCount >= 2) return 0x1626ba7e; // EIP-1271 magic value
    return 0xffffffff;
  }
}
```

На фронте — проверка EIP-1271:

```
import { isErc6492Signature } from 'viem'

// Проверка с учётом EIP-1271 (контрактные кошельки!)
const isValid = await verifyMessage({
  address: smartWalletAddress, // может быть контрактом!
  message: 'Hello',
  signature, // может содержать ERC-6492 обёртку
})
// viem автоматически проверит EIP-1271, если address — контракт
```

Что хотят услышать

EIP-191: префикс \x19Ethereum Signed Message:\n<length><message>

useSignMessage / signMessage → MetaMask → verifyMessage

Верификация на клиенте и на бэкенде

EIP-1271: контракты тоже могут «подписывать» через isValidSignature

Use-case'ы: аутентификация, газлесс-транзакции, off-chain ордера

Чек-лист для подготовки

Перед собеседованием:

- [] Можешь написать компонент ConnectWallet + TokenTransfer без подсказок
- [] Понимаешь жизненный цикл транзакции (5 состояний) и можешь нарисовать схему
- [] Знаешь ERC-20 и ERC-721 интерфейсы наизусть (основные функции)
- [] Можешь объяснить Uniswap: АММ, формула, slippage
- [] Знаешь 3+ уязвимости смарт-контрактов
- [] Можешь написать approve + transfer / swap flow
- [] Понимаешь, зачем нужен Subgraph
- [] Знаешь разницу между viem, wagmi, ethers.js

Типичные практические задачи на собеседовании:

1. «Покажи баланс токена» — useReadContract + форматирование
 2. «Отправь токены» — useWriteContract + состояния + ошибки
 3. «Сделай своп» — approve + swap, два шага
 4. «Подпиши сообщение и проверь» — sign + verify
 5. «Слушай события контракта» — watchEvent
 6. «Расшифруй revert-ошибку» — decodeErrorResult
-

Связанное

- [wiki/web3-фронтендер-план-трудоустройства](#) — полный план подготовки к web3-собеседованию
- [wiki/DeFi-для-фронтендера](#) — DeFi-концепции: Uniswap, Aave, стейкинг
- [wiki/wagmi-RainbowKit-фронтенд](#) — полный гайд по wagmi + RainbowKit
- [wiki/Паттерны-транзакций-React](#) — продвинутая работа с транзакциями
- [wiki/Словарь-web3](#) — глоссарий web3-терминов

- [wiki/Блокчейн-как-это-работает](#) — фундаментальные основы блокчейна
- [wiki/Solidity-основы](#) — Solidity для фронтендеров
- [wiki/ERC-20-стандарт-токенов](#) — стандарт ERC-20
- [wiki/ERC-721-NFT-стандарт](#) — стандарт ERC-721
- [wiki/Сравнение-ethers-viem-wagmi](#) — сравнение библиотек
- [wiki/OpenZeppelin-безопасные-контракты](#) — безопасность смарт-контрактов
- [wiki/Subgraph-The-Graph](#) — индексация ончейн-данных

GitHub Commit Notary (пет-проект)

GitHub Commit Notary

Нотаризация коммитов через Ethereum — докажи, что ты написал код первым.

Проблема

Через несколько лет сложно доказать: «Я написал этот код первым». Ни git-даты (можно подделать), ни репозиторий (можно удалить) не дают надёжного доказательства авторства и времени.

Решение

CLI-инструмент, который:

```
commit → hash → Ethereum-транзакция → timestamp
```

1. Разработчик пишет код, делает коммит
2. CLI берёт хеш коммита (SHA-256)
3. Отправляет транзакцию в Ethereum с хешем в calldata
4. Блокчейн сохраняет: **чей адрес, какой хеш, когда**

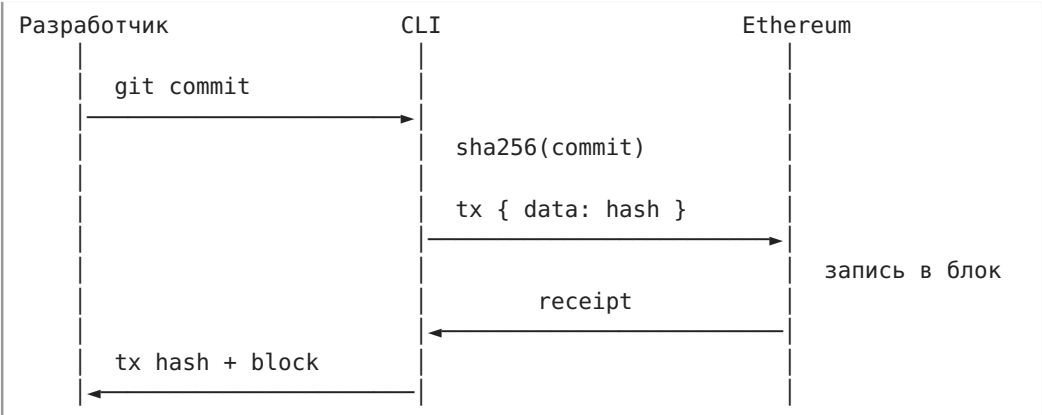
Потом можно показать:

- «Я владел этим кодом»
- «Вот хеш»
- «Вот дата в блокчейне»

Никто не может подделать timestamp в блокчейне задним числом.

Как читать tx { data: hash }: «отправь транзакцию в Ethereum, положи хеш коммита в поле *data* (calldata) — блокчейн навсегда запомнит: чей адрес, какой хеш, когда. Сам перевод нулевой, платишь только газ». Мнемоника: calldata = бесплатный нотариальный журнал внутри блокчейна; пишу хеш, плачу газ, получаю вечный timestamp.

Как работает



Ключевые технологии

- **Криптография:** SHA-256, понимание хеш-функций
- **Ethereum-транзакции:** calldata, gas, nonce
- **CLI на Node.js/TypeScript:** Commander.js, ethers.js/viem
- **Верификация:** любой может проверить хеш → транзакцию через Etherscan

Почему это хороший проект для портфолио

- Относительно простой старт — один смарт-контракт не нужен
- Показывает понимание криптографии и блокчейна
- Решает реальную проблему (доказательство авторства)
- Демонстрирует навыки CLI-разработки + web3
- Можно расширить: верификация через веб-интерфейс, батчинг коммитов

Стек

Слой	Технология
CLI	Node.js + TypeScript + Commander.js
Блокчейн	Ethereum (Sepolia testnet → mainnet)
Библиотека	viem (легче ethers.js, TypeScript-first)
Верификация	Веб-интерфейс на React (позже)

Связанное

- [wiki/Блокчейн-как-это-работает](#)
- [wiki/Сравнение-ethers-viem-wagmi](#)

Proof of Skill

Сервис децентрализованных сертификатов — твои навыки, подтверждённые и неизменяемые.

Проблема

Любой может написать в резюме «Senior React», «Solidity Developer», «AWS Certified» — но проверить эти утверждения сложно. Бумажные сертификаты теряются, ссылки на курсы протухают, а рекомендации в LinkedIn ни к чему не обязывают.

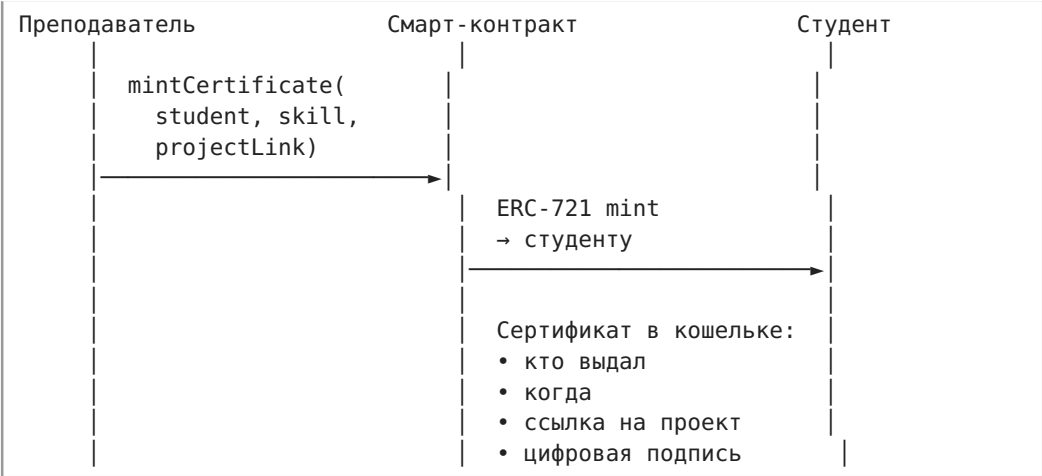
Решение

Сервис, где преподаватели, работодатели или менторы **подписывают сертификат своим кошельком**. Сертификат хранится в адресе пользователя — никто не может его удалить или подделать.

Wallet: 0x1234...abcd

React	— выдан: Ivanov.eth, 2024-03-15, проект: github.com/...
Solidity	— выдан: CryptoAcademy.eth, 2024-06-01, курс: SC-101
Docker	— выдан: DevOpsGuru.eth, 2024-01-10
AWS	— выдан: CloudCert.eth, 2023-11-22

Как работает



Структура сертификата

Каждый сертификат — это NFT (ERC-721) с метаданными на IPFS:

Как читать `mintCertificate(student, skill, projectLink)`:

«преподаватель вызывает функцию контракта: "выдаю сертификат студенту `student` за навык `skill`, доказательство — ссылка `projectLink`". Контракт чеканит Soulbound NFT на адрес студента с подписью эмитента в метаданных». Мнемоника: `mintCertificate` = цифровая печать преподавателя в блокчейне — один вызов, вечный сертификат.

```
{
  "name": "Proof of Skill: Solidity",
  "issuer": "0xIssuerAddress",
  "recipient": "0xStudentAddress",
  "skill": "Solidity",
  "issuedAt": "2024-06-01T00:00:00Z",
  "projectLink": "https://github.com/student/smart-contract-project",
  "signature": "0x..."
}
```

Почему это интересно

- **DID (Decentralized Identifiers)** — идентификация через адрес кошелька
- **Цифровые подписи** — эмитент подписывает сертификат своим ключом
- **NFT не торговые** — Soulbound-токены, которые нельзя передать

Как читать Soulbound NFT (SBT): «это NFT, у которого метод *transfer* намеренно заблокирован — токен навсегда привязан к кошельку получателя. В коде это выглядит как `_beforeTokenTransfer` с `revert("Soulbound: non-transferable")` — в момент любой попытки перевода контракт говорит НЕТ». Мнемоника: Soulbound = токен-паспорт, приклеен к адресу — выдать можно, продать/подарить нельзя.

- **IPFS** — децентрализованное хранение метаданных сертификата
- **Верификация** — любой может проверить подпись и issuer on-chain

Потенциал развития

- Интеграция с LinkedIn / GitHub профилем
- Верификация через ZK (доказать наличие сертификата, не раскрывая адрес)
- Репутационная система на основе сертификатов
- Рынок труда с верифицированными навыками

Стек

Слой	Технология
Смарт-контракт	Solidity + ERC-721 (Soulbound) + OpenZeppelin
Метаданные	IPFS (Pinata / Web3.Storage)
Фронтенд	React + TypeScript + wagmi + RainbowKit
Подписи	EIP-712 (Typed Structured Data)
Деплой	Hardhat + Sepolia testnet

Связанное

- [wiki/Блокчейн-как-это-работает](#)
- [wiki/Словарь-web3](#)

Open Source Sponsor Escrow (пет-проект)

Open Source Sponsor Escrow

Эскроу-сервис для опенсорса: деньги выплачиваются только после merge PR.

Проблема

Разработчик обещает: «Сделаю фичу за \$100». Спонсор переводит деньги заранее. Если разработчик исчезает — денег нет, фичи нет. Нет механизма, который защищает обе стороны.

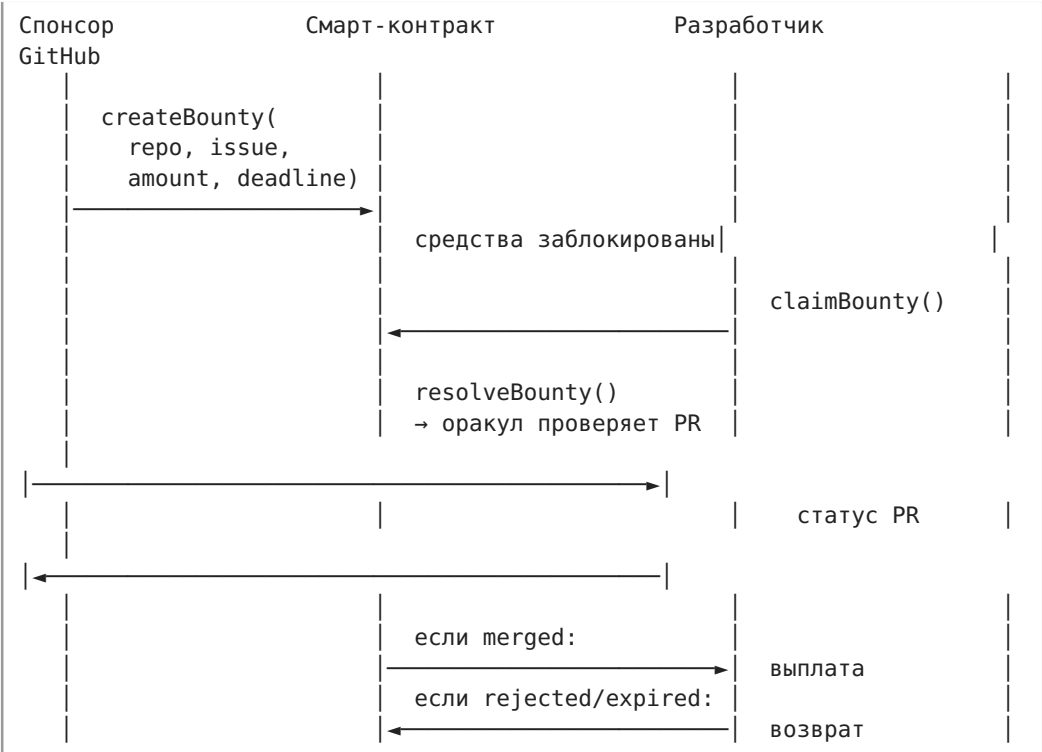
Решение

Смарт-контракт эскроу с интеграцией GitHub:

Sponsor → Smart Contract → Developer → Merge PR → Автоматическая выплата

1. Спонсор создаёт задачу и вносит средства в смарт-контракт
2. Разработчик берёт задачу, пишет код, открывает PR
3. Контракт проверяет статус PR через GitHub API (офчейн-оракул)
4. PR смержен → автоматическая выплата разработчику
5. PR отклонён / просрочен → возврат спонсору

Как работает



Ключевые механики

- **Эскроу:** средства заблокированы в контракте, ни одна сторона не может вывести единолично
- **Как читать `createBounty(repo, issue, amount, deadline)`:** «спонсор говорит контракту: "блокирую amount ETH за решение issue в репозитории hero, крайний срок deadline". Контракт запирает деньги — ни спонсор, ни разработчик не могут забрать их в одиночку». Мнемоника: `createBounty` = депозит в сейф с двумя ключами — открывается только когда оракул подтвердит merge PR.
- **Как читать `resolveBounty()` → оракул → выплата/возврат:** «контракт спрашивает оракула: "PR смержен?" — если да, деньги уходят разработчику; если нет и дедлайн прошёл — возвращаются спонсору». Мнемоника: `resolveBounty` = судья смотрит на GitHub и решает, кто получит деньги.
- **Оракул:** проверка статуса PR через GitHub API (начальный вариант — централизованный оракул, конечный — Chainlink Functions)
- **Дедлайн:** если PR не смержен за N дней, спонсор может запросить возврат

- **Дисконтные споры:** механизм разрешения конфликтов (спонсор и разработчик не согласны)

Почему это сильный проект для портфолио

- **Смарт-контракты:** нетривиальная логика эскроу, обработка краевых случаев
- **Безопасность:** защита от reentrancy, проверка прав доступа, правильная работа с ETH
- **Интеграция с внешним API:** GitHub API через оракул — мост между web2 и web3
- **Реальные сценарии:** проблема опенсорс-спонсорства существует здесь и сейчас
- **Платёжная логика:** работа с ETH, расчёт комиссий, возвраты

План развития

Версия	Что входит
v0.1	Базовый эскроу: создать задачу, принять, смержить (ручной resolve)
v0.2	GitHub-оракул: автоматическая проверка статуса PR
v0.3	Децентрализованный оракул (Chainlink Functions)
v0.4	Веб-интерфейс: список задач, профиль, история
v1.0	Мульти-чейн: Arbitrum, Base, Optimism

Стек

Слой	Технология
Смарт-контракты	Solidity + Hardhat + OpenZeppelin
Оракул (v0.2)	Node.js севвер с GitHub API
Оракул (v0.3)	Chainlink Functions
Фронтенд	React + TypeScript + wagmi + RainbowKit
Блокчейн	Ethereum (Sepolia → mainnet/Arbitrum)
Тестирование	Hardhat tests + форки mainnet

Связанное

- [wiki/Блокчейн-как-это-работает](#)
- [wiki/Словарь-web3](#)
- [wiki/Сравнение-ethers-viem-wagmi](#)